

AWS VPC PEERING

Same Region · Different Region · Different Account

Concepts, Architecture and Hands-on Lab Guide

Table of Contents

- 1. Introduction to VPC Peering
- 2. Part A — VPC Peering within the Same Region
- 3. Part B — VPC Peering across Different Regions
- 4. Part C — VPC Peering across Different AWS Accounts
- 5. Comparison Summary
- 6. Troubleshooting Checklist

1. Introduction to VPC Peering

1.1 What is a VPC?

An Amazon Virtual Private Cloud (VPC) is a logically isolated section of the AWS Cloud where a user can launch AWS resources such as EC2 instances, RDS databases, and Lambda functions inside a private, user-defined network. Each VPC has its own IP address range (CIDR block), subnets, route tables, and security controls.

1.2 What is VPC Peering?

VPC Peering is a networking connection between two VPCs that enables instances in either VPC to communicate with each other privately, using private IPv4 or IPv6 addresses, as if they were part of the same network. Traffic between peered VPCs stays entirely within the AWS backbone network and never traverses the public internet.

A VPC peering connection is not a gateway or a separate physical device — it is simply a networking construct that AWS uses to route traffic between the two VPCs using their existing infrastructure. Because of this, peering connections are not a single point of failure and do not create a bandwidth bottleneck.

1.3 Why use VPC Peering?

- Securely share resources (databases, internal APIs, file shares) between two VPCs without exposing them to the public internet.
- Connect VPCs belonging to different teams, projects, environments (dev/test/prod), or business units.
- Connect VPCs across regions for disaster recovery, data replication, or serving users closer to their location.
- Connect VPCs owned by different AWS accounts, e.g. a shared-services account and multiple application accounts, or two companies collaborating.

1.4 Types of VPC Peering

Three scenarios are covered in this guide:

- Same-Region Peering — both VPCs are in the same AWS Region and the same account.
- Different-Region (Inter-Region) Peering — the two VPCs are in different AWS Regions.
- Different-Account (Cross-Account) Peering — the two VPCs belong to two different AWS accounts (optionally also in different regions).

1.5 Key Characteristics and Limitations

- CIDR blocks of the two VPCs must NOT overlap — peering cannot be created between VPCs with overlapping or identical IP ranges.

- VPC Peering is non-transitive — if VPC A is peered with VPC B, and VPC B is peered with VPC C, VPC A cannot automatically reach VPC C through B. A separate peering connection (or a transit gateway) is required.
- Edge-to-edge routing is not supported — a VPC in a peering relationship cannot use the peer VPC's Internet Gateway, NAT Gateway, VPN, or Direct Connect connection to reach the internet or on-premises networks.
- Route tables must be updated manually on both sides of the connection — creating/accepting the peering connection alone does not enable traffic flow.
- Security Groups and Network ACLs on both sides must explicitly allow the desired traffic from the peer VPC's CIDR block.
- Inter-region peering traffic is automatically encrypted at the physical layer and incurs standard inter-region data-transfer charges.
- Cross-account peering requires the peer AWS Account ID, and the request must be manually accepted by an authorized user in that account.
- DNS resolution across the peering connection can optionally be enabled so that private DNS hostnames resolve to private IP addresses.

1.6 General Workflow (applies to all three scenarios)

- Step 1: Create/identify the two VPCs with non-overlapping CIDR blocks.
- Step 2: Create the VPC Peering Connection request from the requester VPC (specifying the acceptor VPC, and account/region if applicable).
- Step 3: Accept the peering connection request from the acceptor side.
- Step 4: Update the route tables on both sides, adding a route to the peer VPC's CIDR block with the peering connection (pcx-xxxxxxx) as the target.
- Step 5: Update Security Groups / NACLs to permit the required traffic.
- Step 6: Test connectivity between resources in the two VPCs (e.g. ping, SSH, RDP).

2. Part A — VPC Peering within the Same Region

2.1 Theory

Same-region VPC peering connects two VPCs that reside in the same AWS Region, typically (but not necessarily) within the same AWS account. It is the simplest form of peering to configure since both VPCs are visible in the same regional console view, and no cross-account or cross-region acceptance steps are required.

Traffic between the peered VPCs stays on the AWS internal network, giving low latency and no data-transfer charges beyond standard intra-region rates.

2.2 Lab Topology

- VPC "myvpc1" — CIDR 192.168.1.0/24 — contains a public subnet (192.168.1.0/25) with an EC2 instance "publicins" (Windows), reachable from the internet via an Internet Gateway.
- VPC "myvpc2" — CIDR 192.168.2.0/24 — contains a subnet (192.168.2.0/25) with an EC2 instance "privateins" (Windows), used to test private connectivity.
- A VPC peering connection "vpc1-vpc2" is created between the two VPCs, then accepted and wired into both route tables so the two instances can reach each other over their private IP addresses.

2.3 Steps Performed

1. Create the two VPCs (myvpc1 and myvpc2) with non-overlapping CIDR blocks.
2. Create a subnet inside each VPC.
3. Create a route table for the public VPC (myvpc1) and associate it with its subnet.
4. Create an Internet Gateway, attach it to myvpc1, and add a 0.0.0.0/0 route in its route table pointing to the Internet Gateway.
5. Launch an EC2 instance in myvpc1's public subnet with a security group allowing RDP (3389) and ICMP.
6. Launch a second EC2 instance in myvpc2 with a security group allowing RDP (3389) and ICMP.
7. Create a VPC Peering Connection: select myvpc1 as the requester, choose "This Region", and select myvpc2 as the acceptor VPC.
8. Go to Peering Connections, select the pending request, and choose Actions → Accept Request.
9. Create a route table for myvpc2 and associate it with its subnet.
10. Edit the route table of myvpc2 to add a route for 192.168.1.0/24 (myvpc1's CIDR) targeting the Peering Connection.
11. Edit the route table of myvpc1 to add a route for 192.168.2.0/24 (myvpc2's CIDR) targeting the same Peering Connection.
12. Verify the peering connection status shows "Active", then RDP into an instance and test connectivity to the peer instance's private IP.

2.4 Screenshots

Creating the VPCs

us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#CreateVpc:createMode=vpcOnly

Search [Alt+S]

United States (N. Virginia) trushti11 (663972508681) trushti11

VPC > Your VPCs > Create VPC

Resources to create Info
Create only the VPC resource or the VPC and other networking resources.

☒ VPC only ☐ VPC and more

Name tag - optional
Creates a tag with a key of 'Name' and a value that you specify.

myvpc2

IPv4 CIDR block Info
☒ IPv4 CIDR manual input
☐ IPAM-allocated IPv4 CIDR block

IPv4 CIDR
192.168.2.0/24
CIDR block size must be between /16 and /28.

IPv6 CIDR block Info
☒ No IPv6 CIDR block
☐ IPAM-allocated IPv6 CIDR block
☐ Amazon-provided IPv6 CIDR block
☐ IPv6 CIDR owned by me

Tenancy Info
Default

CloudShell Agent Toolkit for AWS Feedback Console mobile app © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Type here to search 31°C ENG IN 11:38 PM 9/7/2026

Figure A.1 — AWS Console

Creating subnets

us-east-1.console.aws.amazon.com/vpcconsole/home?region=us-east-1#CreateSubnet:VpcId=vpc-0d135d96e54ac7ca4

Search [Alt+S]

United States (N. Virginia) trushti11 (663972508681) trushti11

VPC > Subnets > Create subnet

vpc-0d135d96e54ac7ca4 (myvpc1)

Associated VPC CIDRs

IPv4 CIDRs
192.168.1.0/24

Subnet settings
Specify the CIDR blocks and Availability Zone for the subnet.

Subnet 1 of 1

Subnet name
Create a tag with a key of 'Name' and a value that you specify.

public_subnet
The name can be up to 256 characters long.

Availability Zone Info
Choose the zone in which your subnet will reside, or let Amazon choose one for you.

No preference

CloudShell Agent Toolkit for AWS Feedback Console mobile app © 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Type here to search 31°C ENG IN 11:39 PM 9/7/2026

Figure A.1 — AWS Console

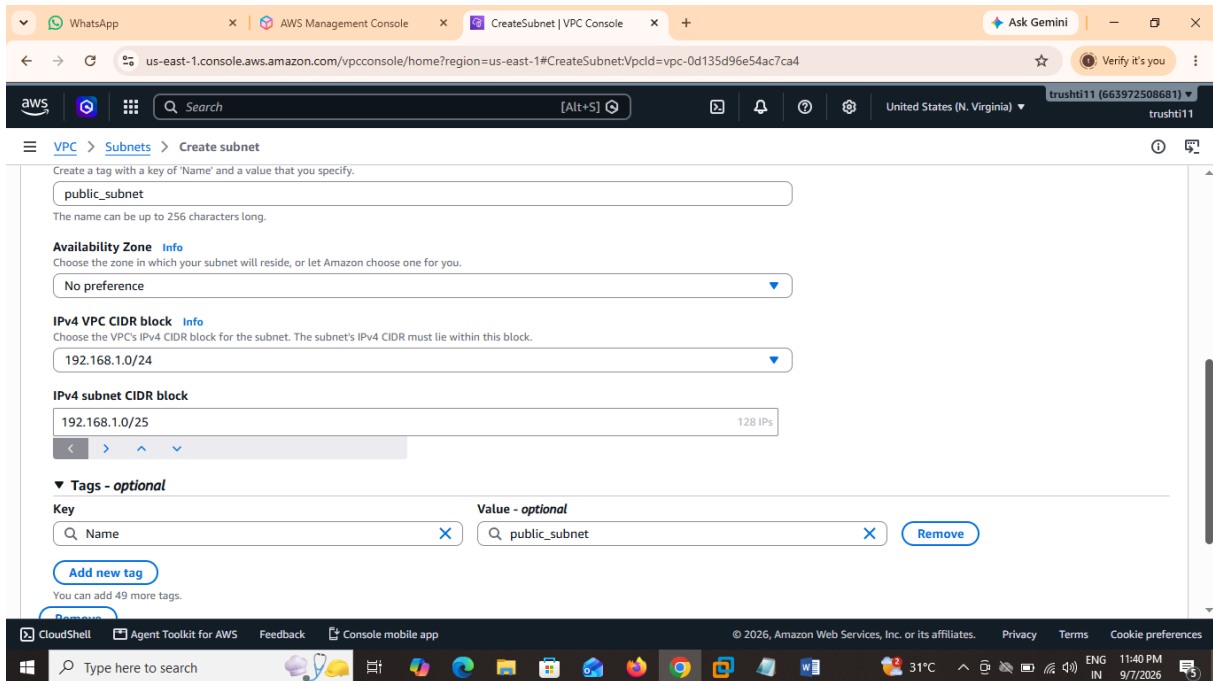


Figure A.2 — AWS Console

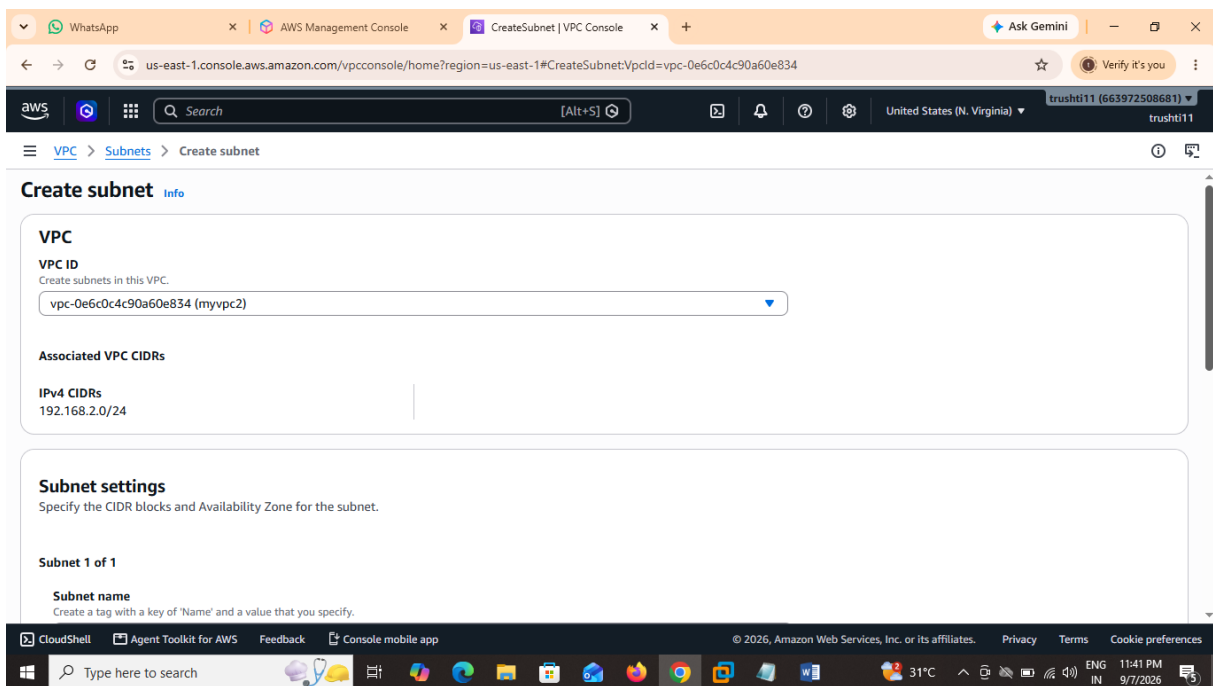


Figure A.3 — AWS Console

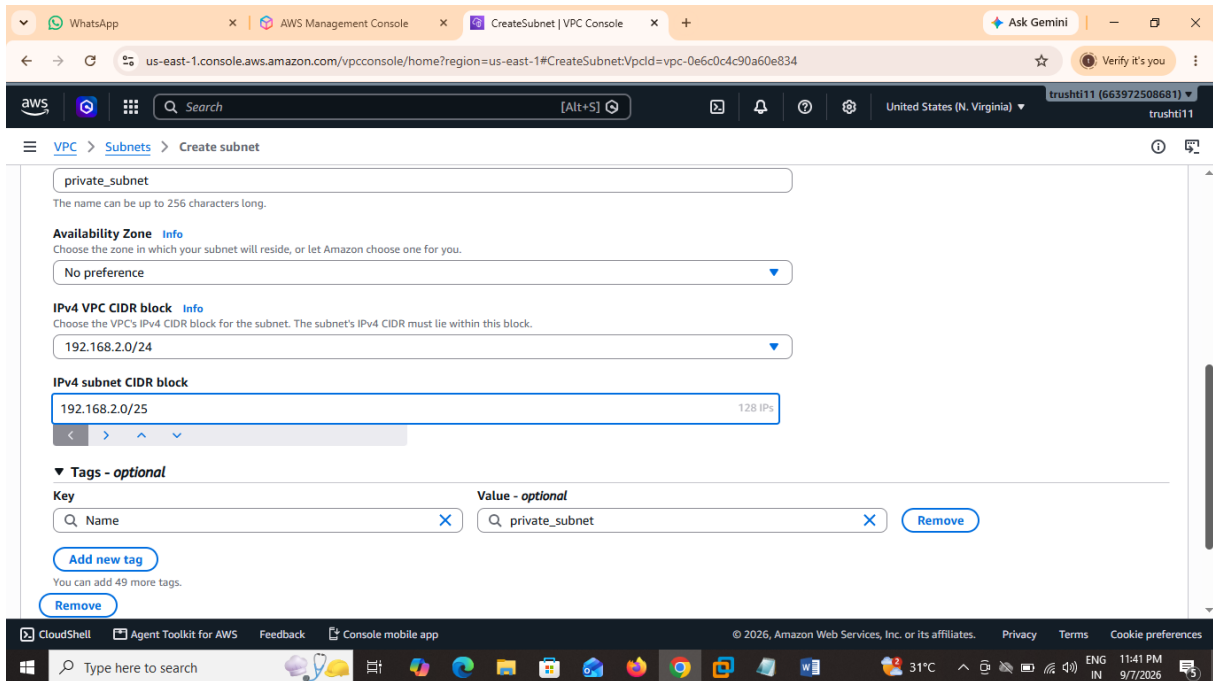


Figure A.4 — AWS Console

Route table creation and subnet association (myvpc1)

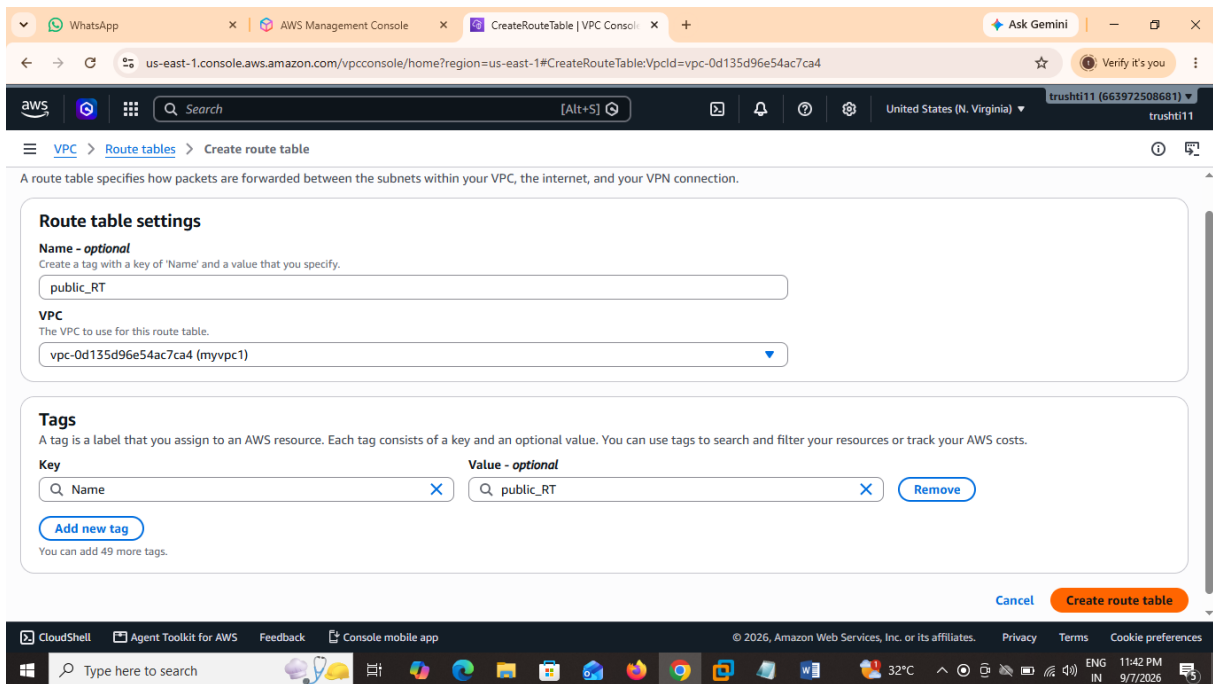


Figure A.1 — AWS Console

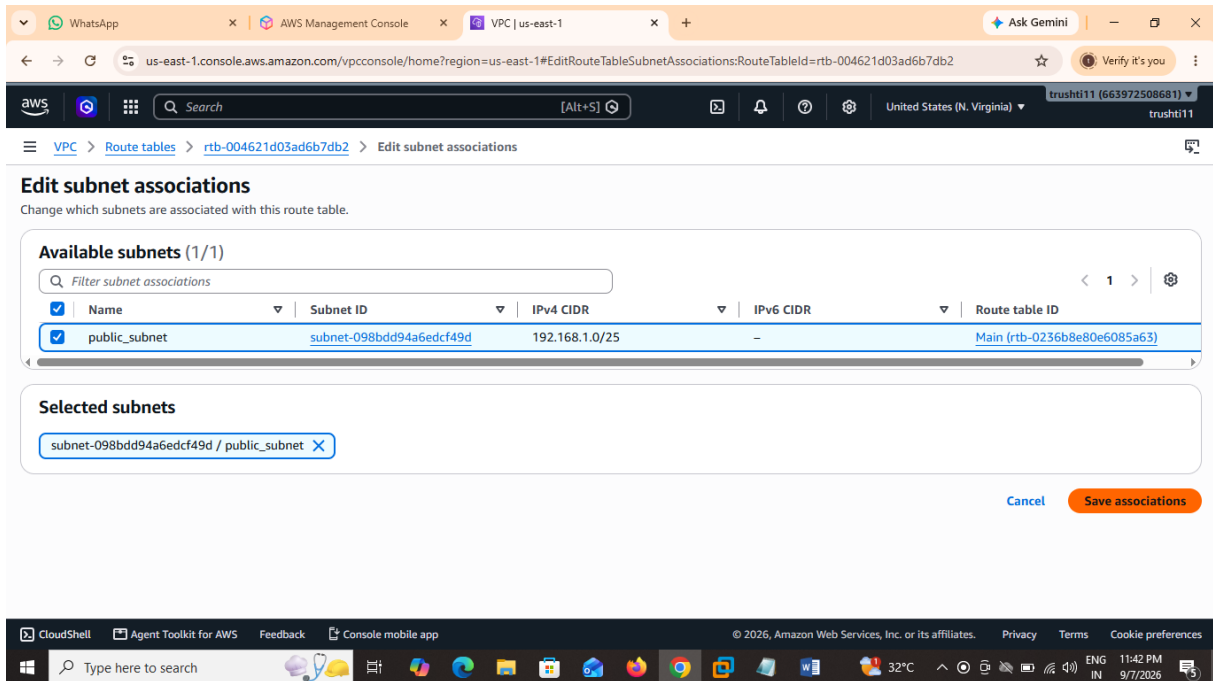


Figure A.2 — AWS Console

Internet Gateway — create and attach

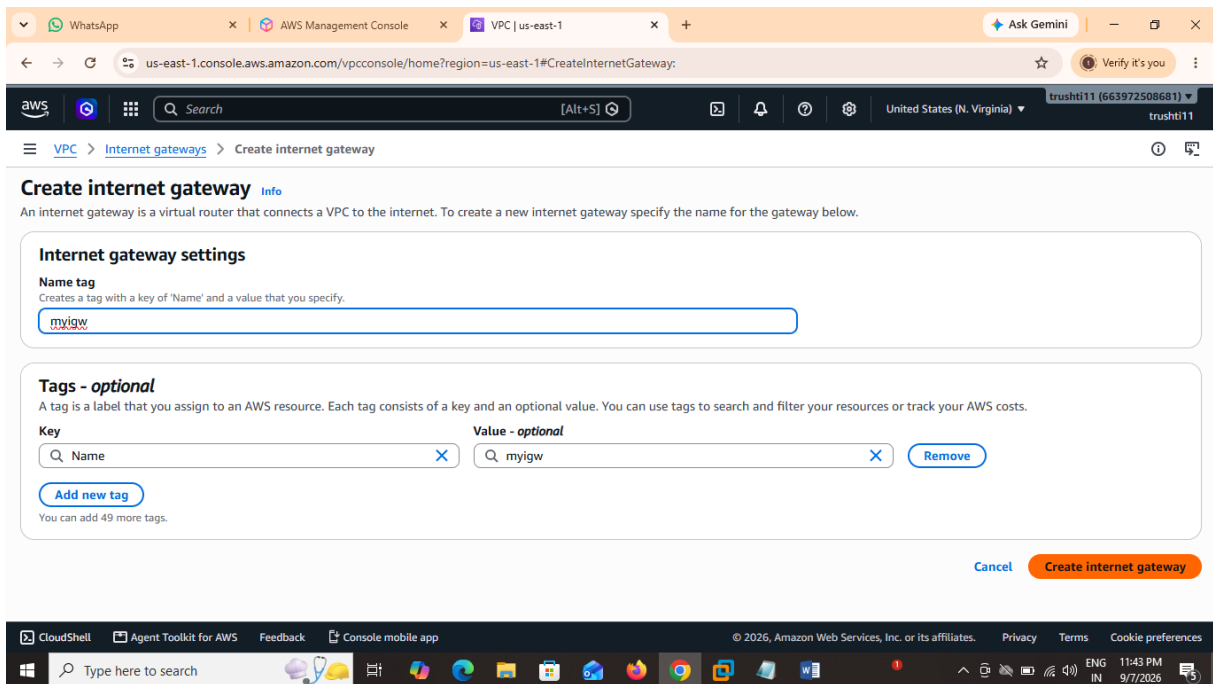


Figure A.1 — AWS Console

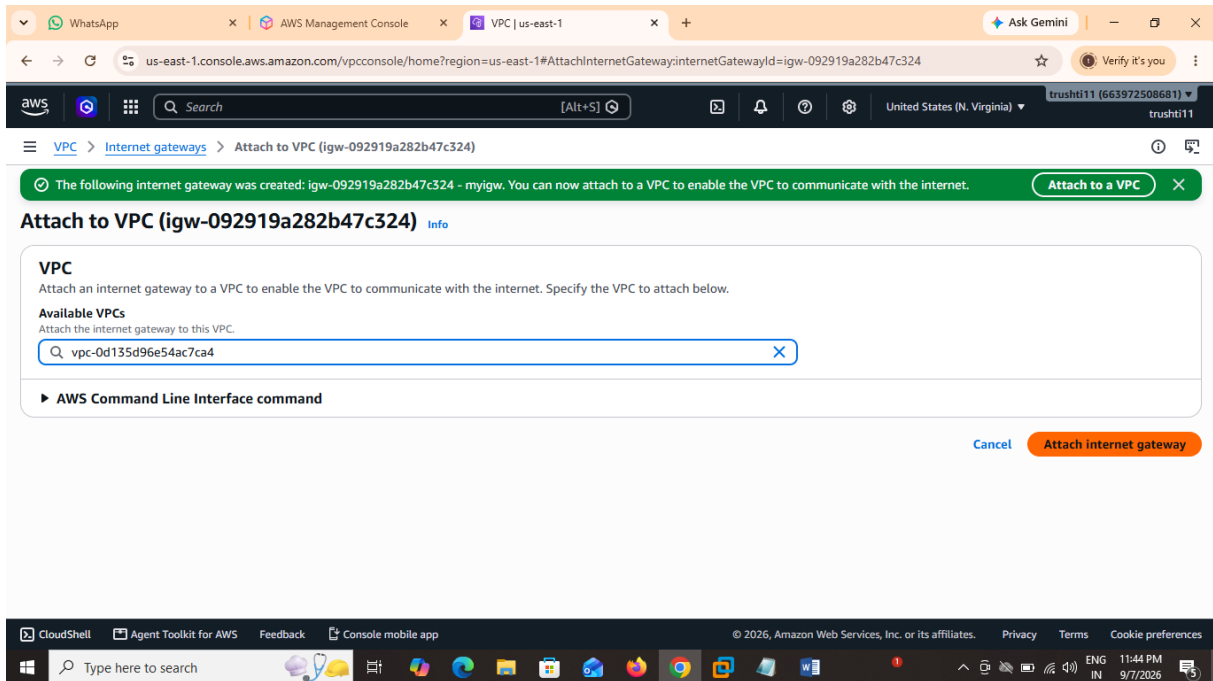


Figure A.2 — AWS Console

Adding the default internet route

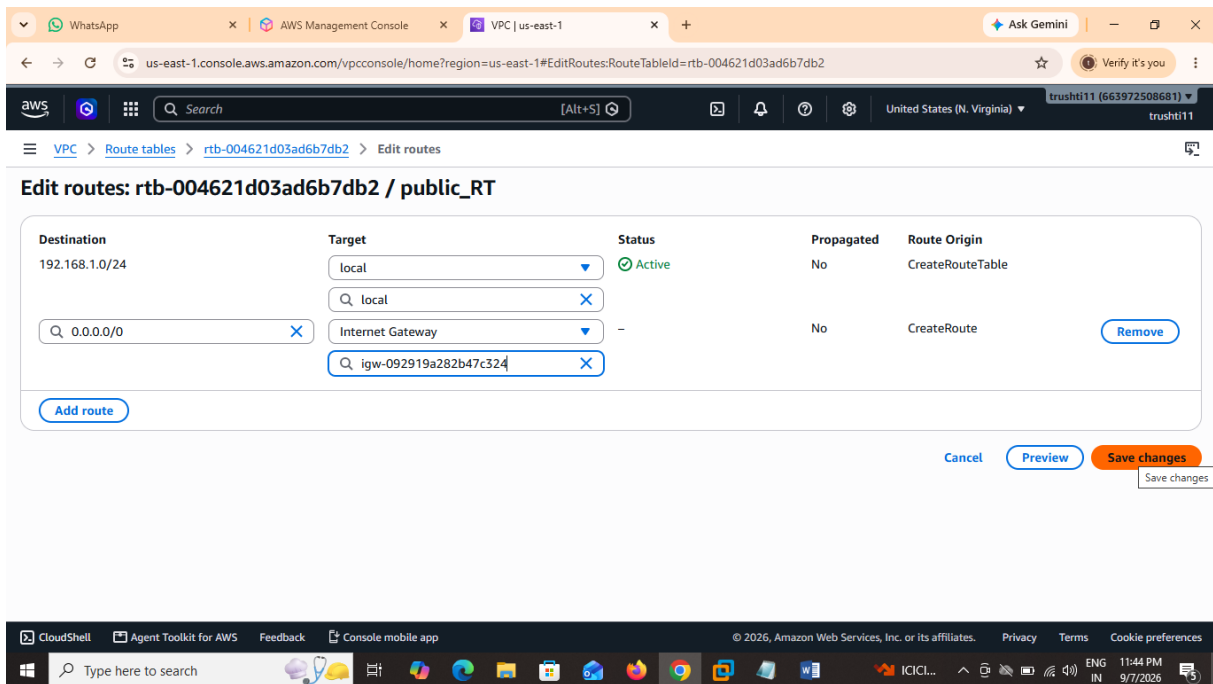


Figure A.1 — AWS Console

Launching the EC2 instance in myvpc1 (publicins)

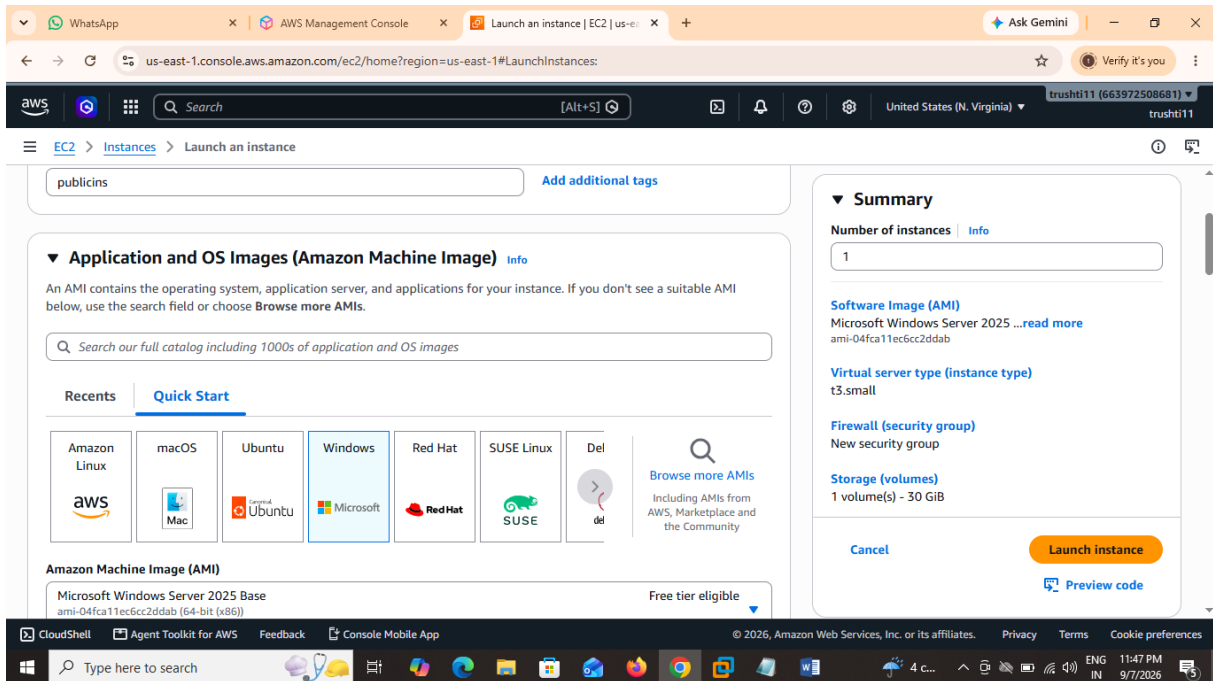


Figure A.1 — AWS Console

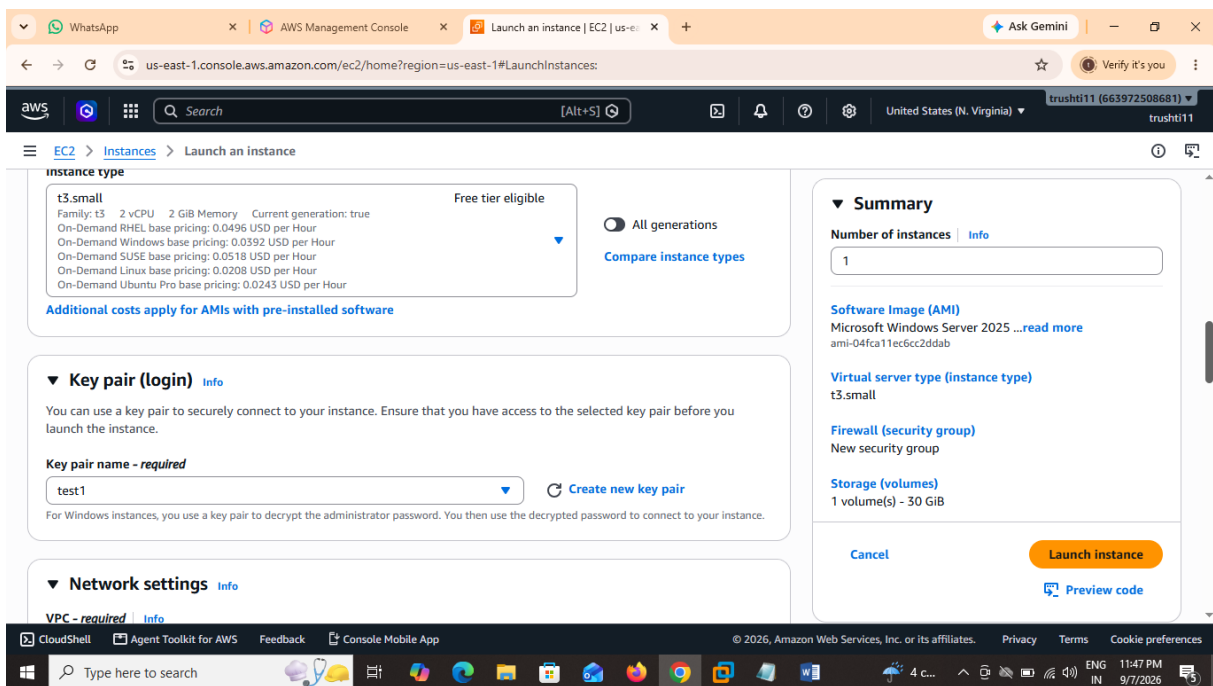


Figure A.2 — AWS Console

Launching the EC2 instance in myvpc2 (privateins) and its security group

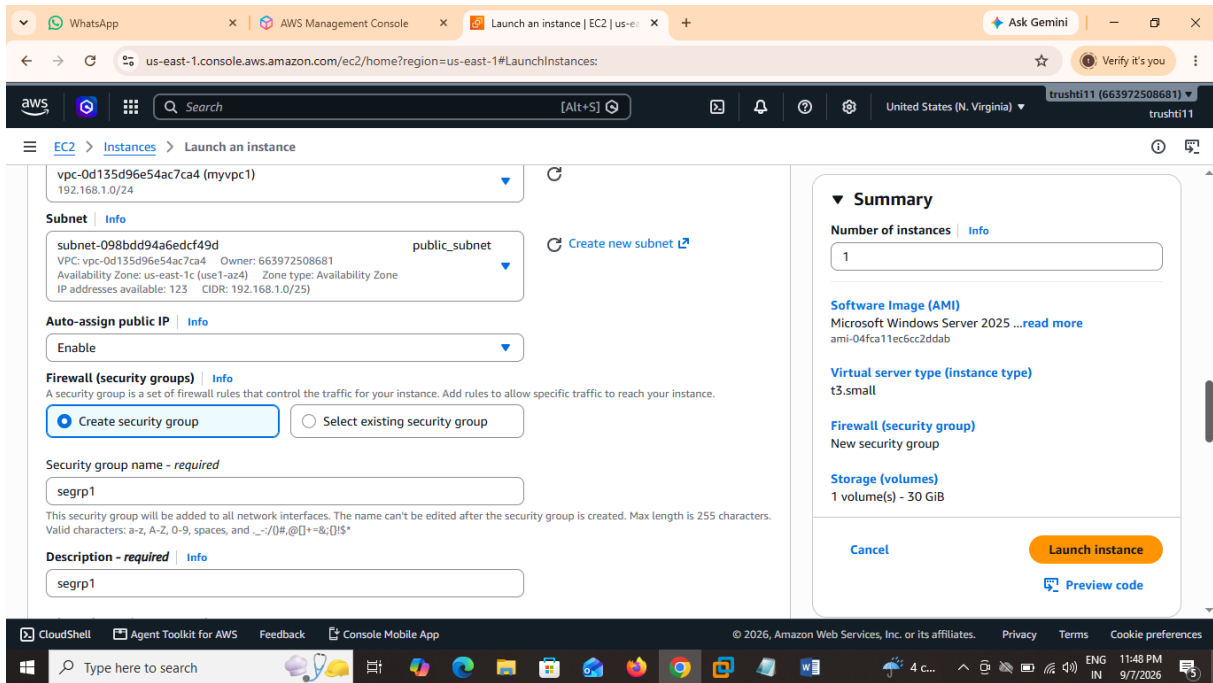


Figure A.1 — AWS Console

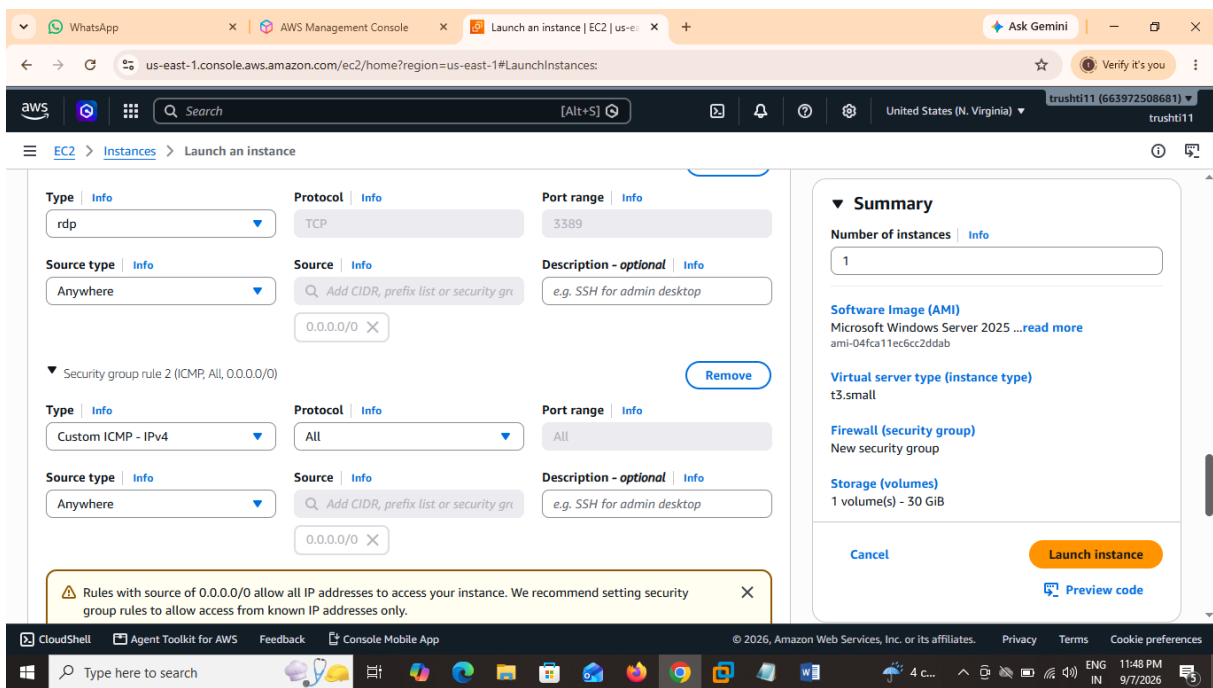


Figure A.2 — AWS Console

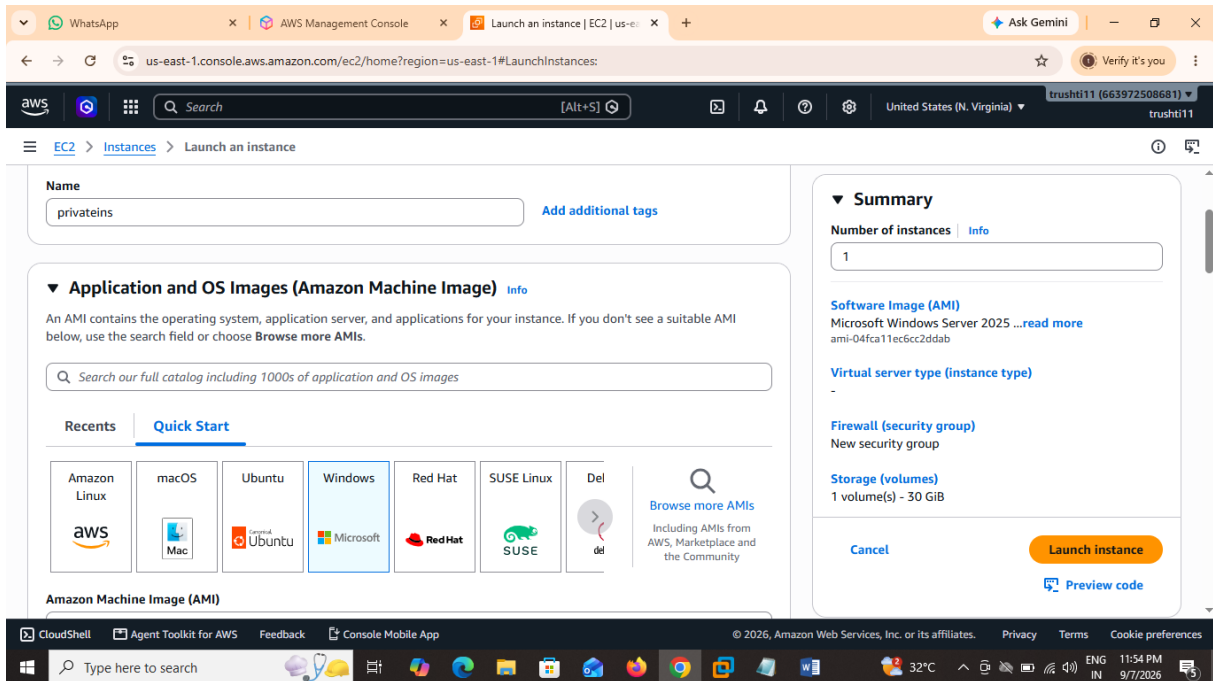


Figure A.3 — AWS Console

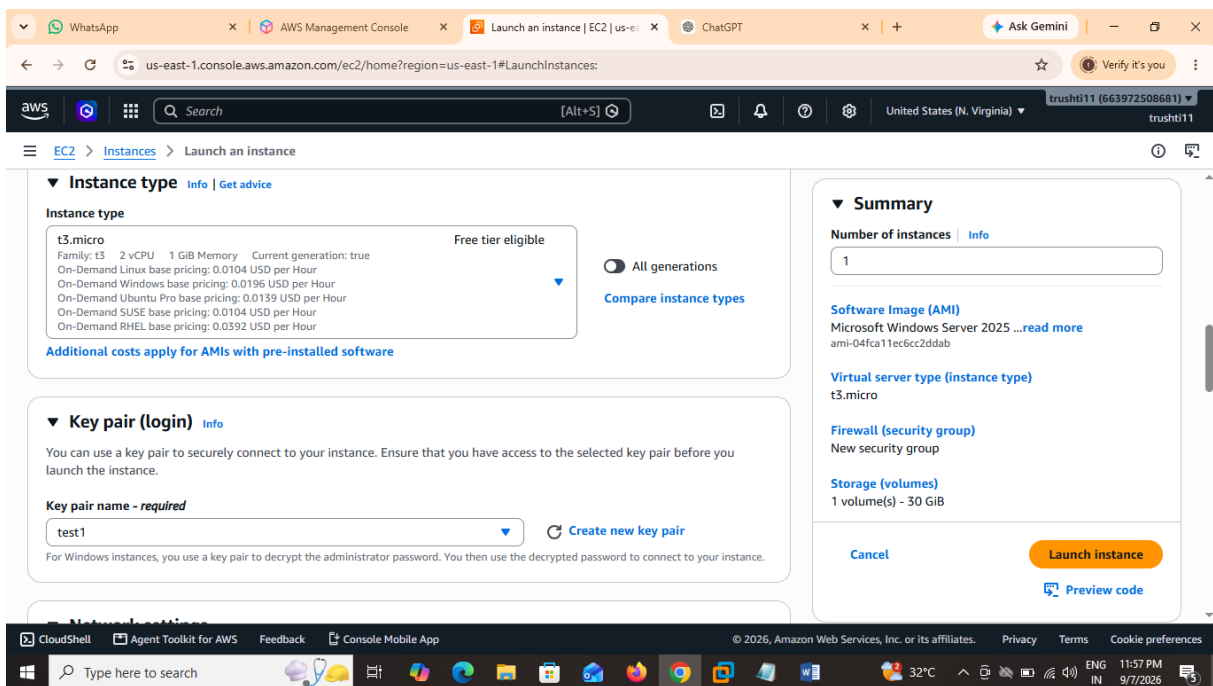


Figure A.4 — AWS Console

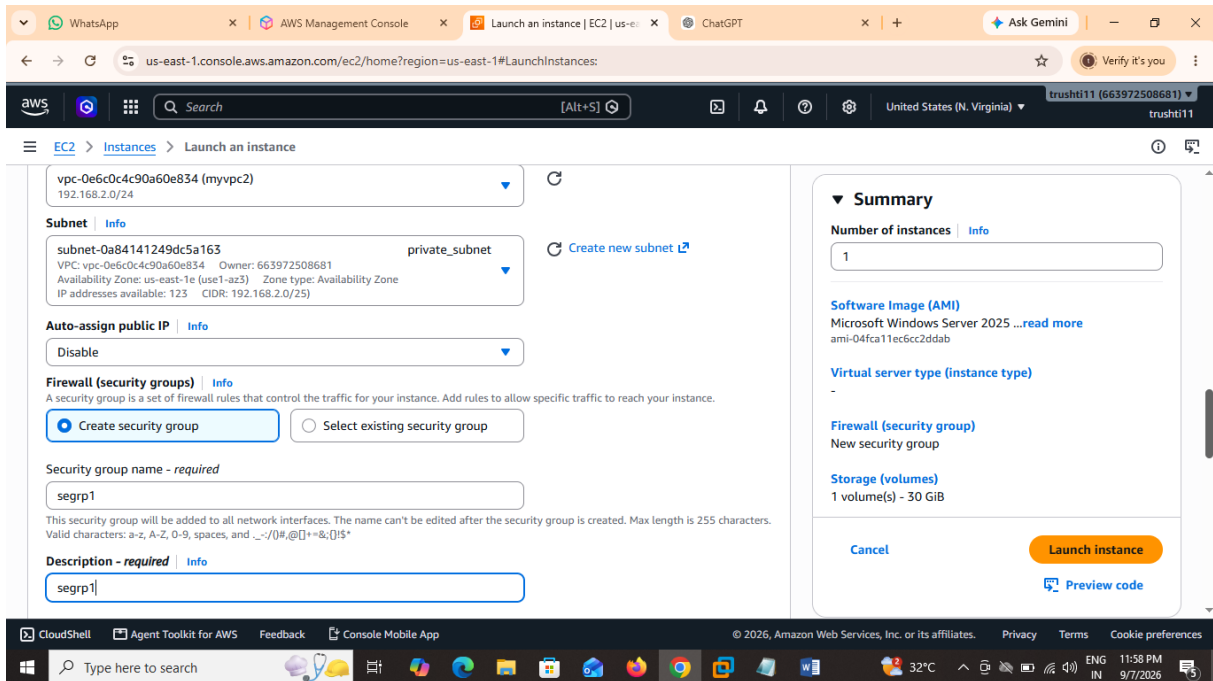


Figure A.5 — AWS Console

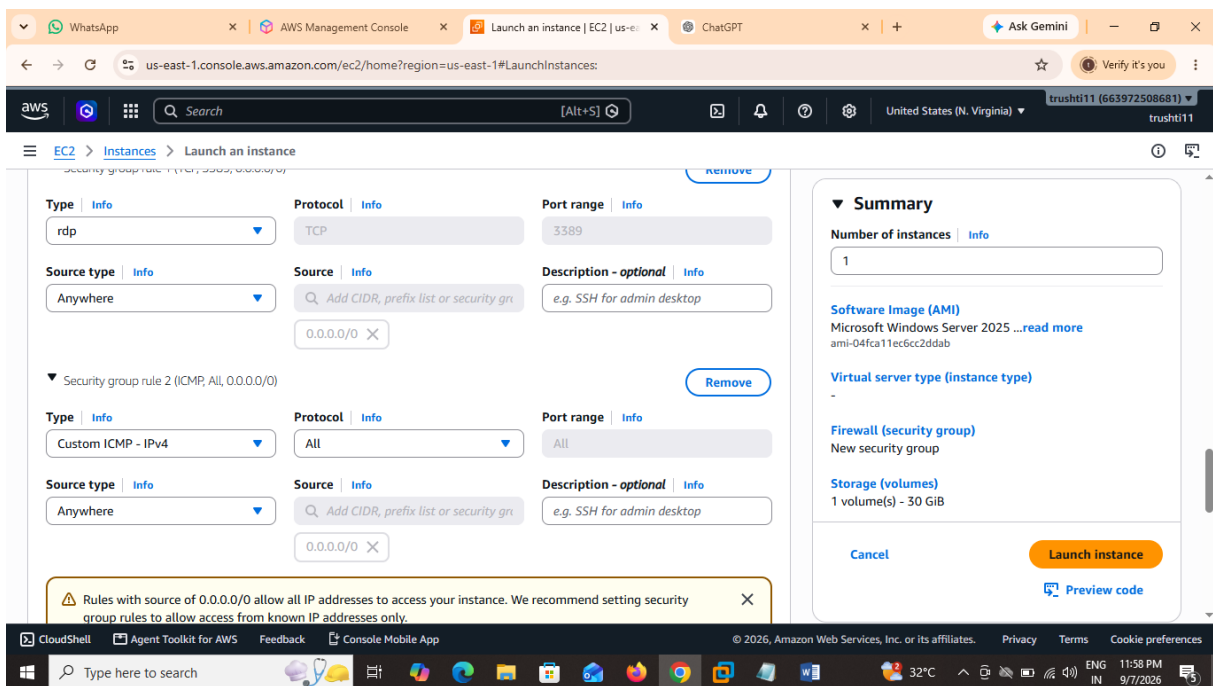


Figure A.6 — AWS Console

Creating the VPC Peering Connection (vpc1-vpc2)

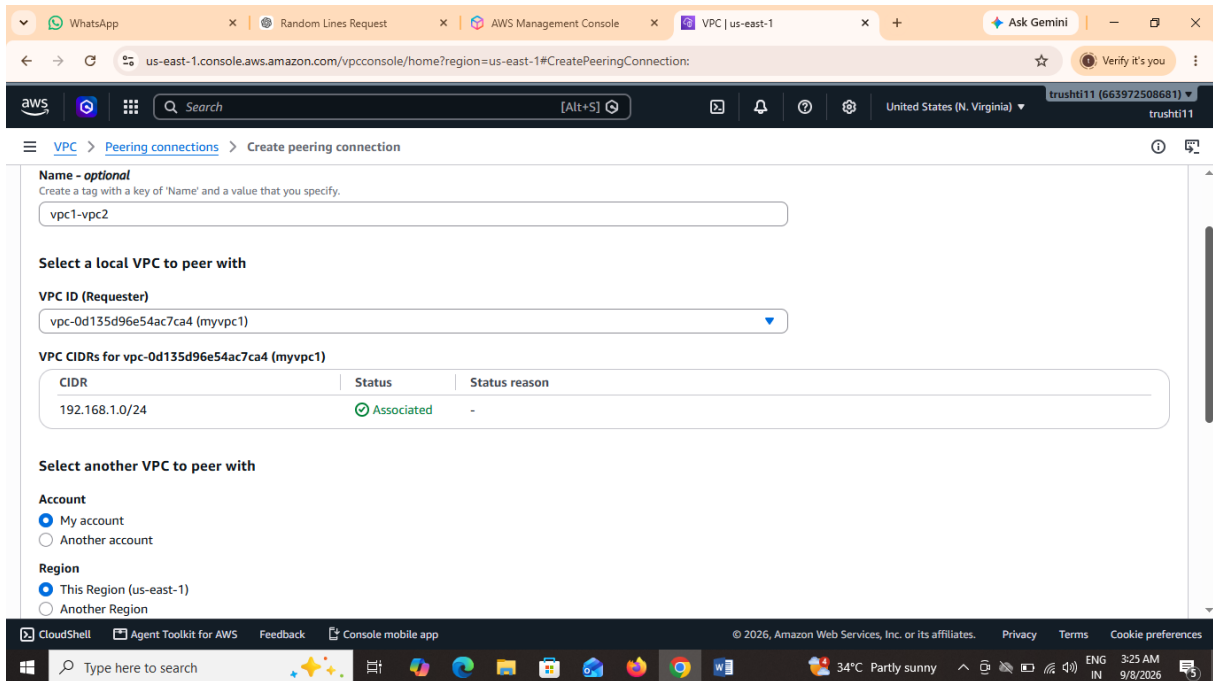


Figure A.1 — AWS Console

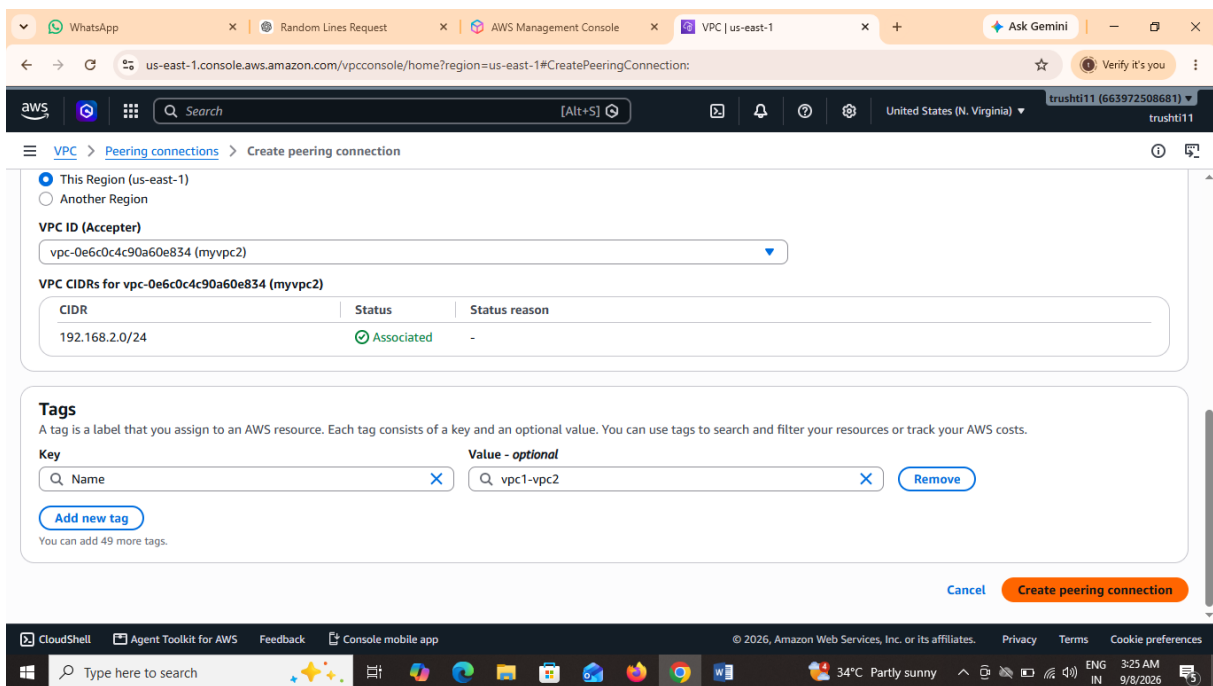


Figure A.2 — AWS Console

Accepting the peering request

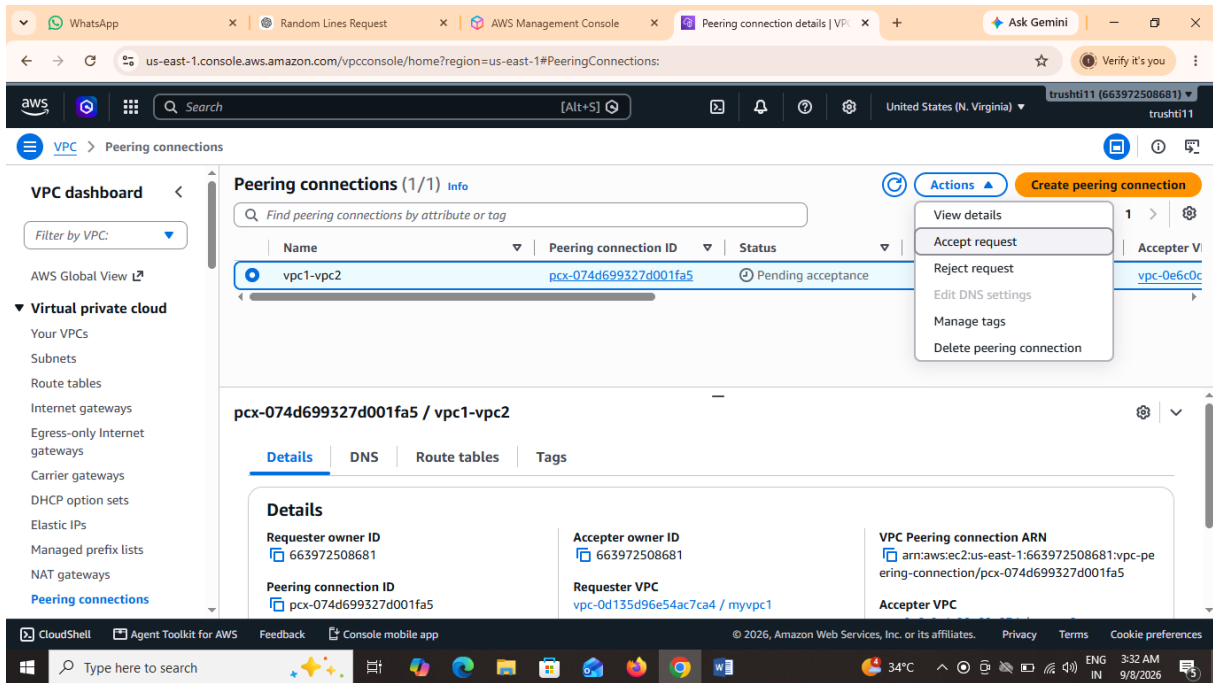


Figure A.1 — AWS Console

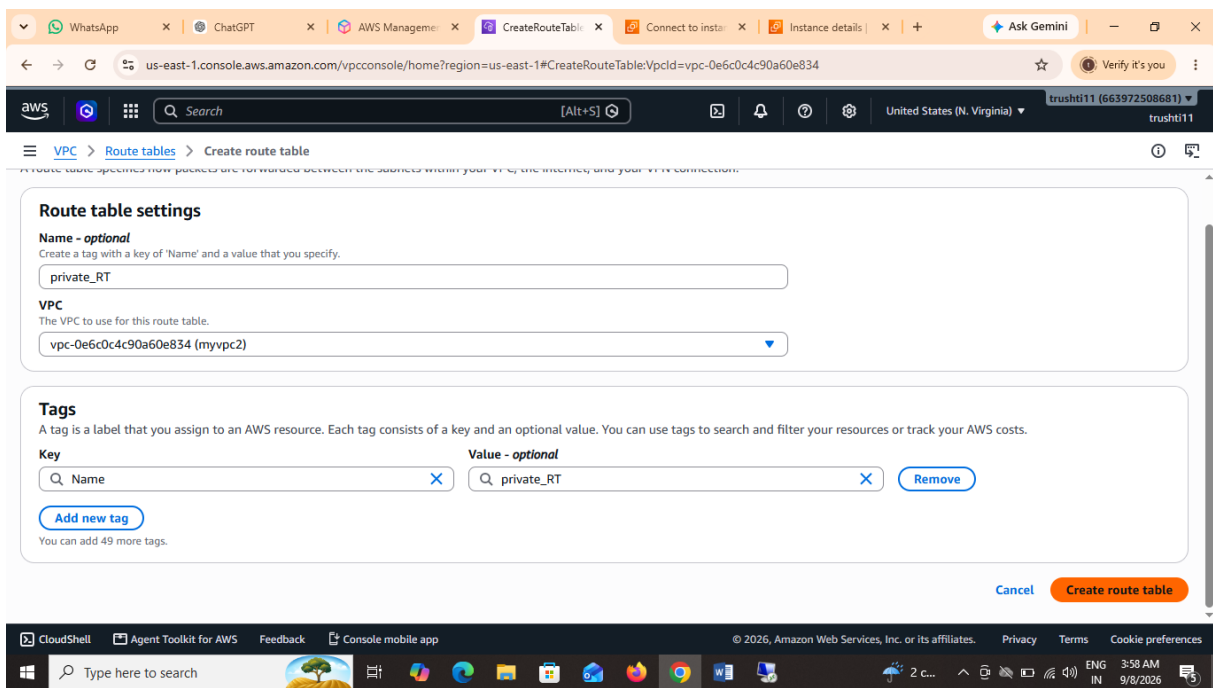


Figure A.2 — AWS Console

Route table for myvpc2 and subnet association

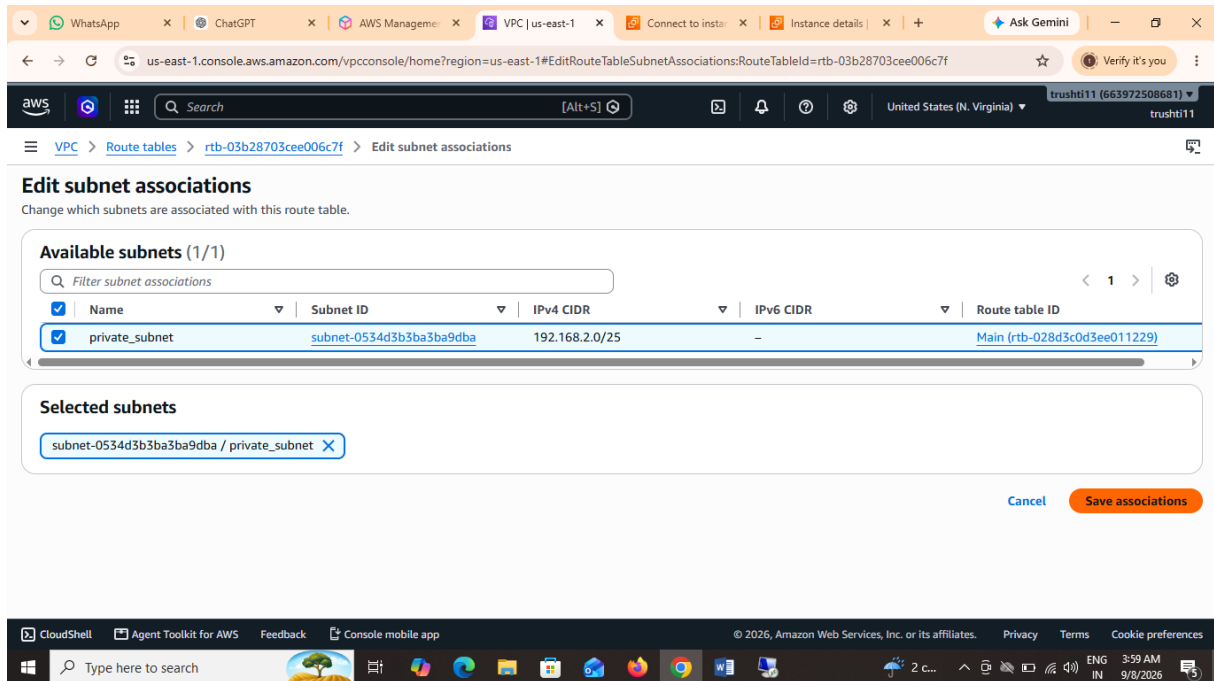


Figure A.1 — AWS Console

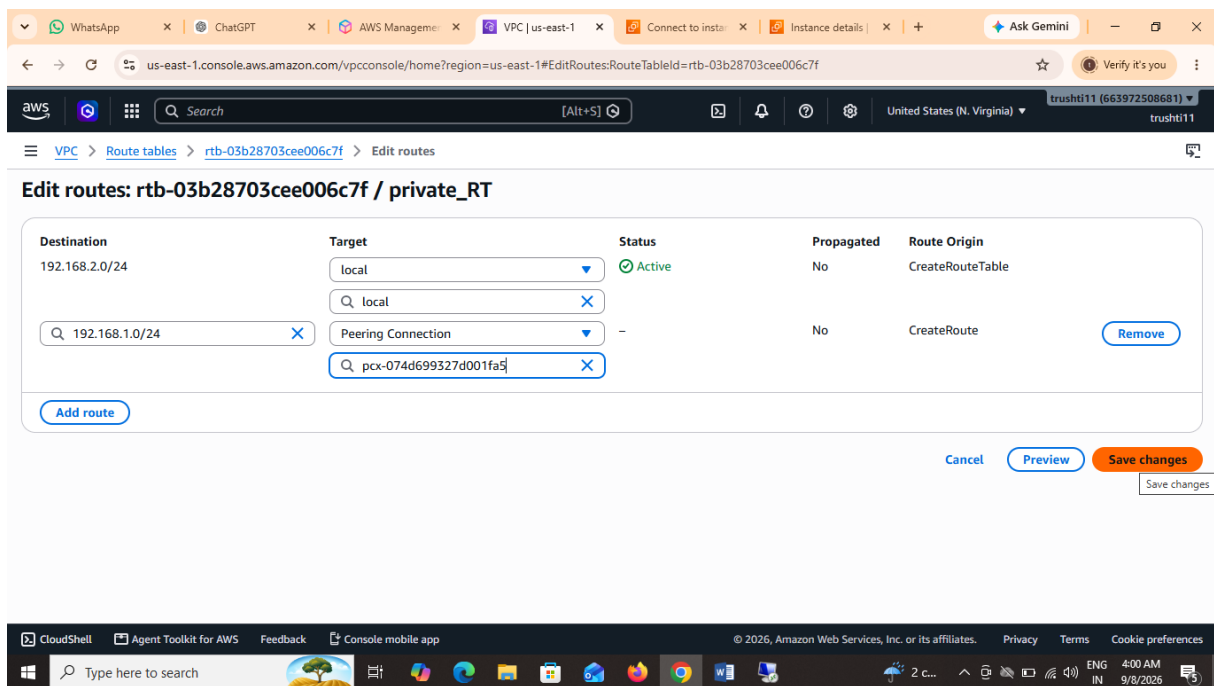


Figure A.2 — AWS Console

Updating both route tables to use the Peering Connection, and final Active status

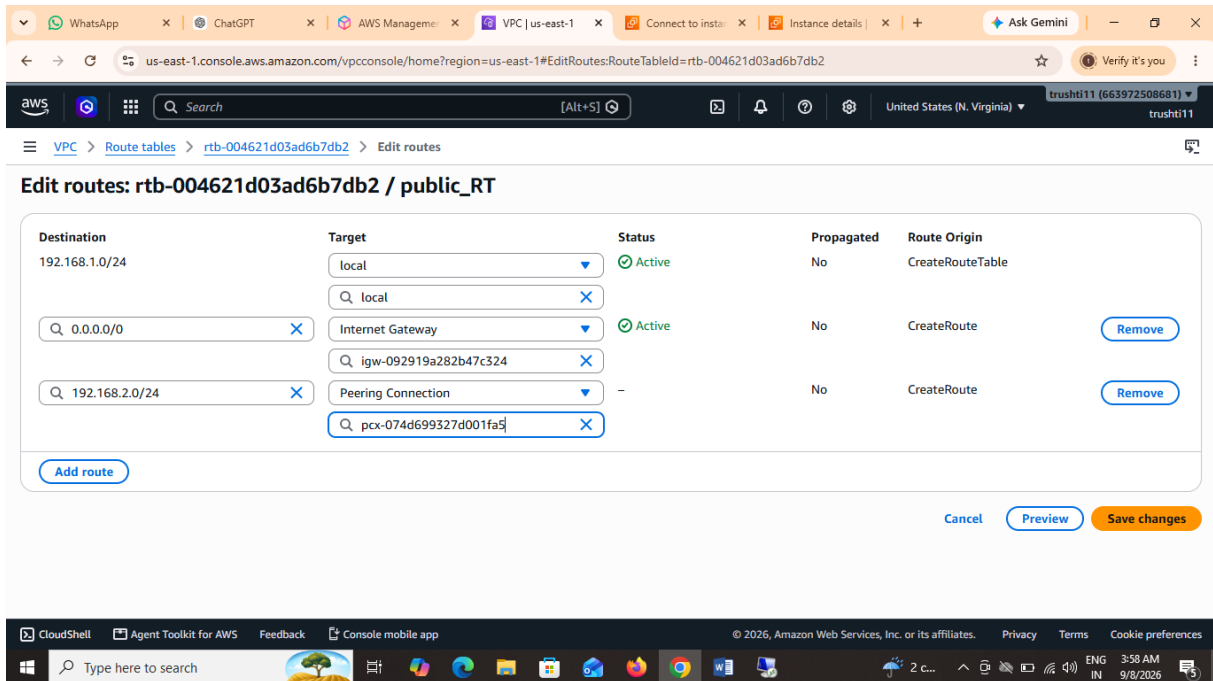


Figure A.1 — AWS Console

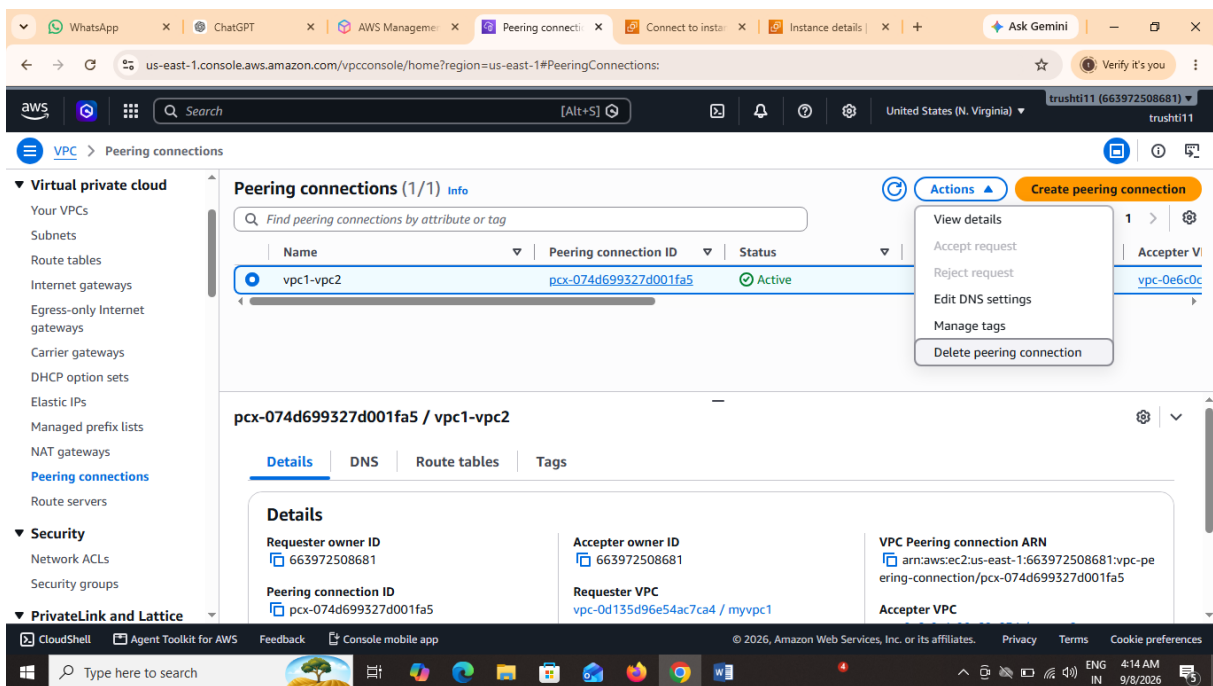


Figure A.2 — AWS Console

3. Part B — VPC Peering across Different Regions

3.1 Theory

Inter-region VPC peering connects VPCs that live in two different AWS Regions (e.g. US East (N. Virginia) and US West (N. California)). It works the same way as same-region peering from a routing perspective, but with a few extra considerations:

- When creating the connection, the requester must choose "Another Region" and select the target region before choosing the acceptor VPC.
- To accept the request, the acceptor must switch the AWS Console (or CLI/SDK region parameter) to the acceptor VPC's region — the pending request will not be visible from the requester's region.
- All inter-region traffic is automatically encrypted at the physical-network layer by AWS and is billed at the standard inter-region data-transfer rate.
- Route tables in both regions must be updated independently, exactly as with same-region peering.

3.2 Lab Topology

- VPC "myvpc1" — CIDR 192.168.1.0/24 — created in the US East (N. Virginia) region, with a public subnet and an EC2 (Ubuntu) instance "publicins1".
- VPC "myvpc2" — CIDR 192.168.2.0/24 — created in the US West (N. California) region, with a public subnet and an EC2 (Ubuntu) instance "publicins2".
- Security groups on both instances allow SSH (22) and ICMP so connectivity can be verified by SSH login and ping.

3.3 Steps Performed

1. In Region A (N. Virginia), create VPC "myvpc1" with CIDR 192.168.1.0/24.
2. Create a subnet inside myvpc1, a route table, and associate the subnet.
3. Create an Internet Gateway, attach it to myvpc1, and add a default route (0.0.0.0/0) to it.
4. Switch the console Region to Region B (N. California) and repeat the same steps to create VPC "myvpc2" (CIDR 192.168.2.0/24), its subnet, route table, Internet Gateway, and default route.
5. Launch an EC2 (Ubuntu) instance in each VPC, each with a security group allowing SSH and ICMP.
6. Connect to each instance over SSH to confirm it is reachable, and note its private IP address.
7. From Region B, create the VPC Peering Connection: select the local VPC as requester, choose "Another Region", pick Region A, and enter the acceptor VPC ID (myvpc1).
8. Switch the console back to Region A (as instructed by the confirmation banner) and accept the pending peering request under Peering Connections.
9. In Region B, edit the route table and add a route for 192.168.1.0/24 targeting the Peering Connection.

- 10. In Region A, edit the route table and add a route for 192.168.2.0/24 targeting the same Peering Connection.
- 11. SSH into an instance in one region and ping the private IP address of the instance in the other region to confirm connectivity.

3.4 Screenshots

Creating VPC "myvpc1" in Region A and its networking (subnet, route table, IGW, default route)

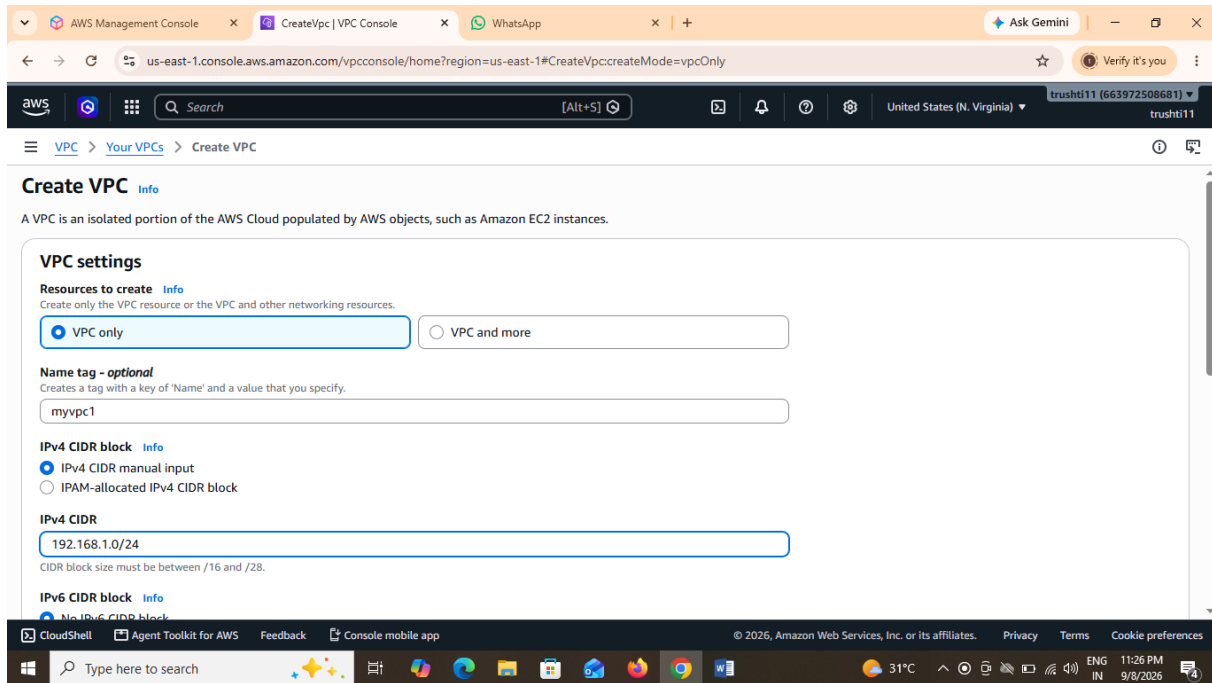


Figure B.1 — AWS Console

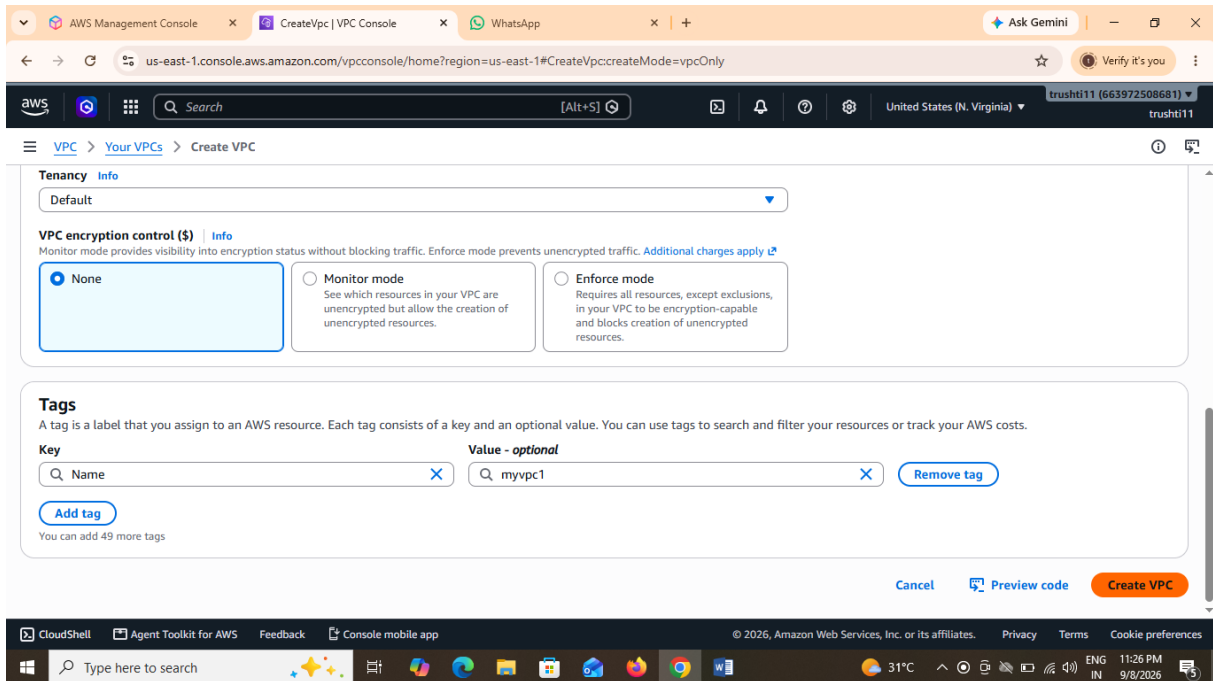


Figure B.2 — AWS Console

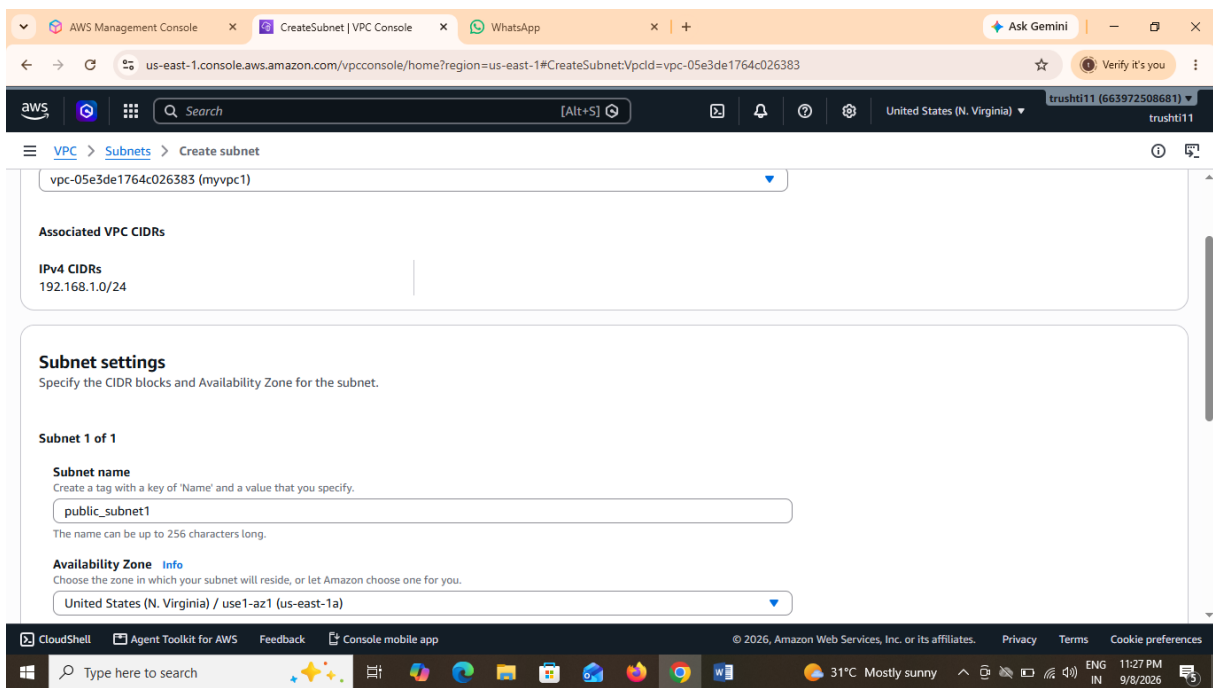


Figure B.3 — AWS Console

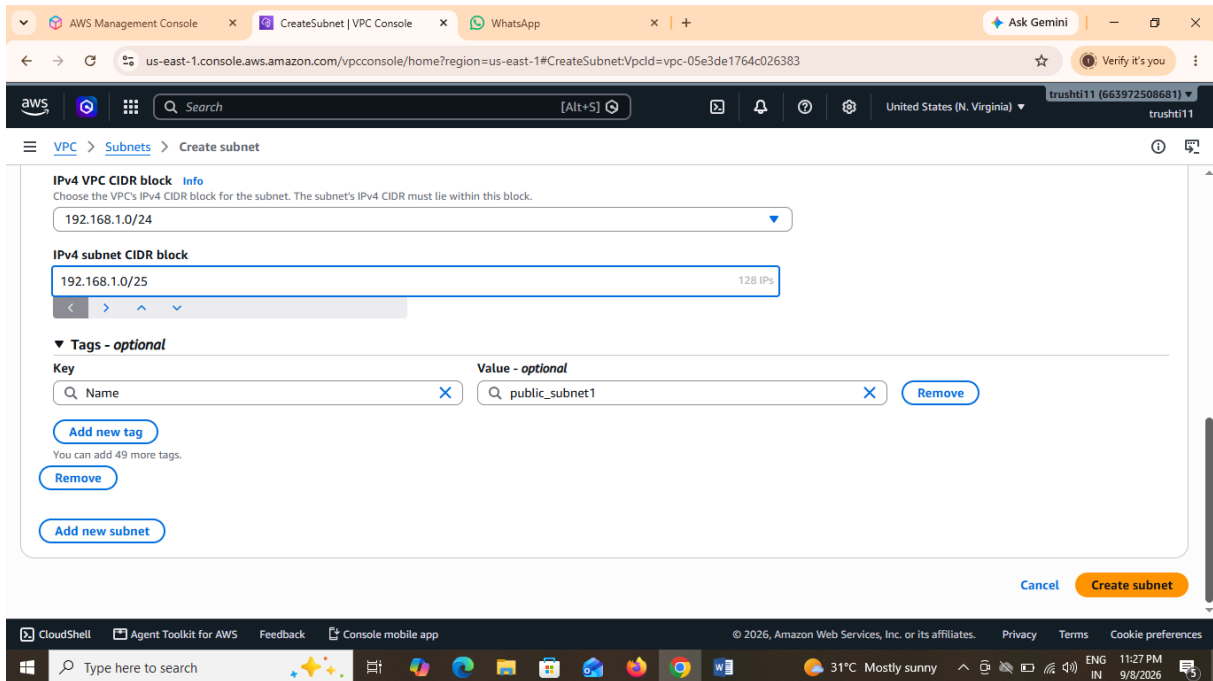


Figure B.4 — AWS Console

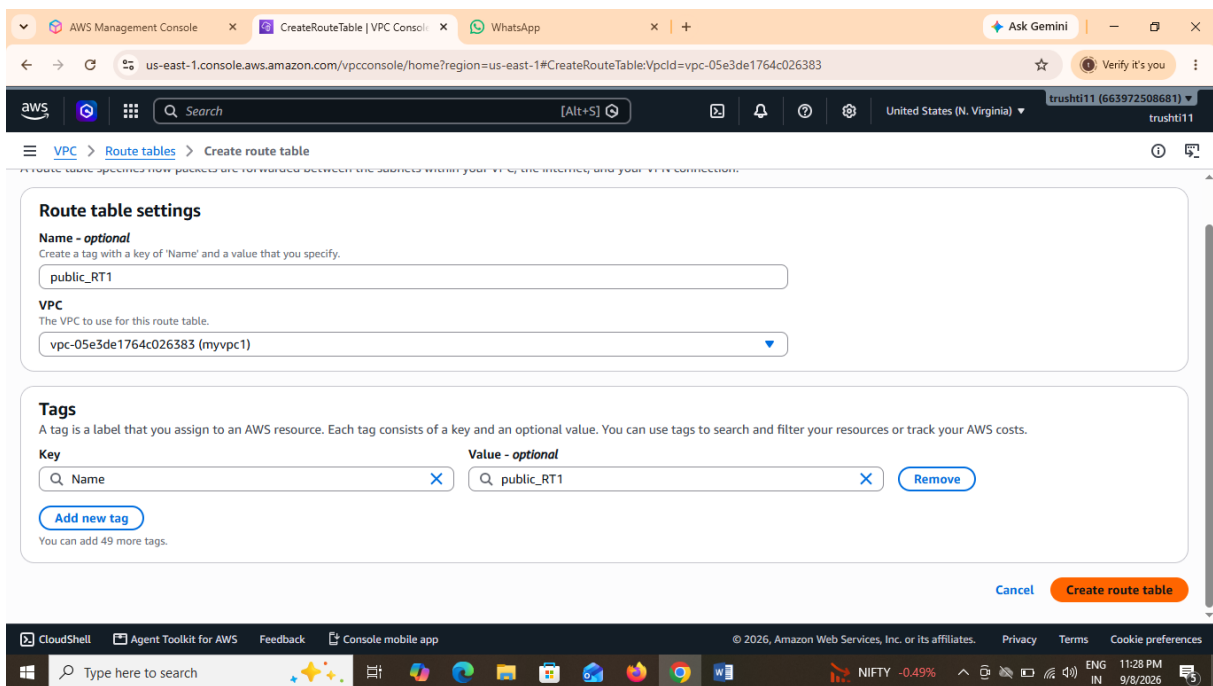


Figure B.5 — AWS Console

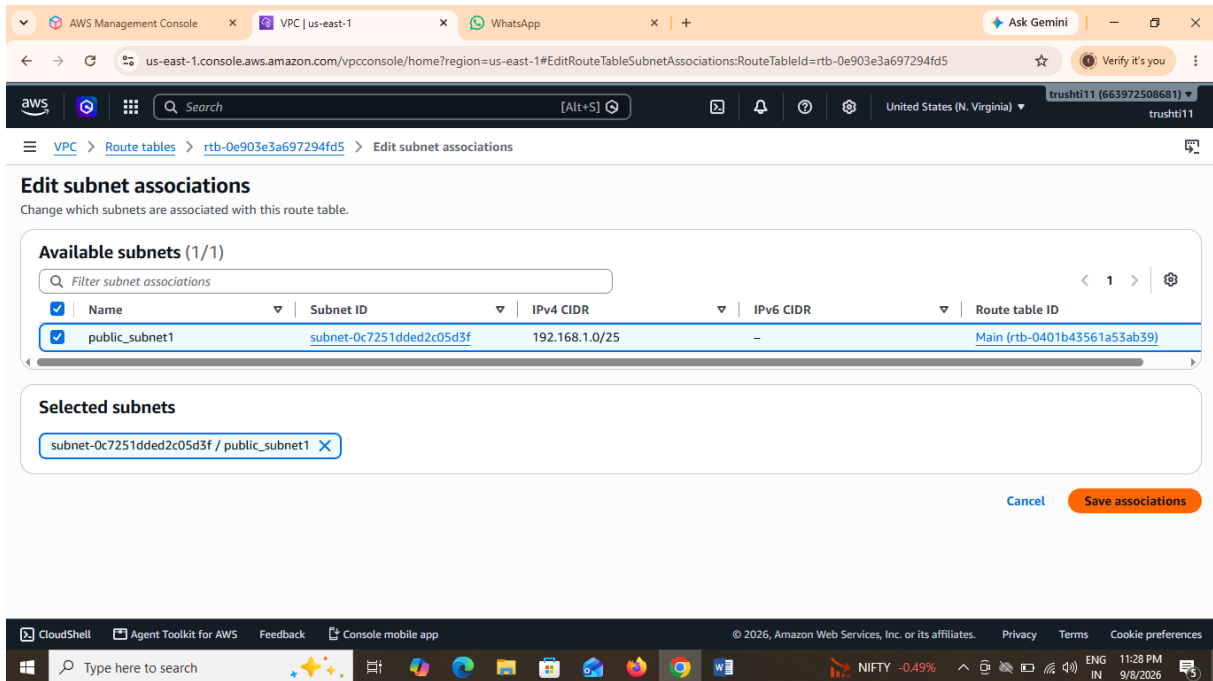


Figure B.6 — AWS Console

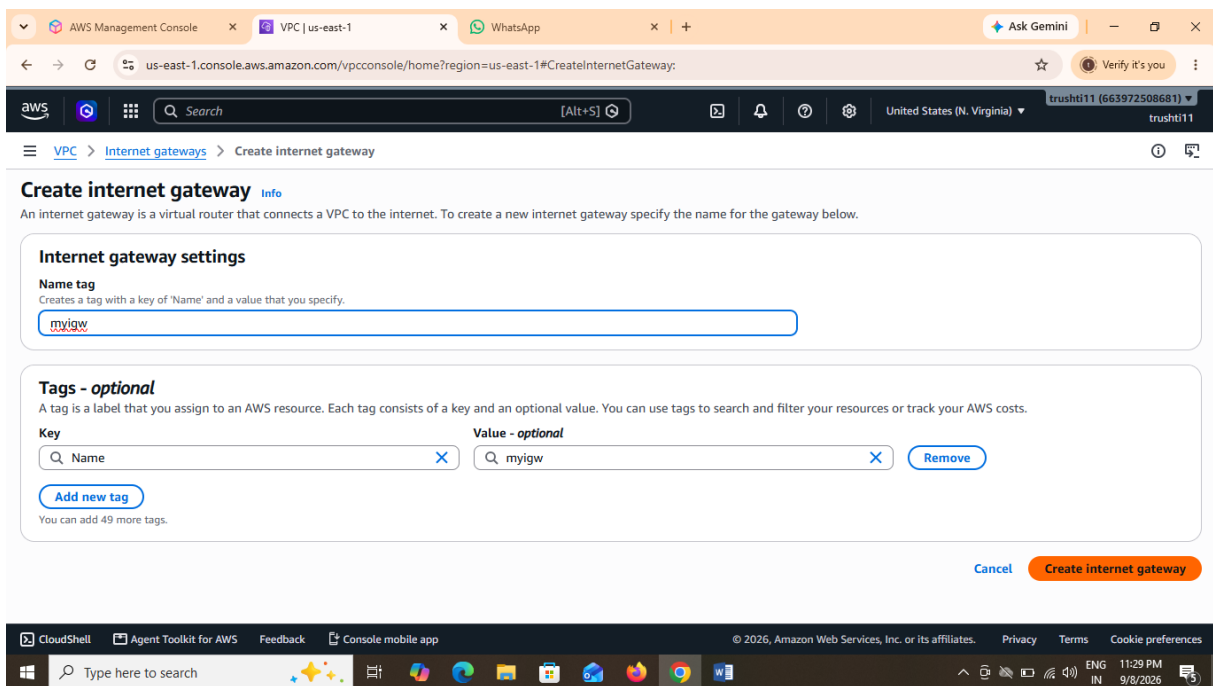


Figure B.7 — AWS Console

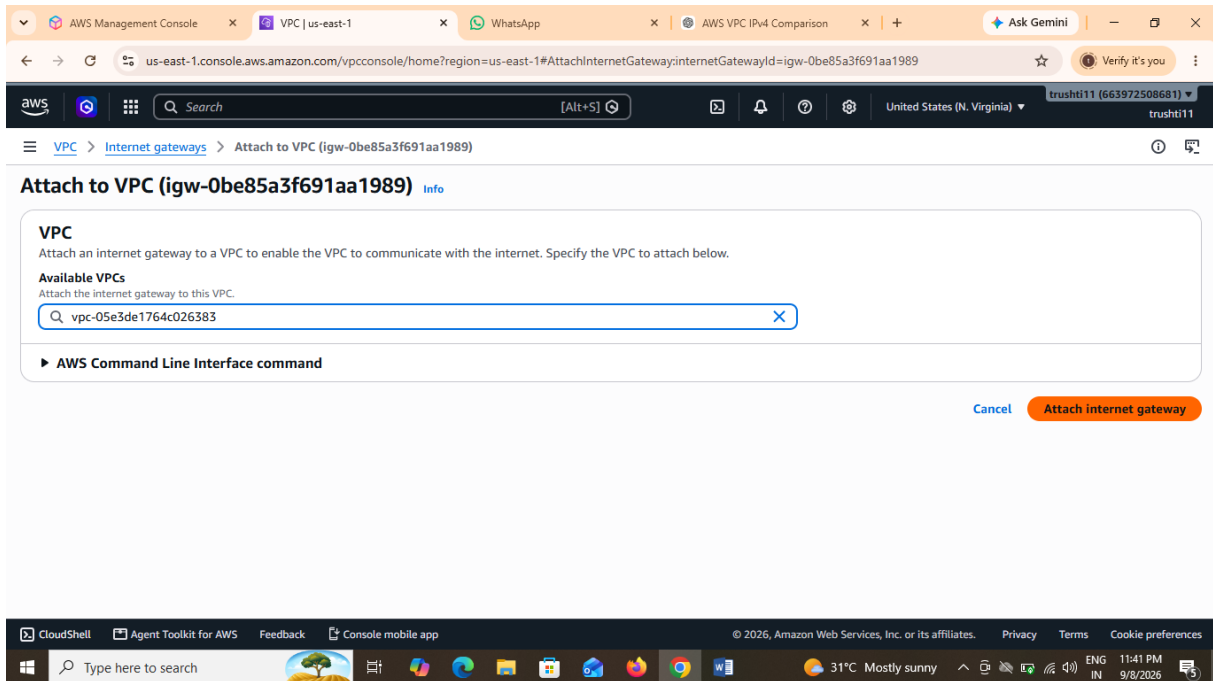


Figure B.8 — AWS Console

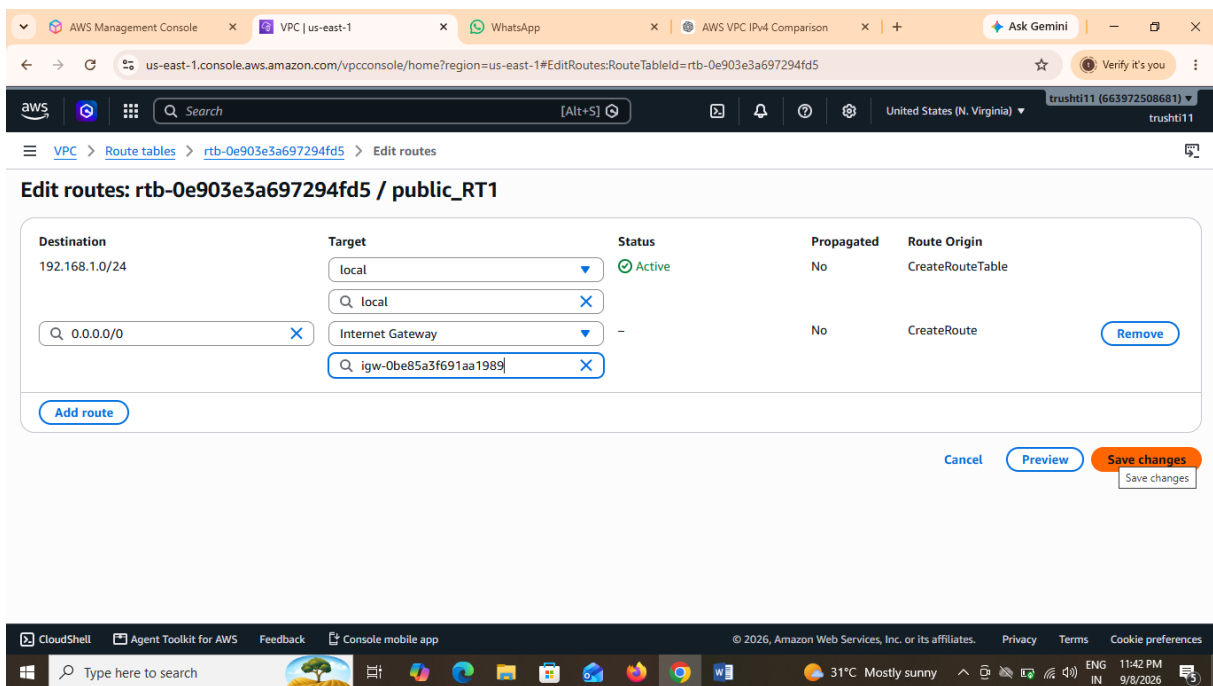


Figure B.9 — AWS Console

Creating VPC "myvpc2" in Region B and its networking (subnet, route table, IGW, default route)

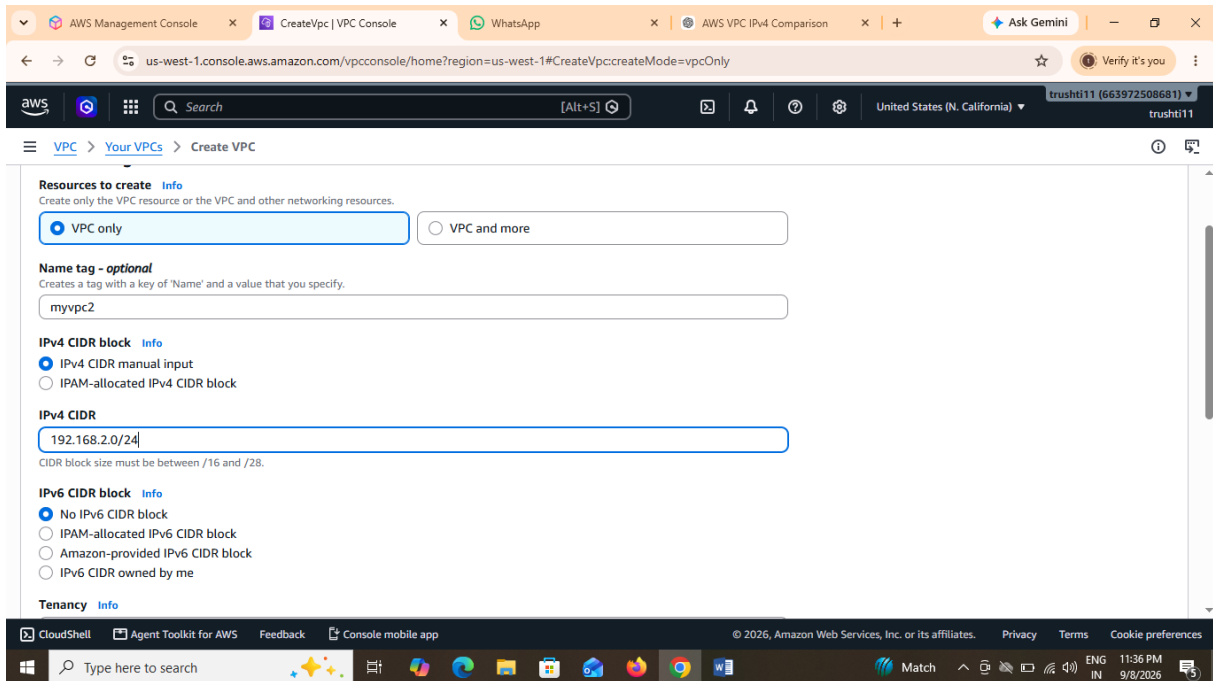


Figure B.1 — AWS Console

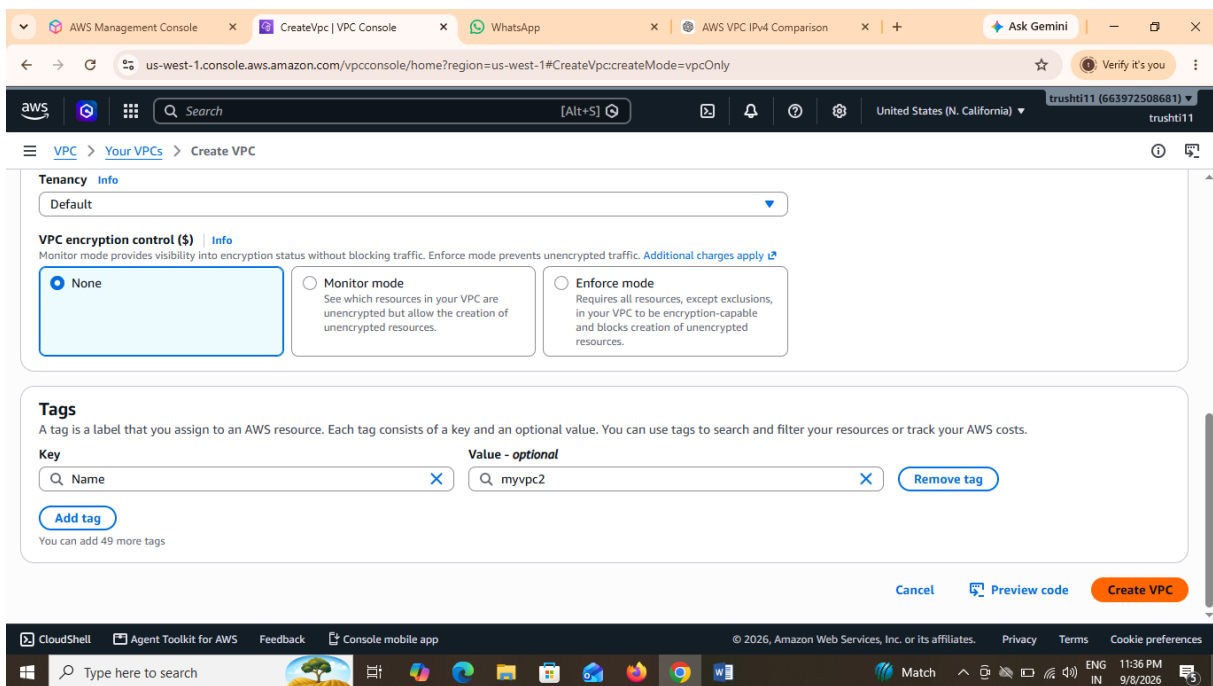


Figure B.2 — AWS Console

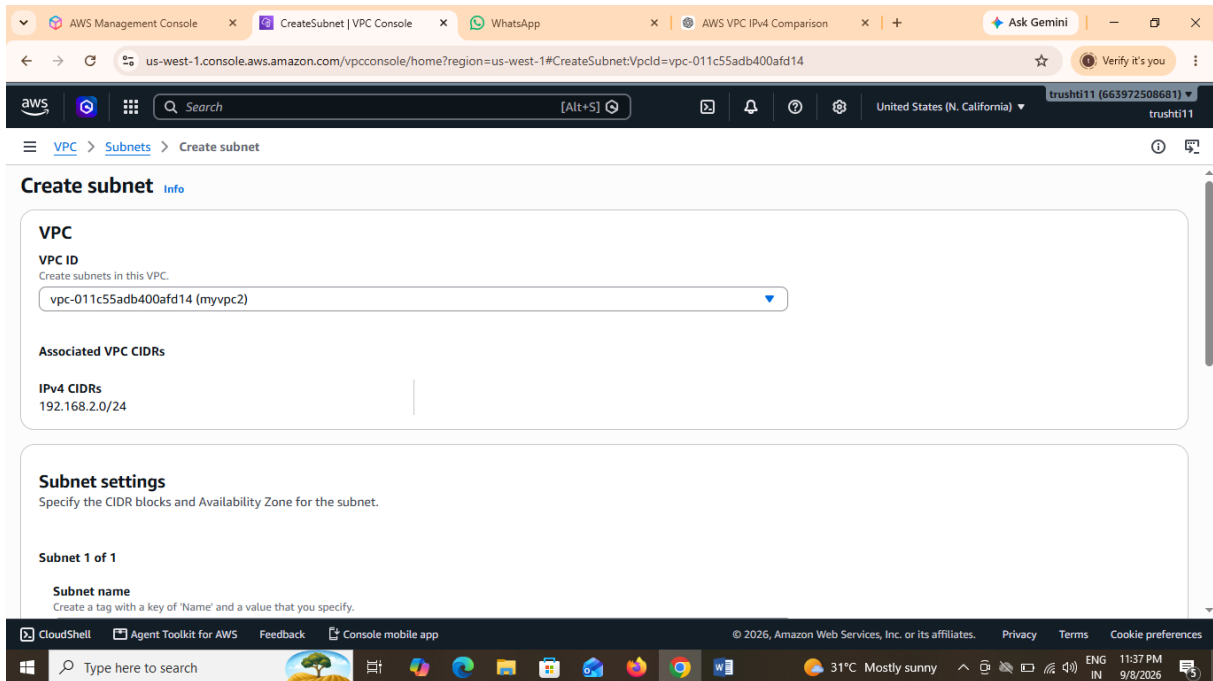


Figure B.3 — AWS Console

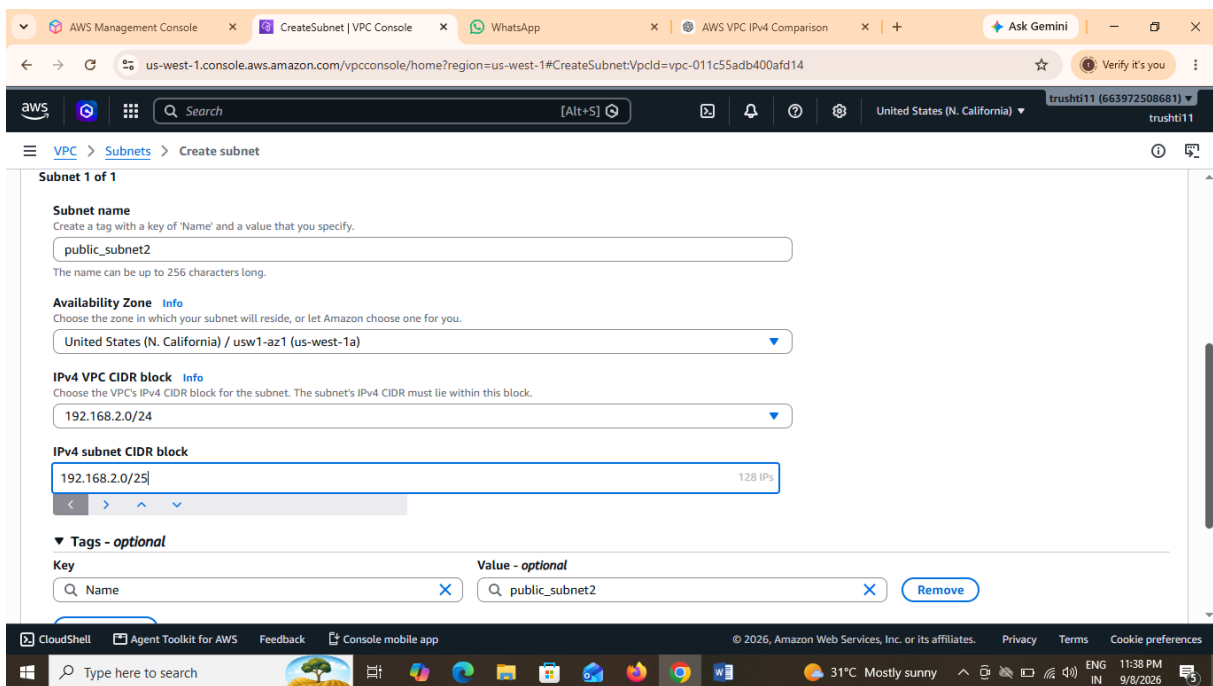


Figure B.4 — AWS Console

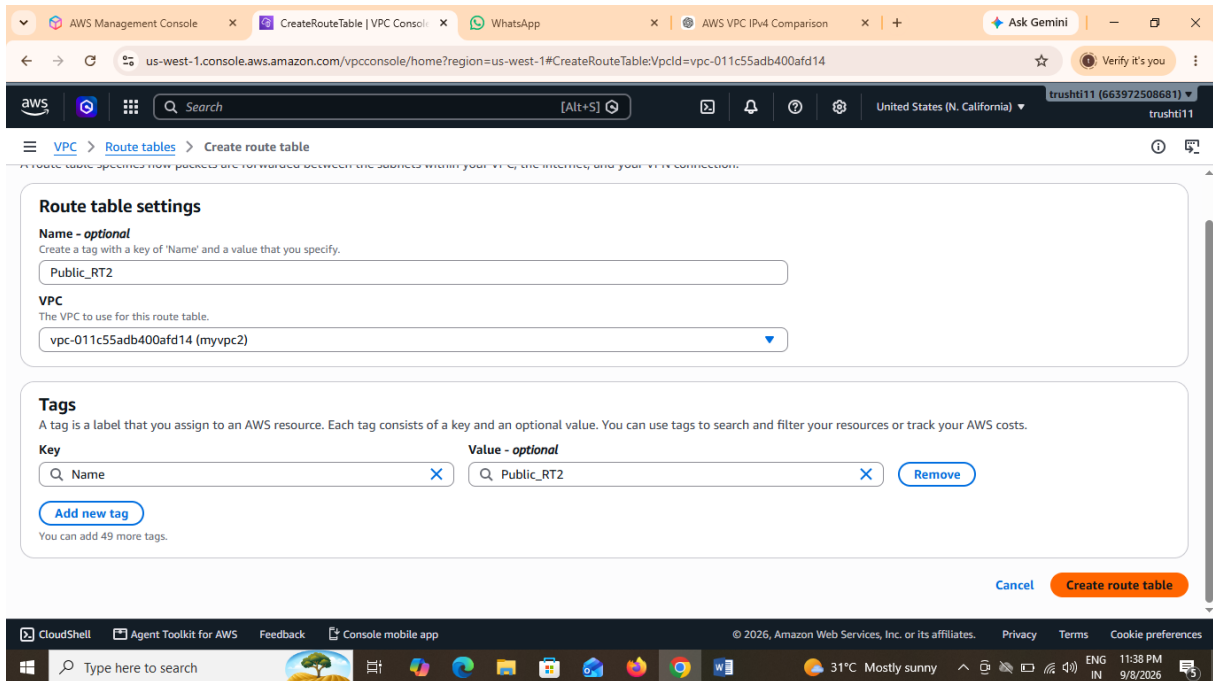


Figure B.5 — AWS Console

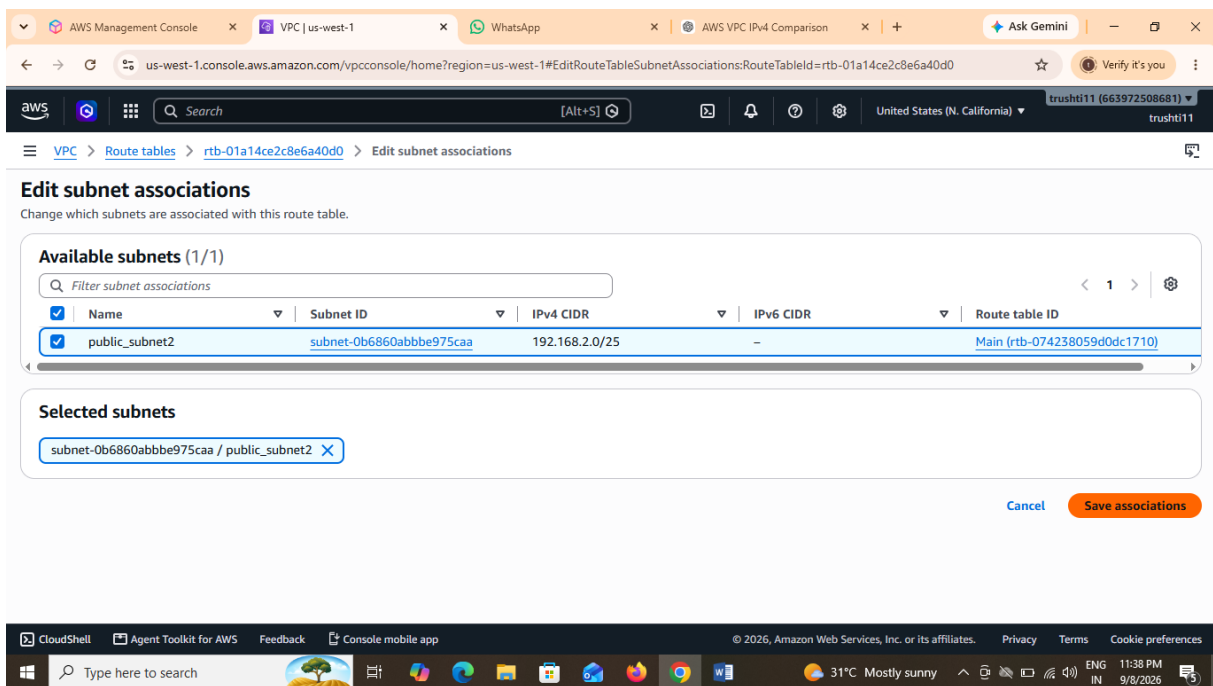


Figure B.6 — AWS Console

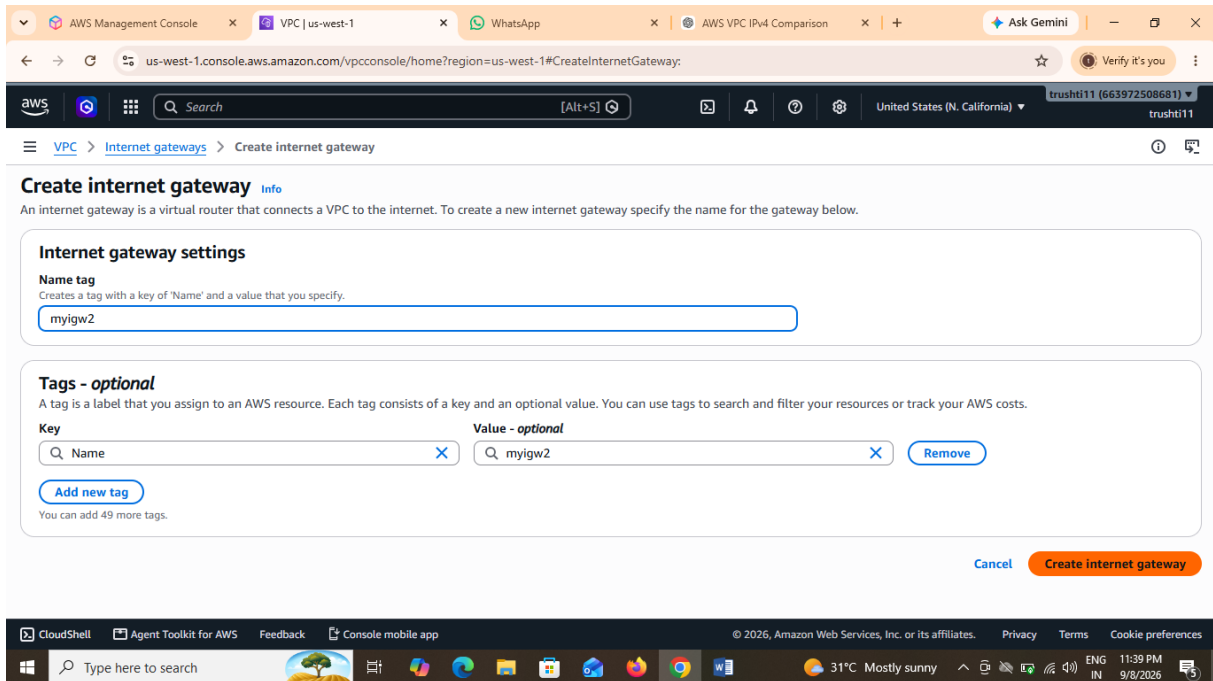


Figure B.7 — AWS Console

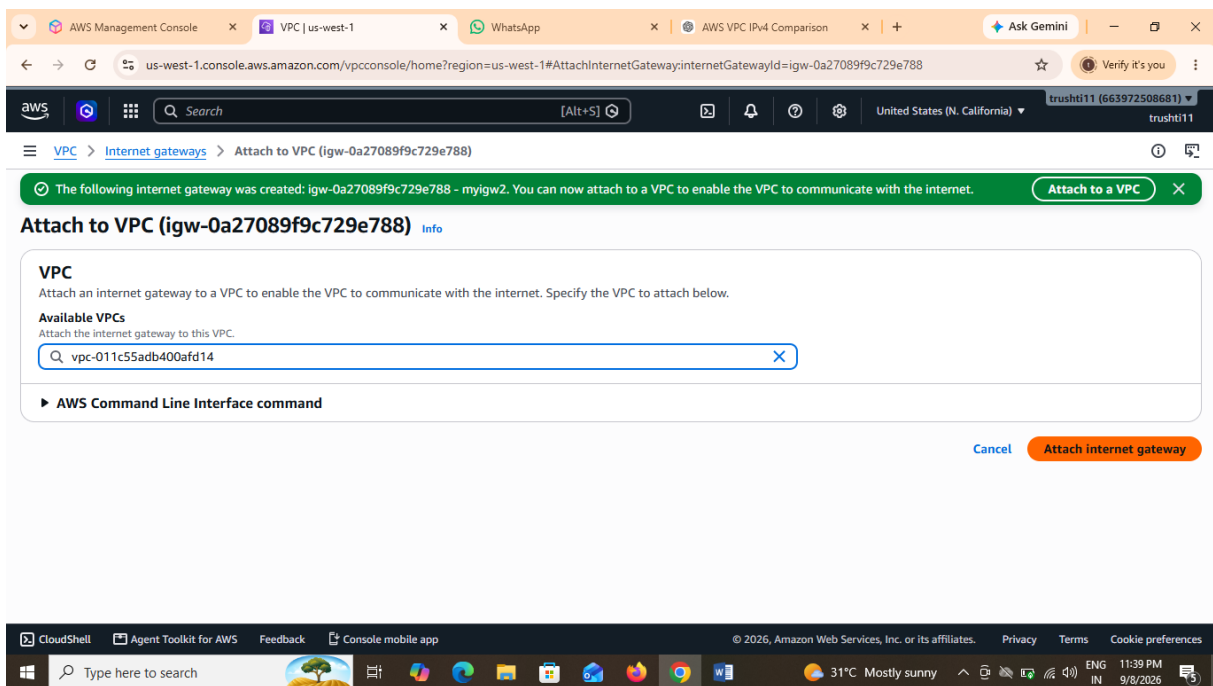


Figure B.8 — AWS Console

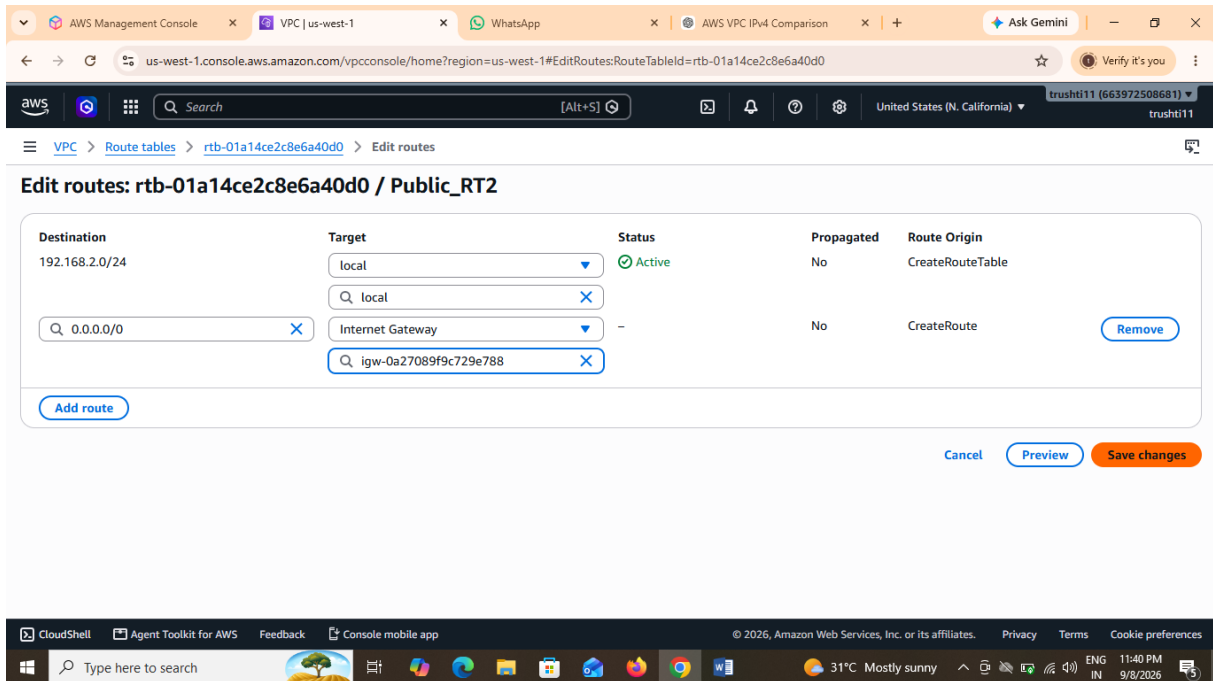


Figure B.9 — AWS Console

Launching EC2 instances in both VPCs with SSH/ICMP security group rules

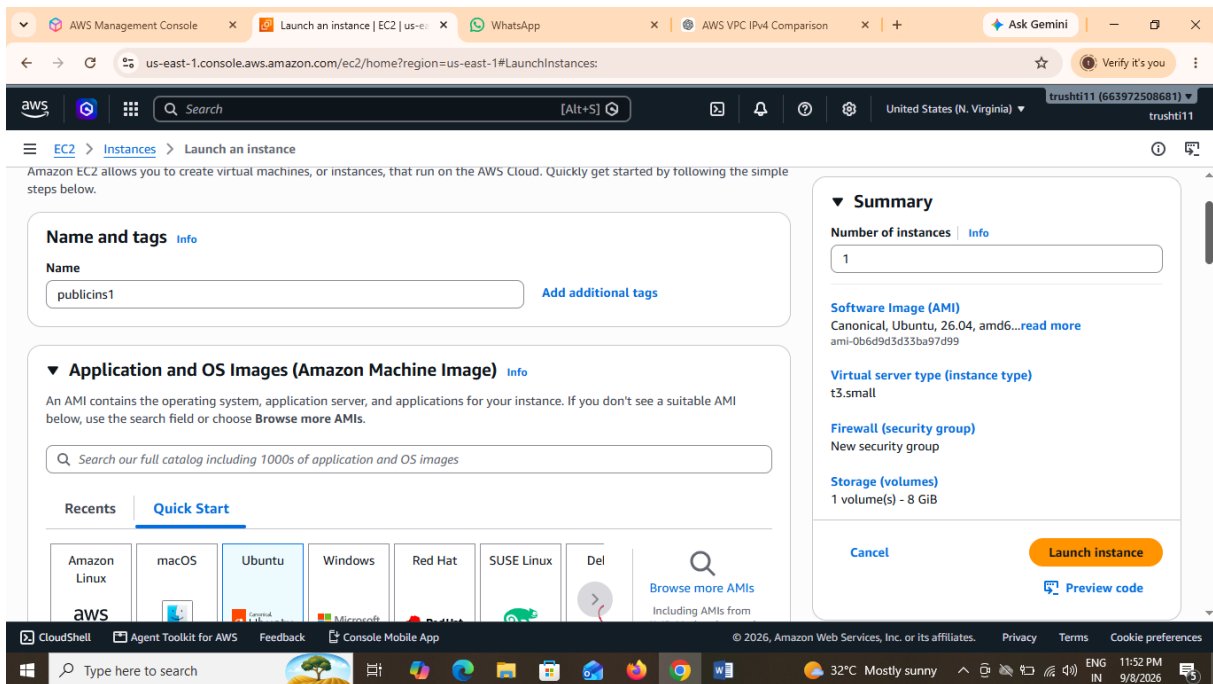


Figure B.1 — AWS Console

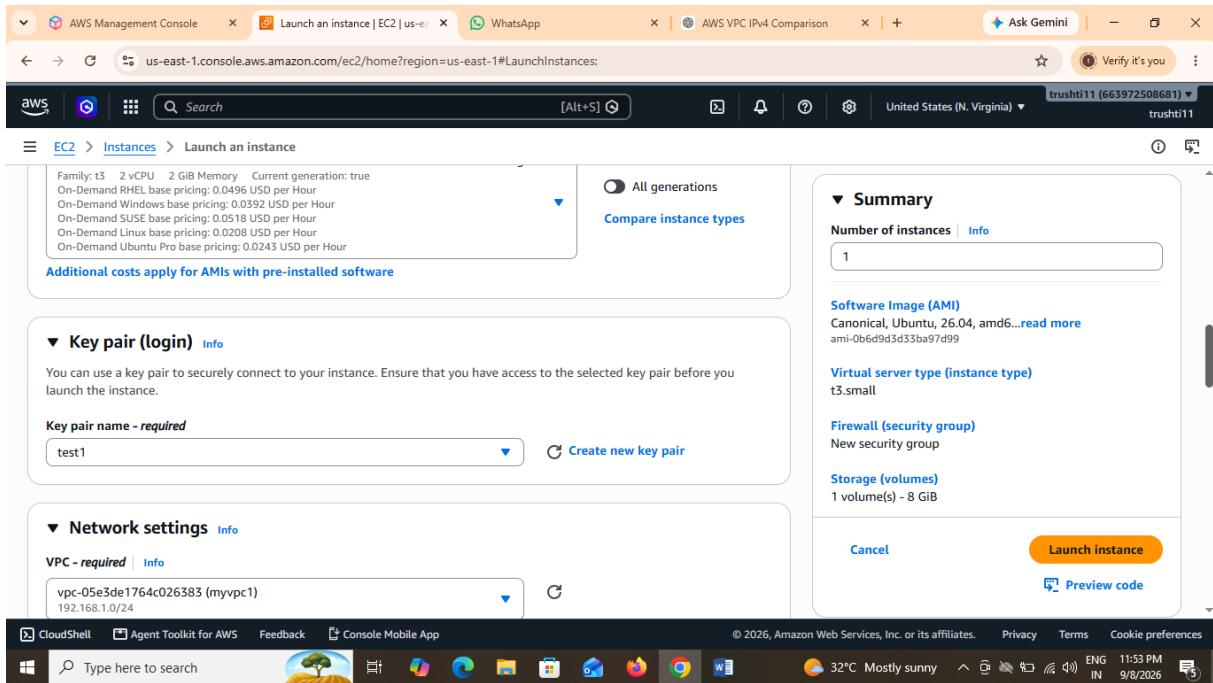


Figure B.2 — AWS Console

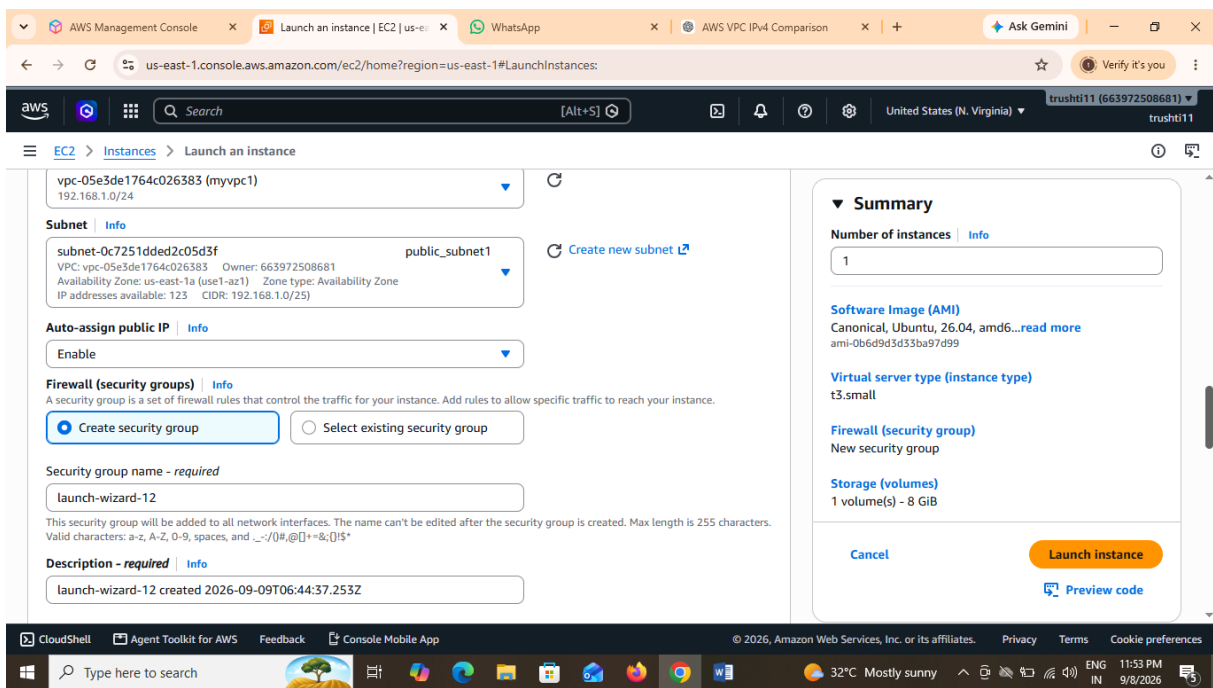


Figure B.3 — AWS Console

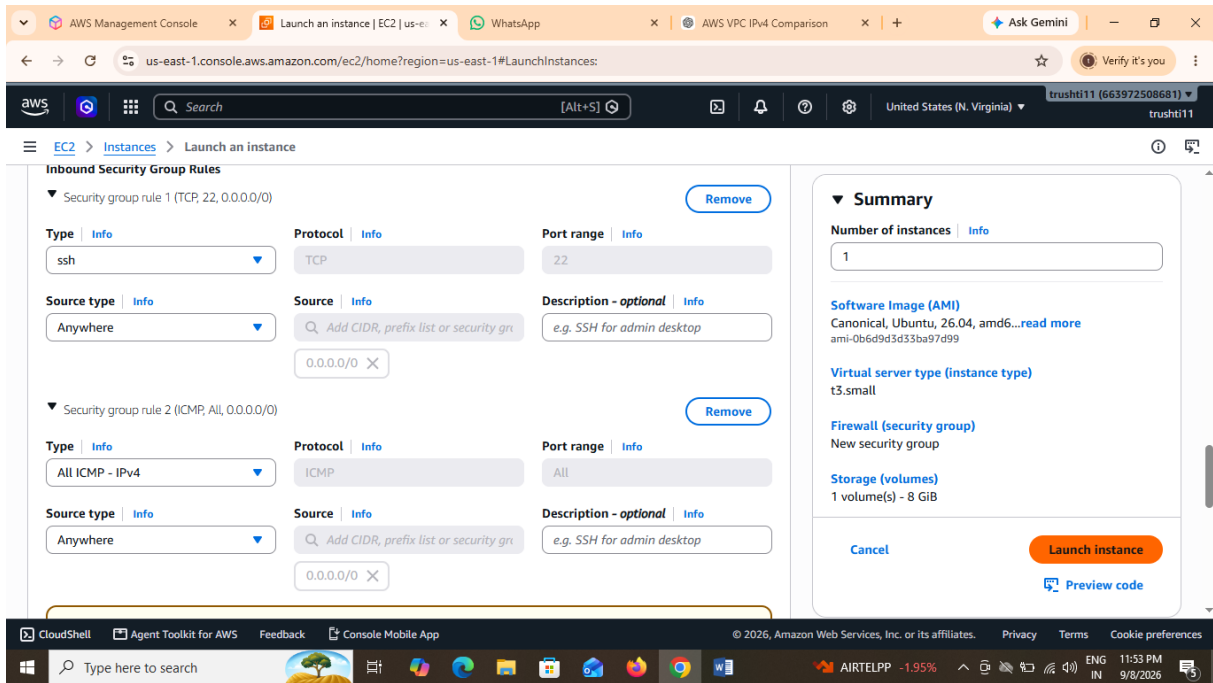


Figure B.4 — AWS Console

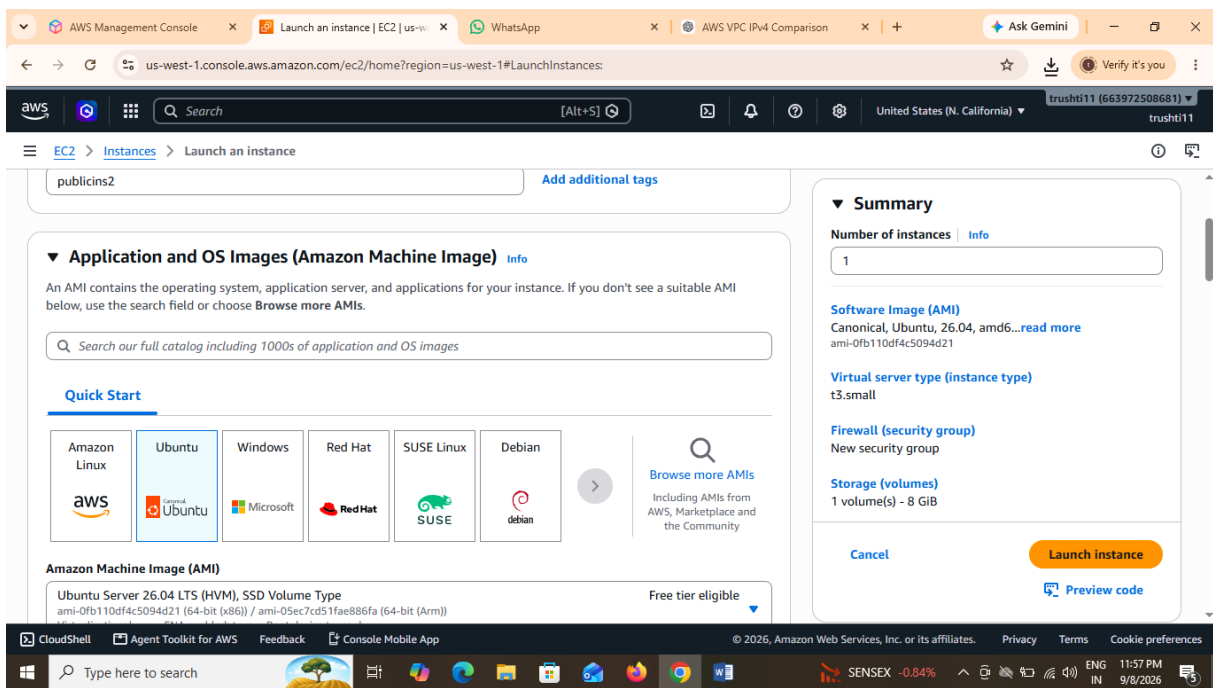


Figure B.5 — AWS Console

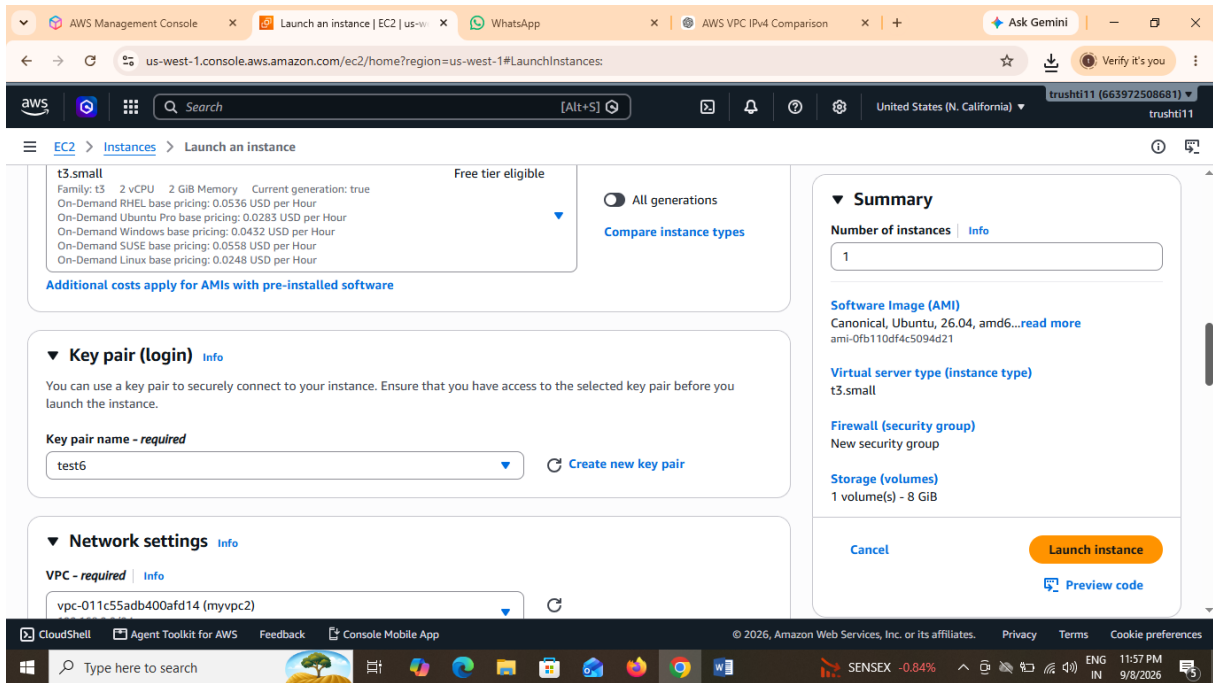


Figure B.6 — AWS Console

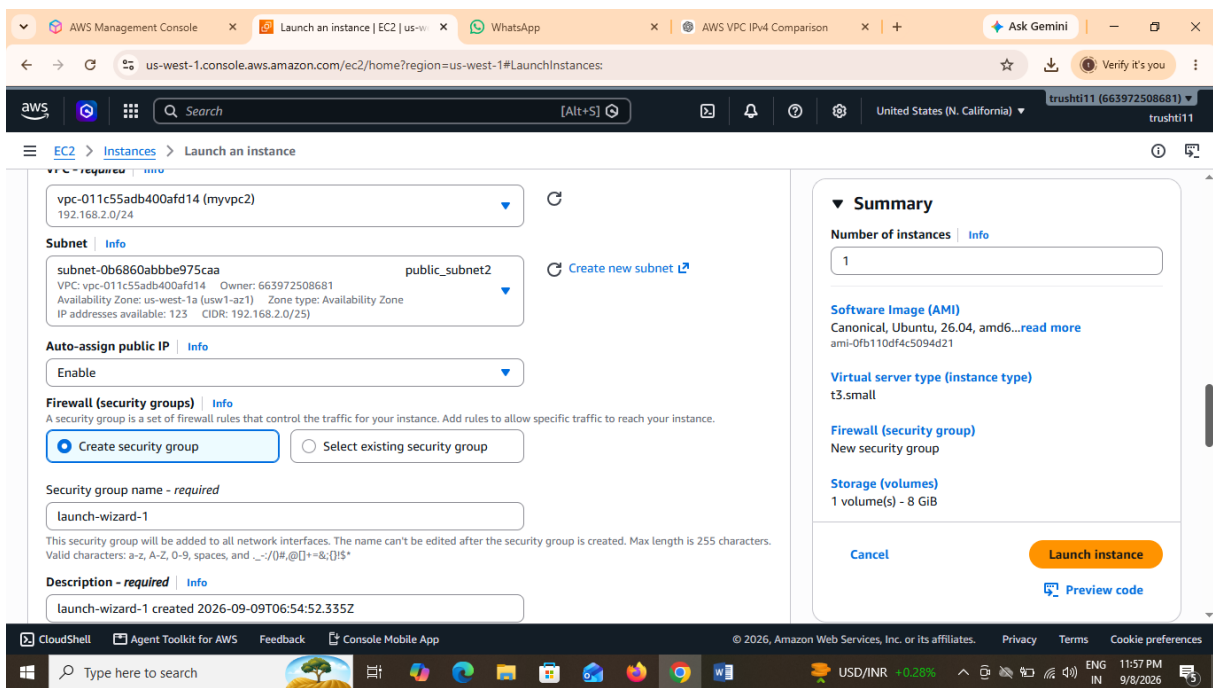


Figure B.7 — AWS Console

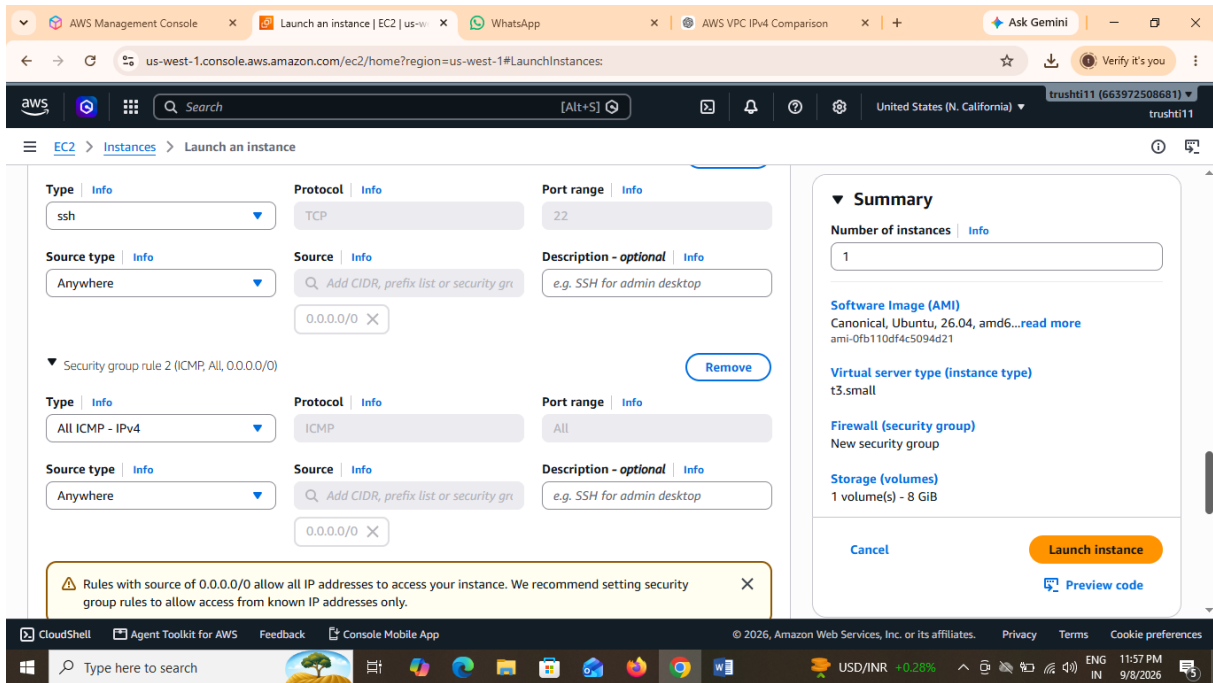


Figure B.8 — AWS Console

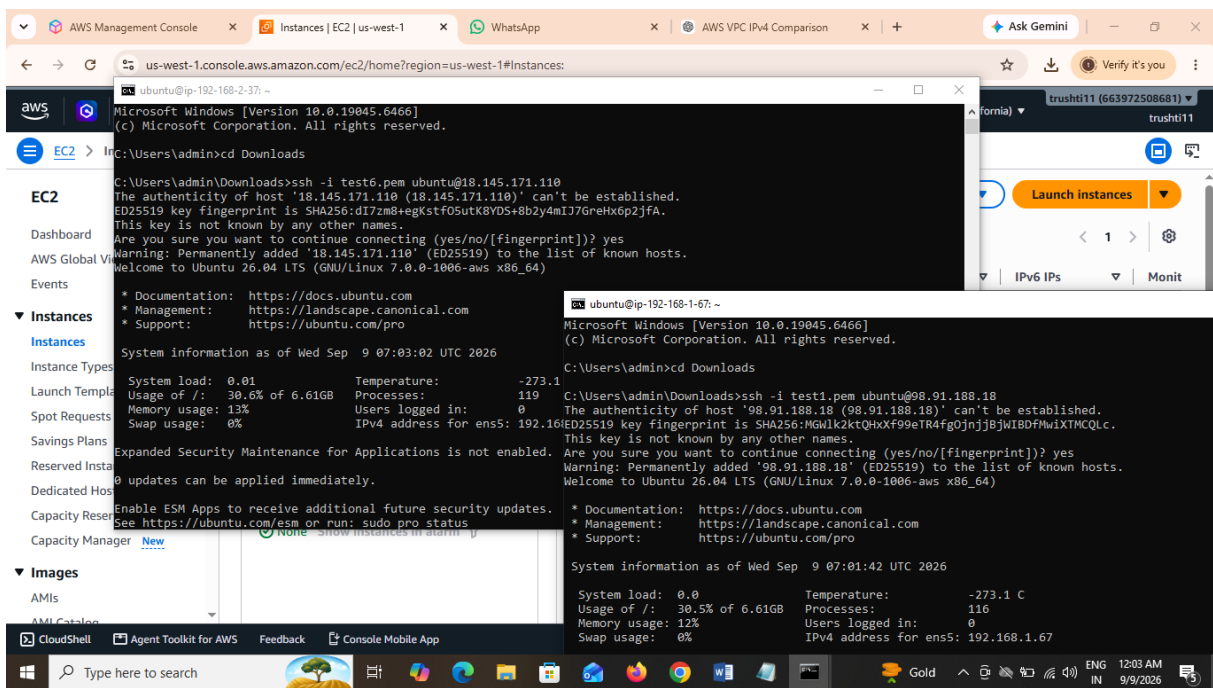


Figure B.9 — AWS Console

SSH connectivity check, then creating the cross-region Peering Connection

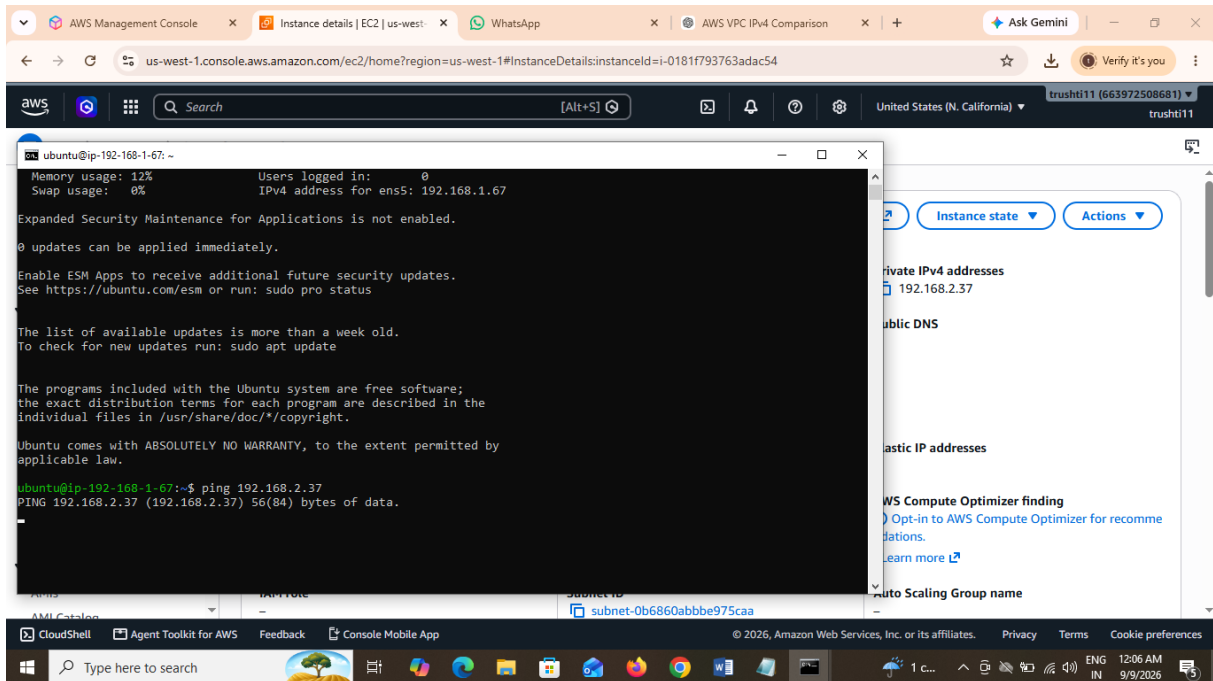


Figure B.1 — AWS Console

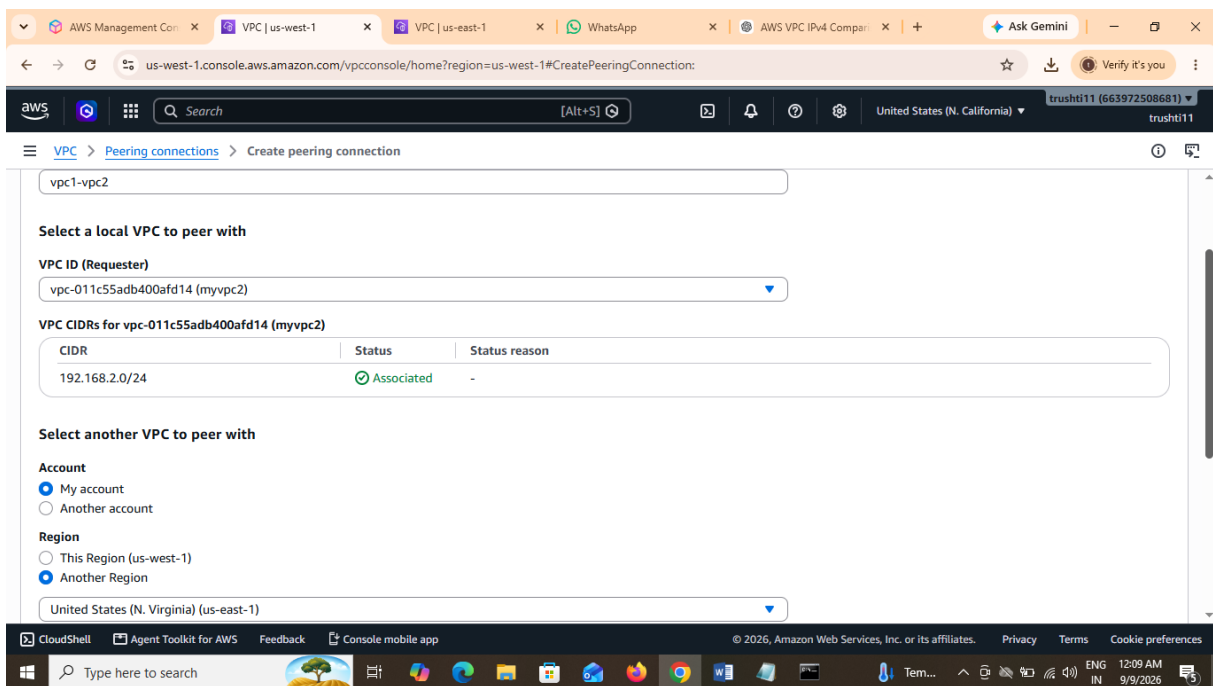


Figure B.2 — AWS Console

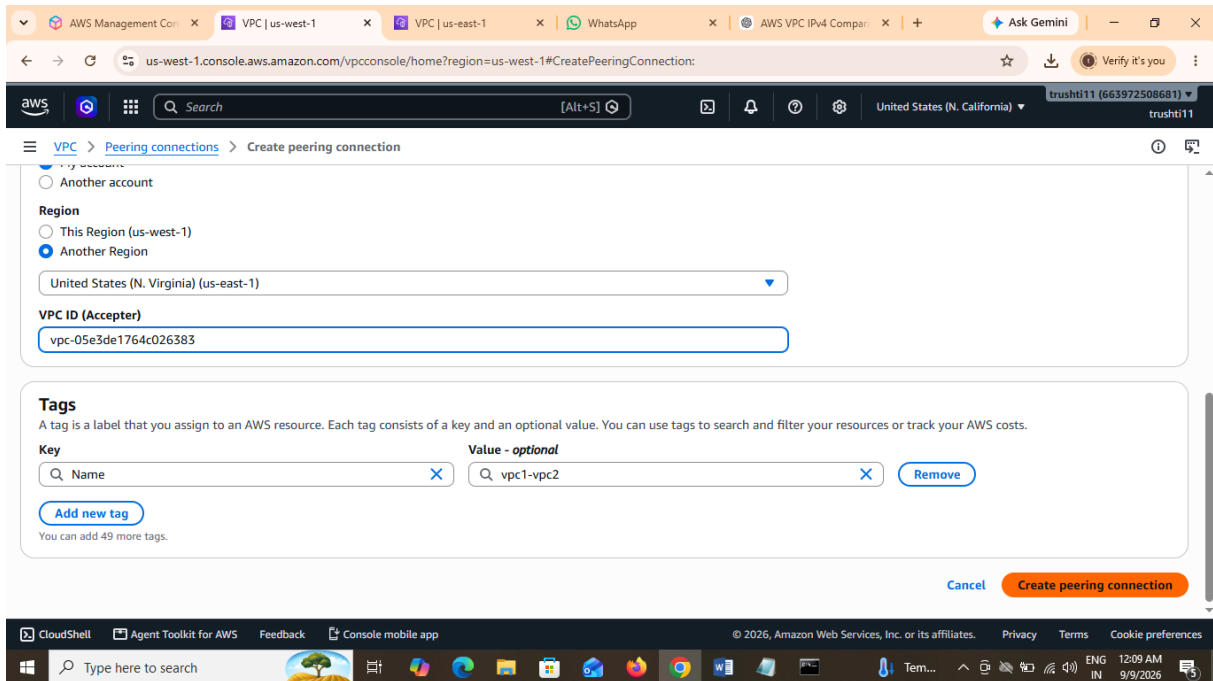


Figure B.3 — AWS Console

Accepting the peering request in the requester's region

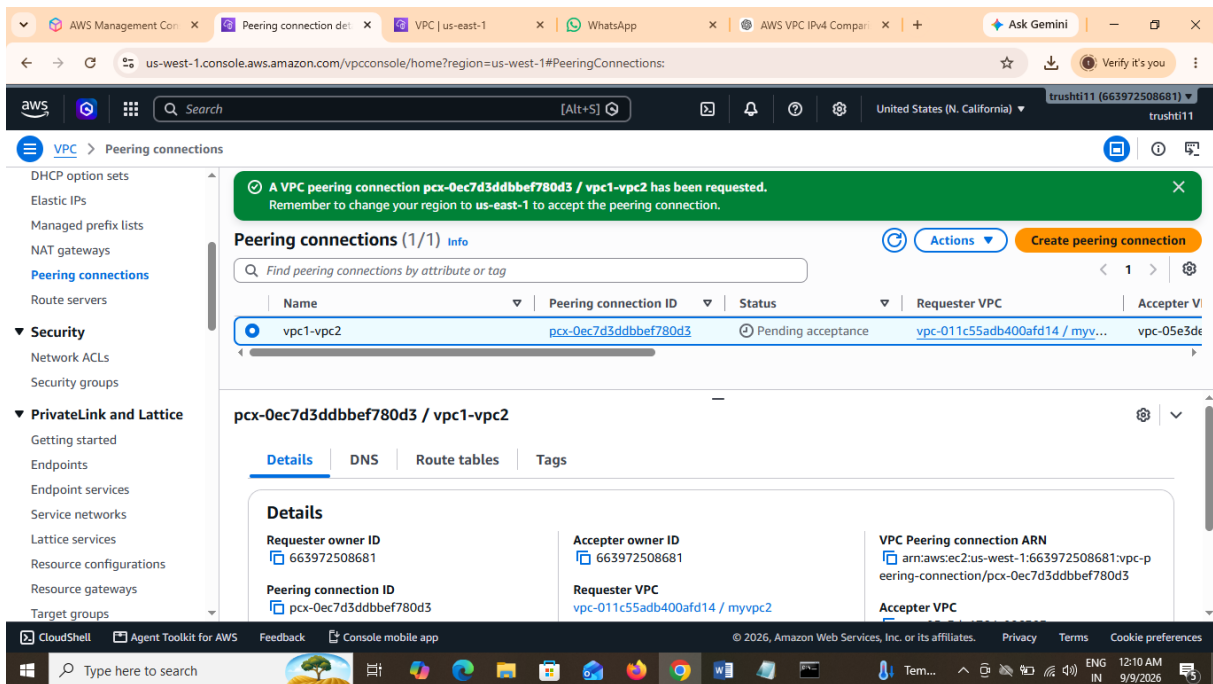


Figure B.1 — AWS Console

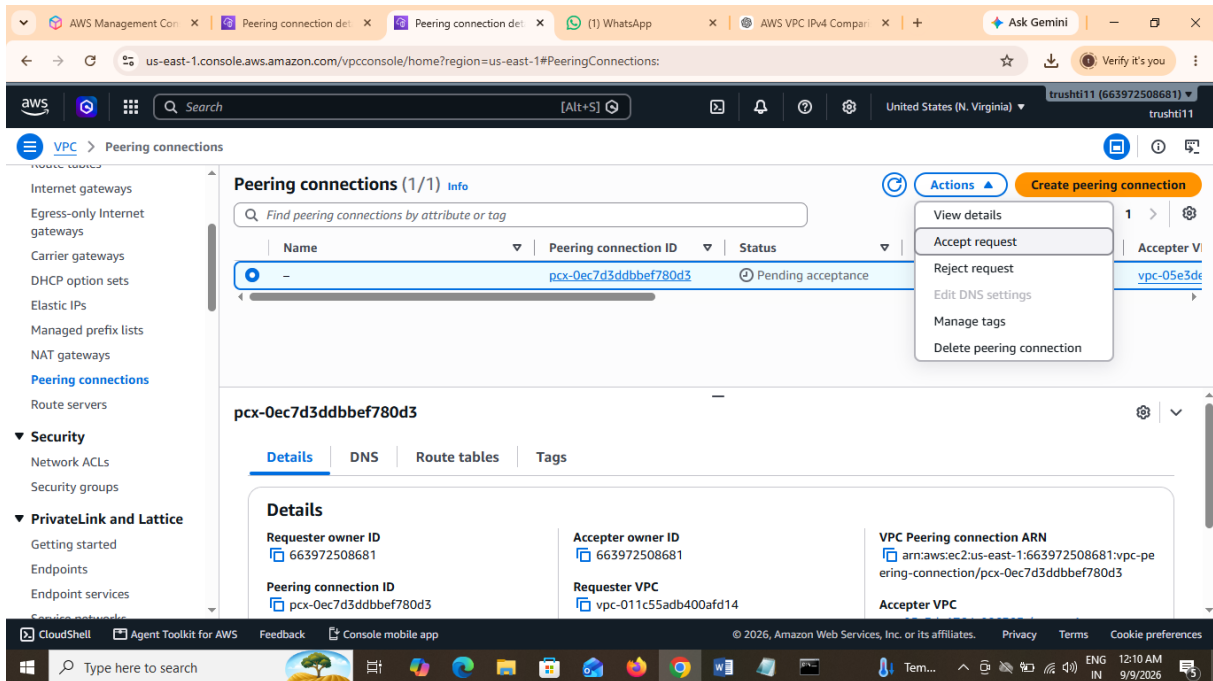


Figure B.2 — AWS Console

Updating route tables in both regions to use the Peering Connection

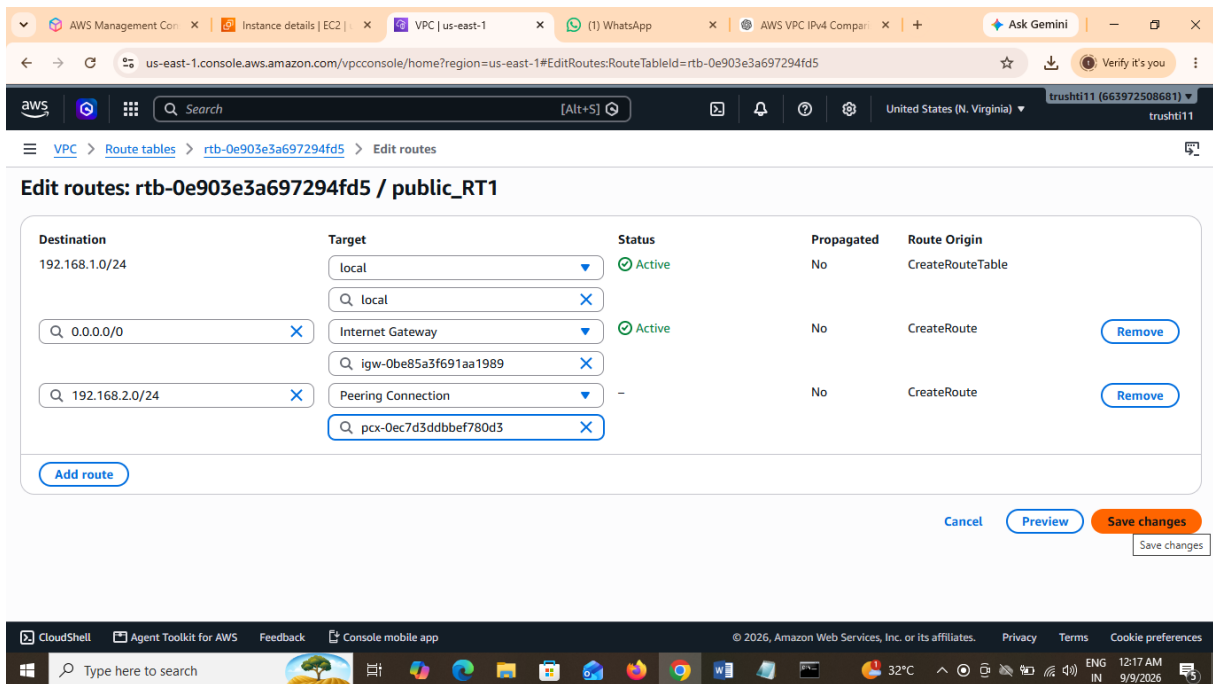


Figure B.1 — AWS Console

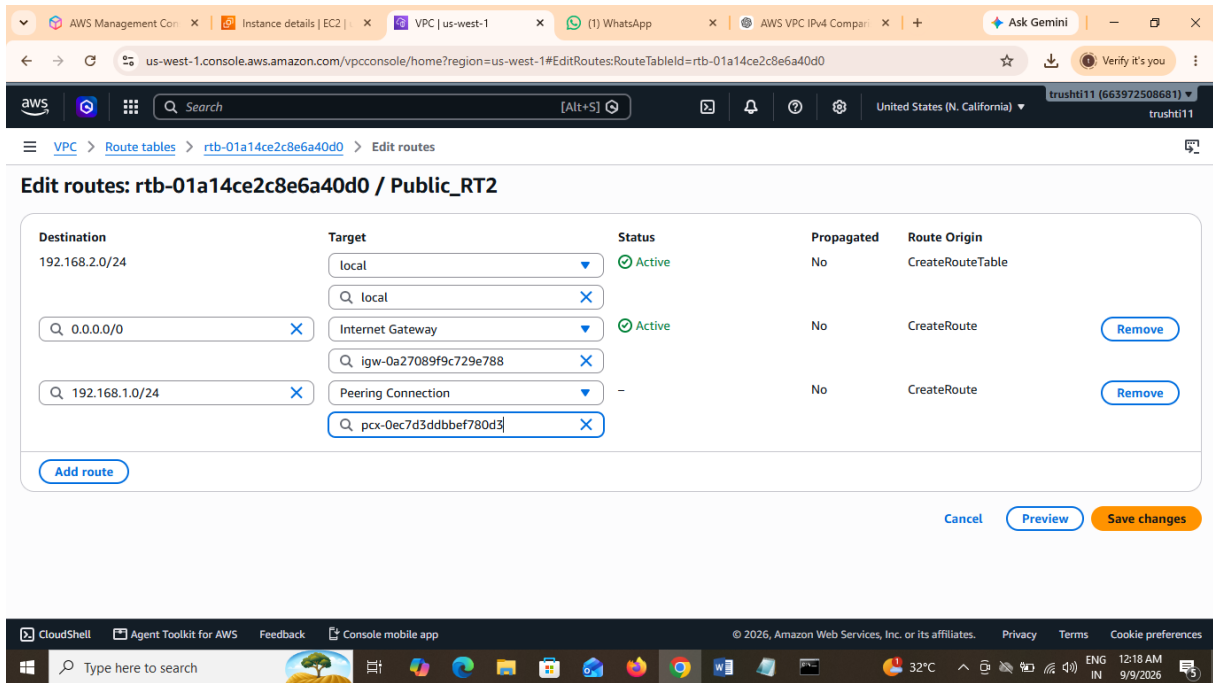


Figure B.2 — AWS Console

Verifying connectivity with ping across regions

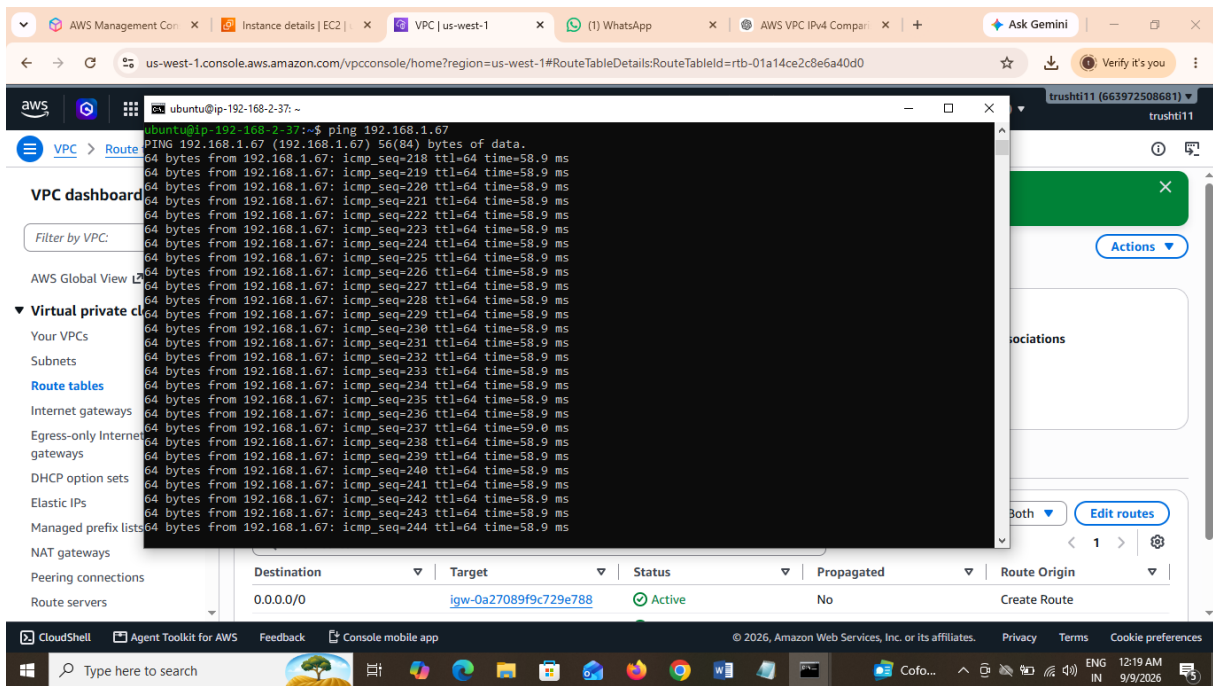


Figure B.1 — AWS Console

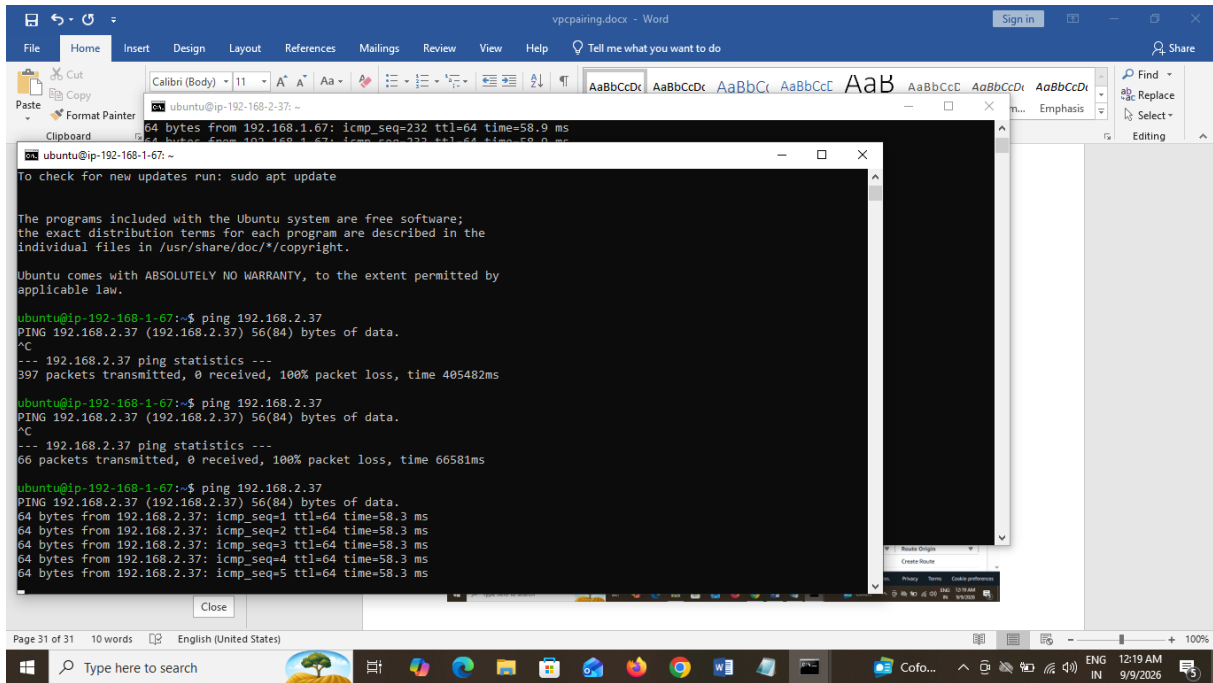


Figure B.2 — AWS Console

4. Part C — VPC Peering across Different AWS Accounts

4.1 Theory

Cross-account VPC peering connects a VPC in one AWS account to a VPC in a completely different AWS account (and, optionally, a different region at the same time). This is common in multi-account setups — for example, a central networking/shared-services account peering with several application accounts, or two separate companies connecting their environments.

- The requester must know the acceptor's 12-digit AWS Account ID (and Region, if it differs from the requester's).
- The peering request is created from the requester's account but must be reviewed and accepted by an authorized user logged into the acceptor's AWS account — it will appear under that account's VPC → Peering Connections console.
- As with the other scenarios, CIDR blocks must not overlap, and route tables on both sides must be updated independently by their respective account owners.
- Security groups can reference the peer VPC's CIDR block (or, within the same account, even the peer security group ID) to control exactly which traffic is allowed over the connection.

4.2 Lab Topology

- Account A: VPC "myvpc1" — CIDR 192.168.1.0/24 — with a public subnet (192.168.1.0/25, instance "publicins") and a private subnet (192.168.1.128/25, instance "privateins"), each with its own route table (Public_RT / Private_RT).
- Account B (Account ID ending consistent with the lab, region Europe/Stockholm): a VPC with CIDR 10.0.0.0/16.
- A cross-account (and cross-region) peering connection is created from Account A to Account B, accepted in Account B, and wired into the route tables of Account A so that instances can reach the 10.0.0.0/16 network.

4.3 Steps Performed

1. In Account A, create VPC "myvpc1" (192.168.1.0/24) with a public subnet and a private subnet.
2. Create a route table for each subnet (Public_RT, Private_RT) and associate them accordingly.
3. Create an Internet Gateway, attach it to the VPC, and add a default route (0.0.0.0/0) in the public route table.
4. Launch an EC2 instance in the public subnet and another in the private subnet, each with a security group allowing SSH and ICMP.
5. Download the key pair (.pem) and connect to the public instance over SSH to confirm it is reachable.
6. Create the VPC Peering Connection: select the local VPC as requester, set Account to "Another account" and enter the peer's 12-digit Account ID, set Region to "Another Region" if the peer VPC is in a different region, and enter the acceptor VPC ID.

- 7. Sign in to the peer AWS account (Account B), open VPC → Peering Connections, select the pending request, and choose Actions → Accept Request.
- 8. Back in Account A, edit the Private_RT and Public_RT route tables to add a route for the peer VPC's CIDR (10.0.0.0/16) targeting the Peering Connection.
- 9. In Account B, the account owner similarly updates their own route table with a route back to 192.168.1.0/24 via the same Peering Connection.
- 10. Test the connection by pinging an instance's private IP address in the peer account's VPC from an instance in Account A.

4.4 Screenshots

Creating the VPC, subnets, route tables and associations (Account A)

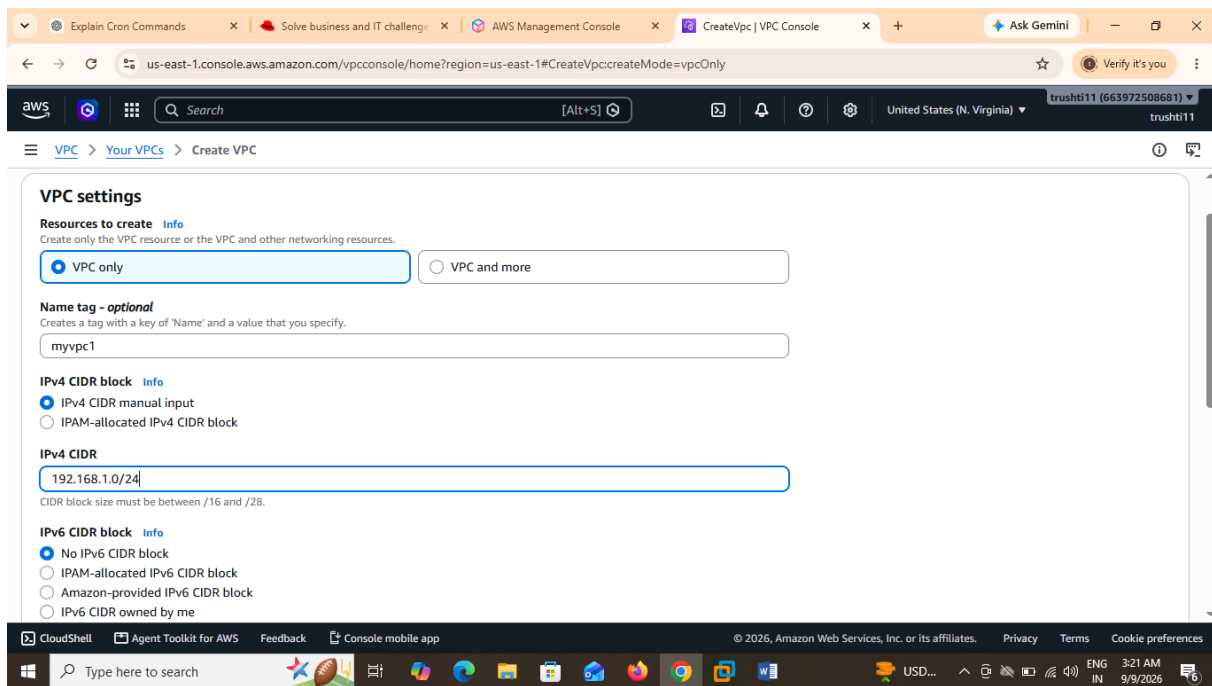


Figure C.1 — AWS Console

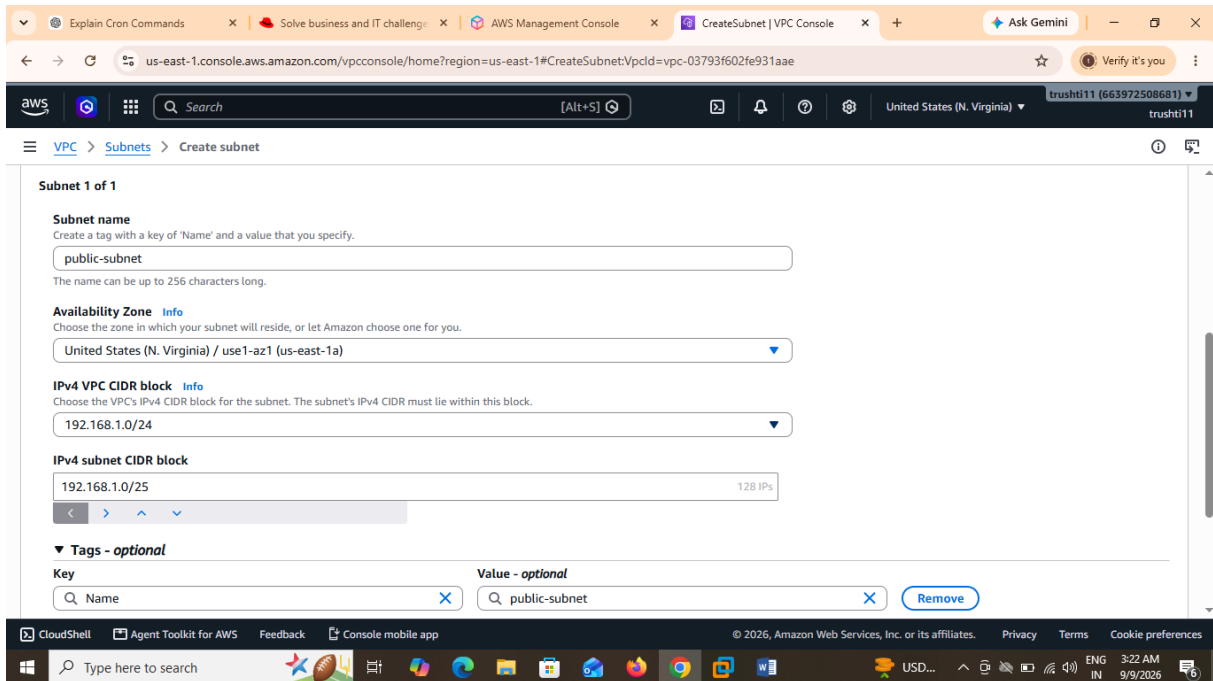


Figure C.2 — AWS Console

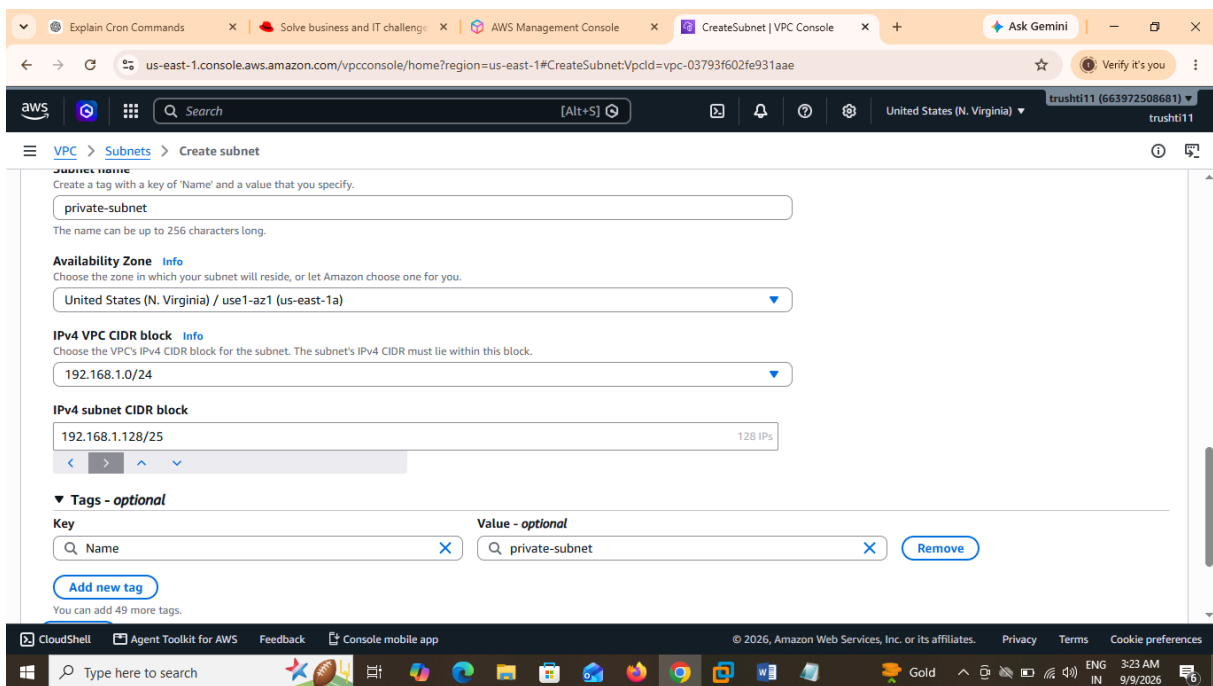


Figure C.3 — AWS Console

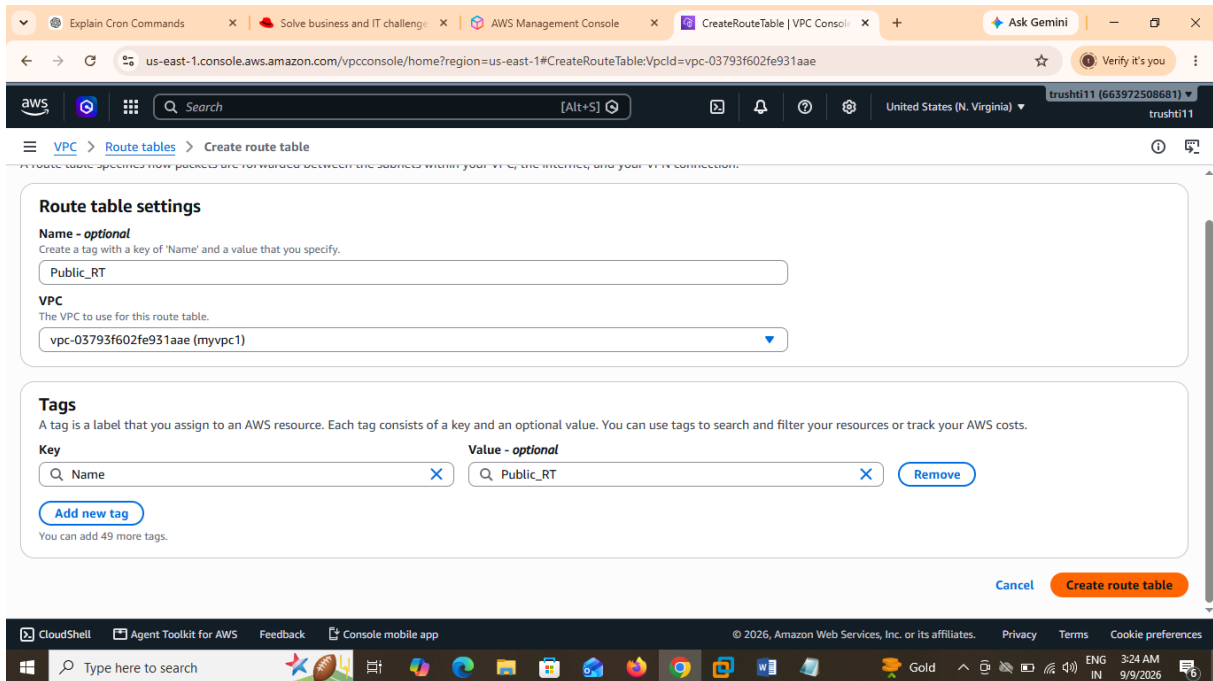


Figure C.4 — AWS Console

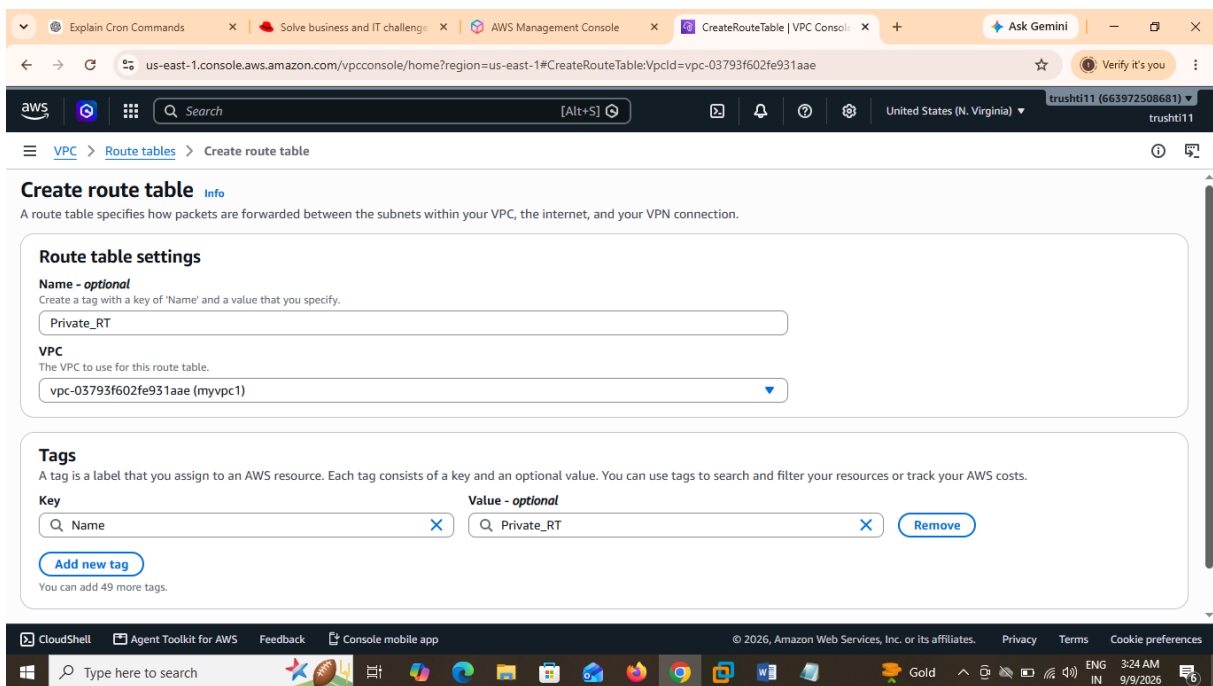


Figure C.5 — AWS Console

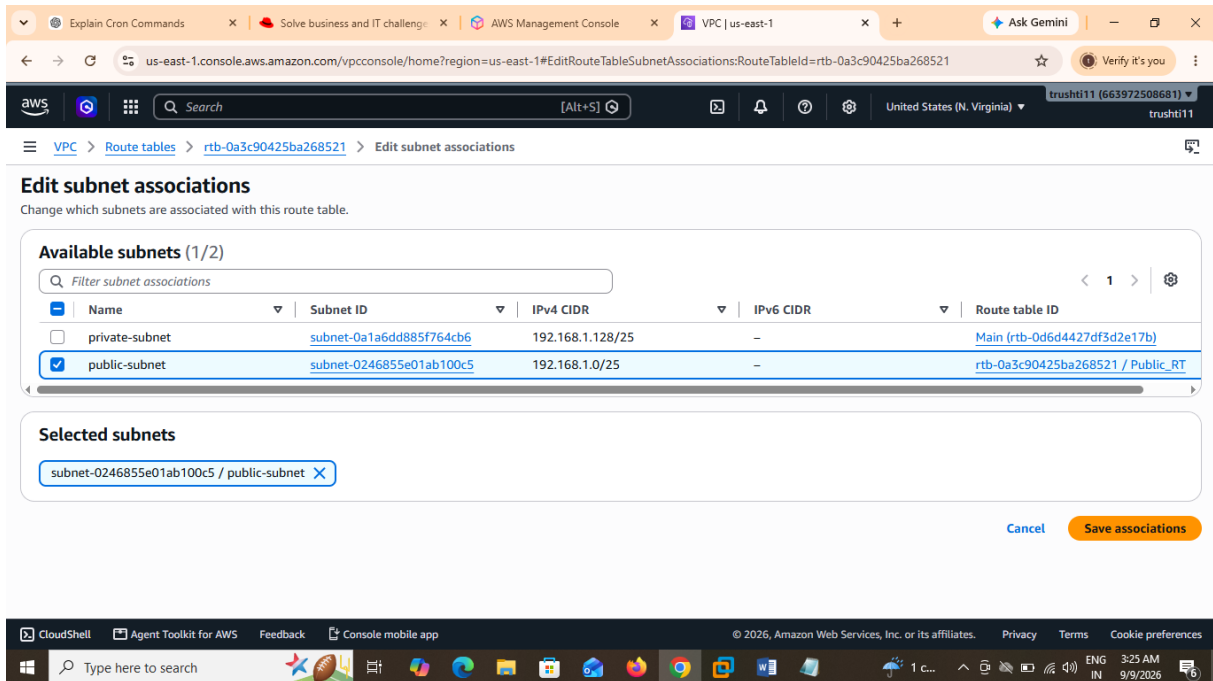


Figure C.6 — AWS Console

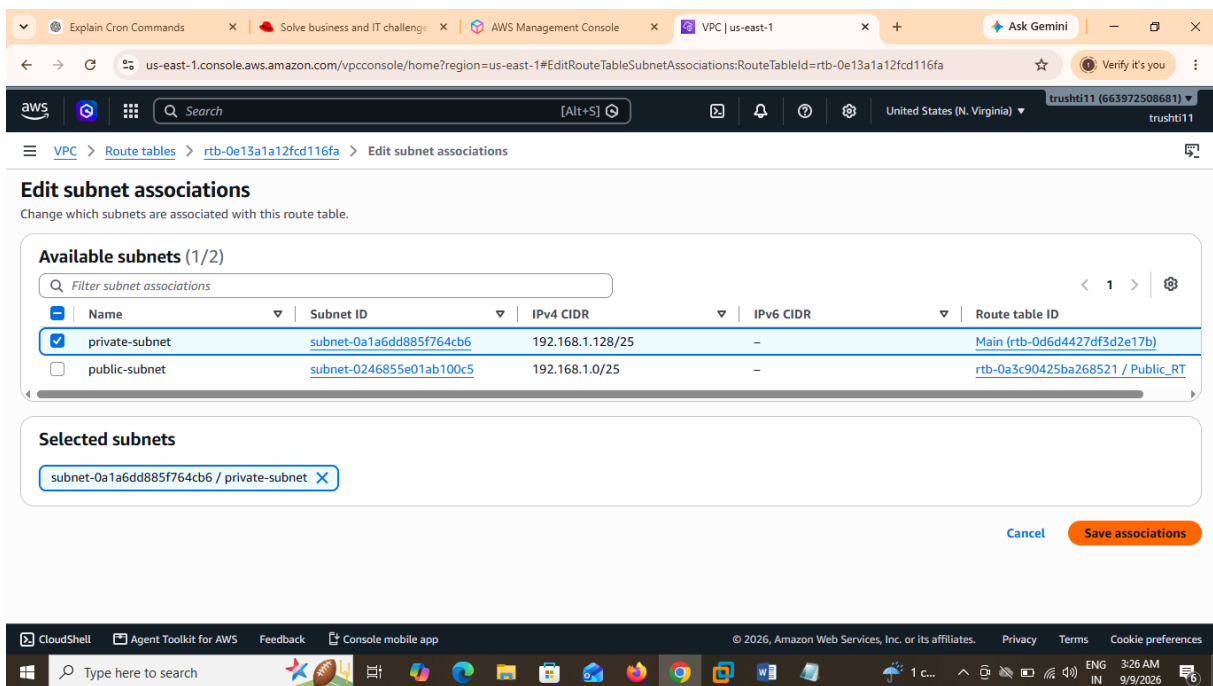


Figure C.7 — AWS Console

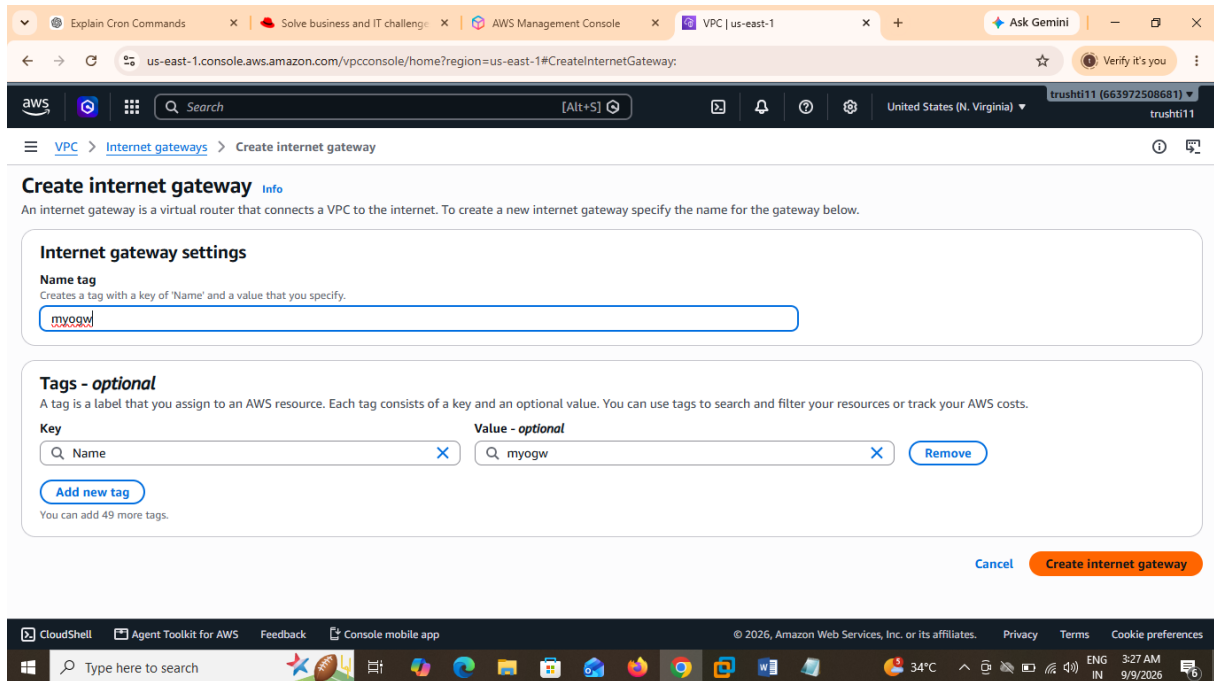


Figure C.8 — AWS Console

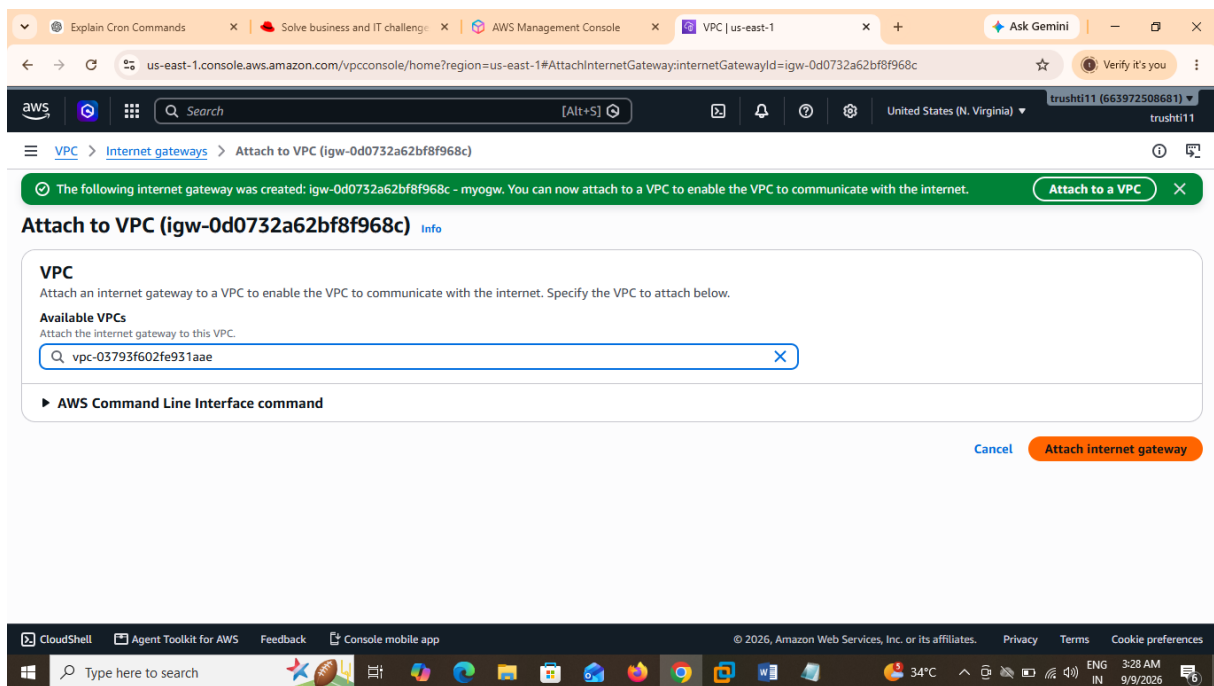


Figure C.9 — AWS Console

Internet Gateway, default route, and launching EC2 instances

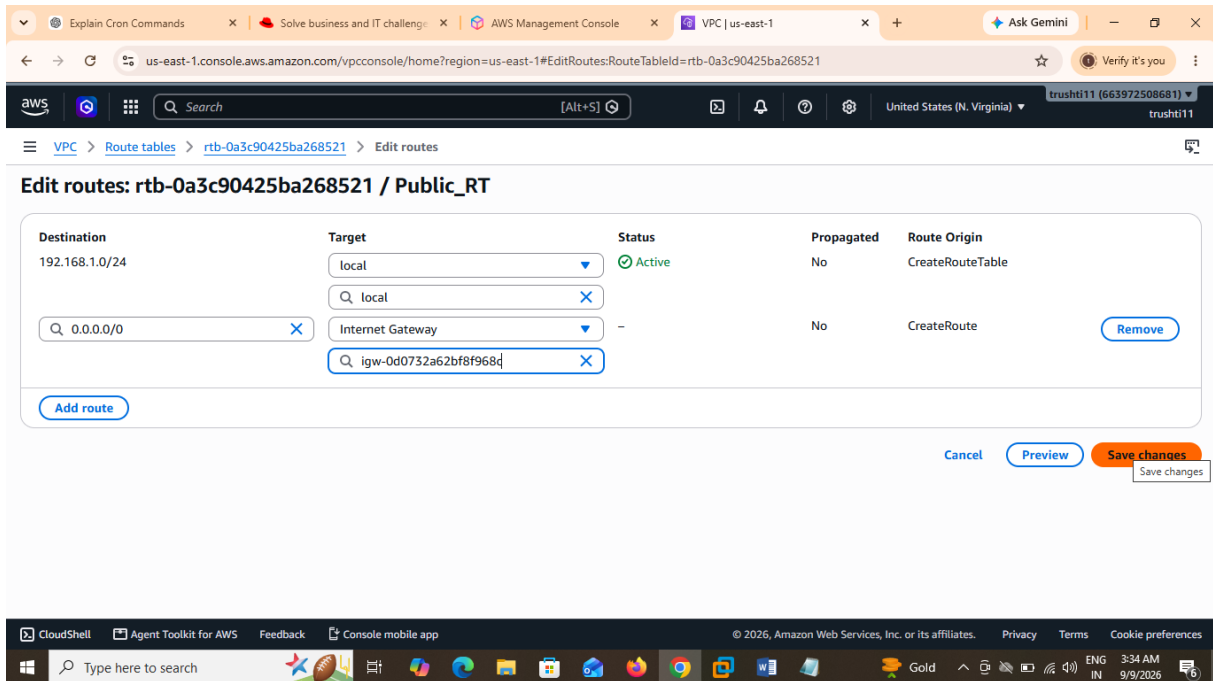


Figure C.1 — AWS Console

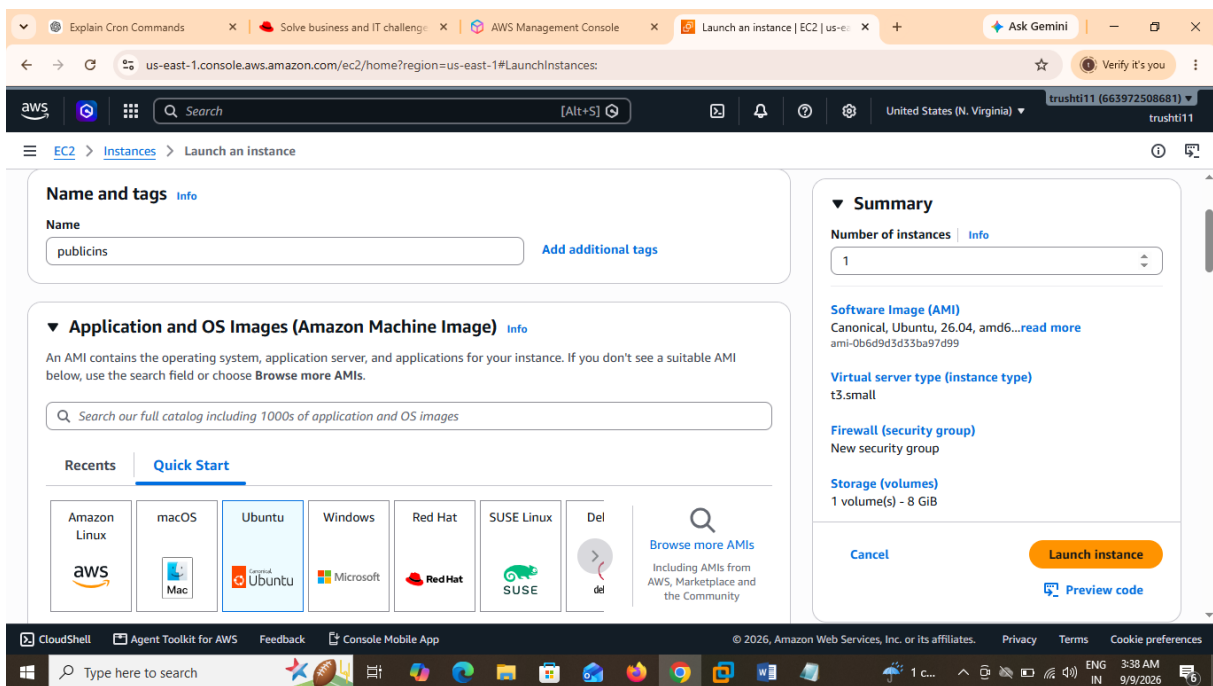


Figure C.2 — AWS Console

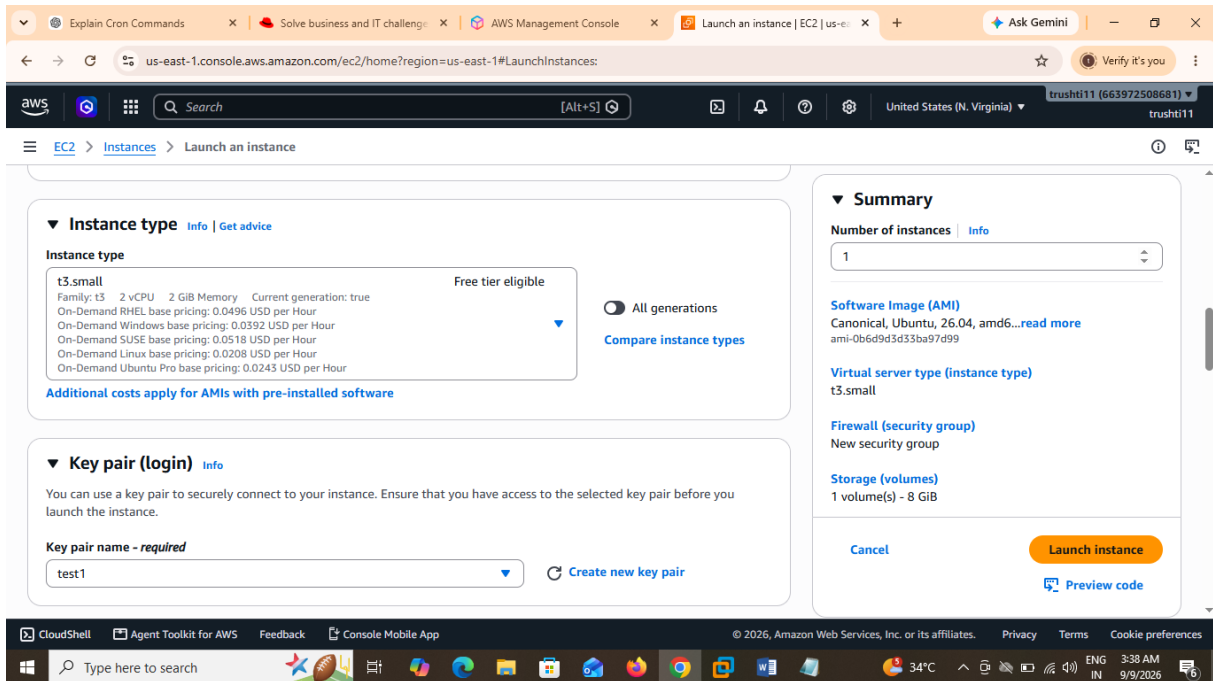


Figure C.3 — AWS Console

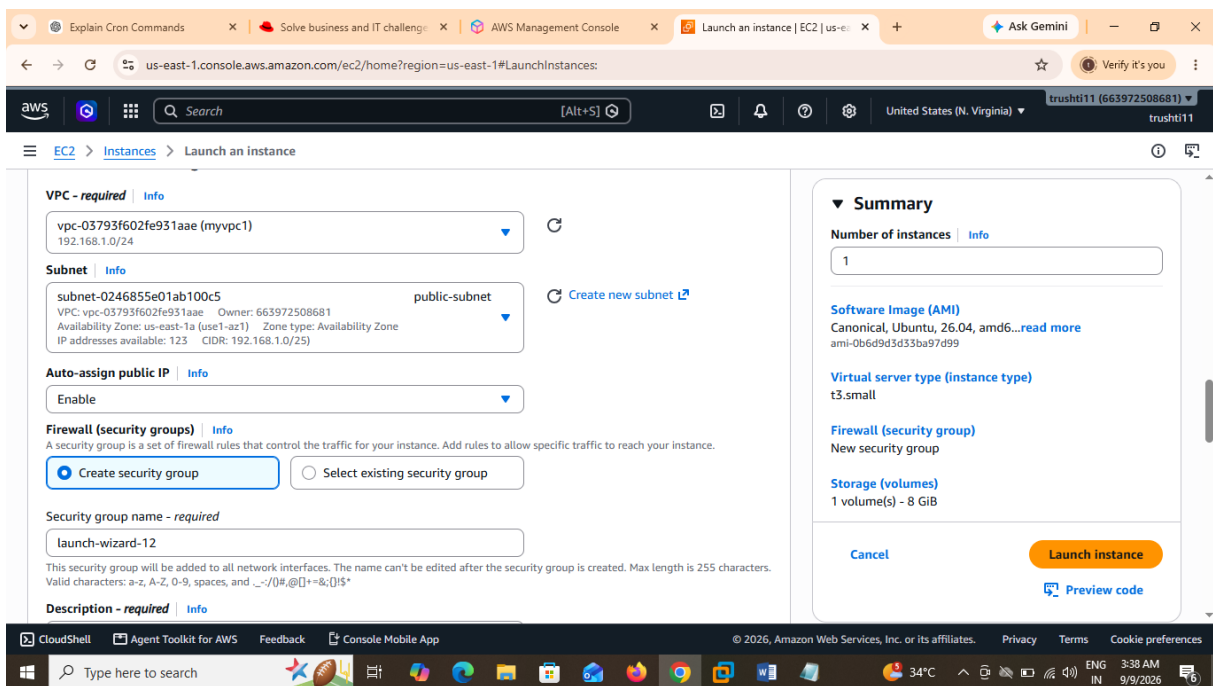


Figure C.4 — AWS Console

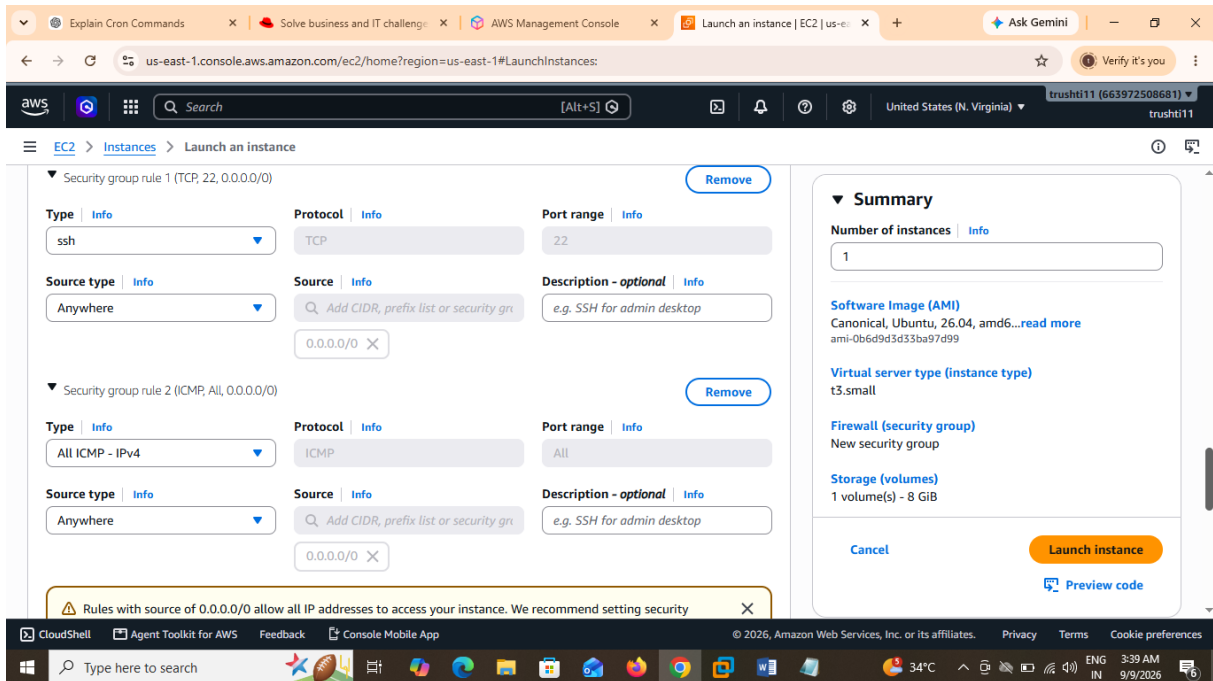


Figure C.5 — AWS Console

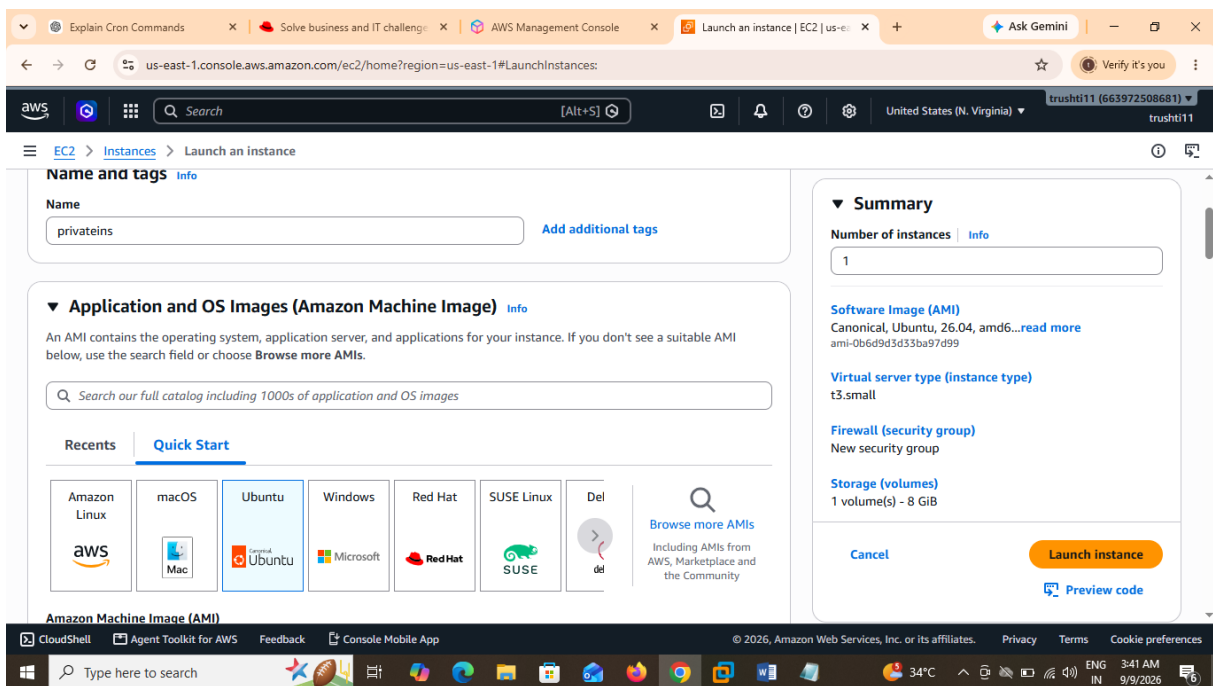


Figure C.6 — AWS Console

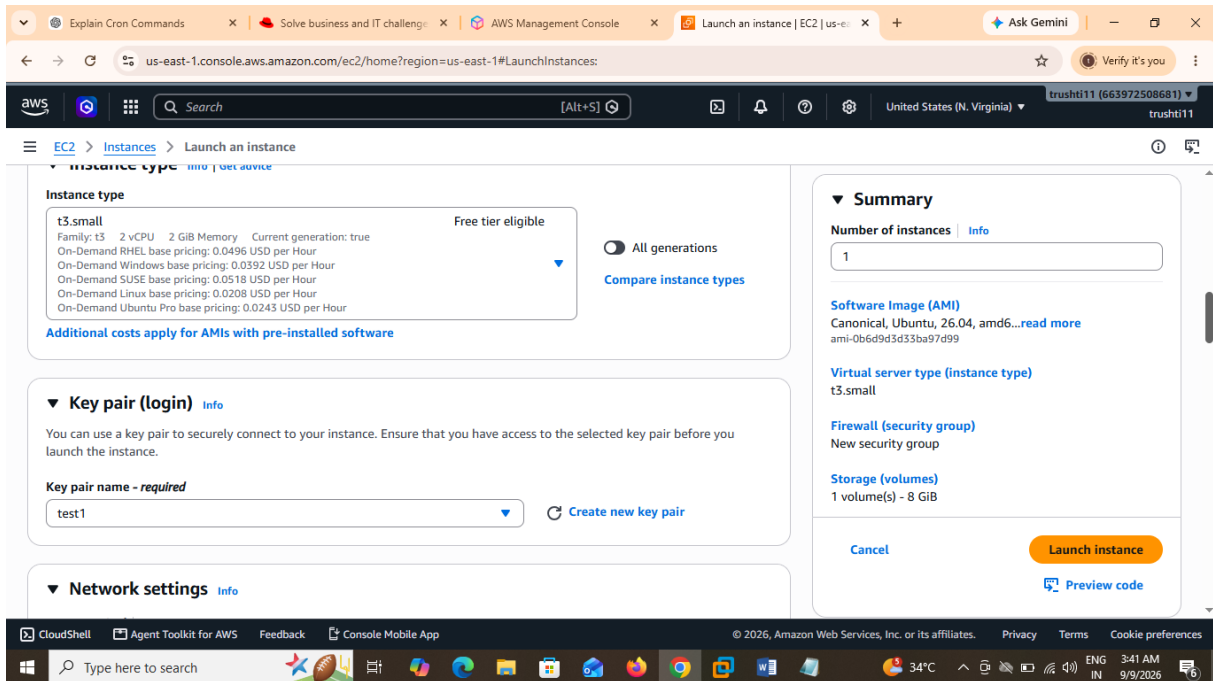


Figure C.7 — AWS Console

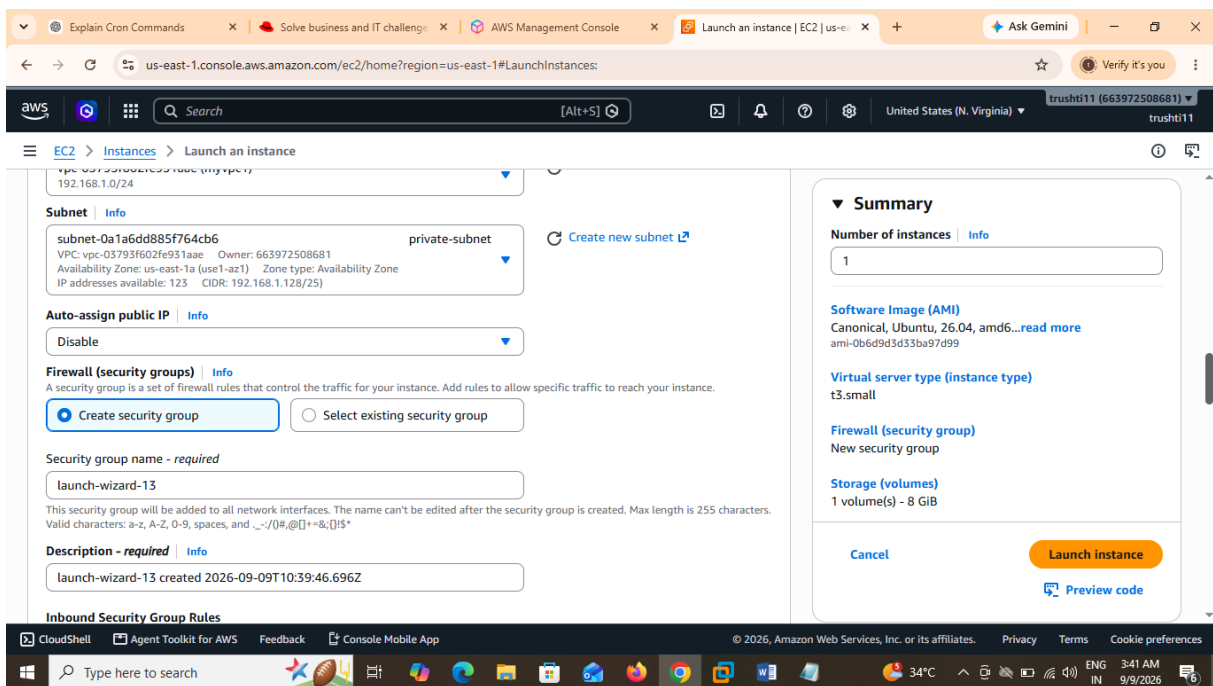


Figure C.8 — AWS Console

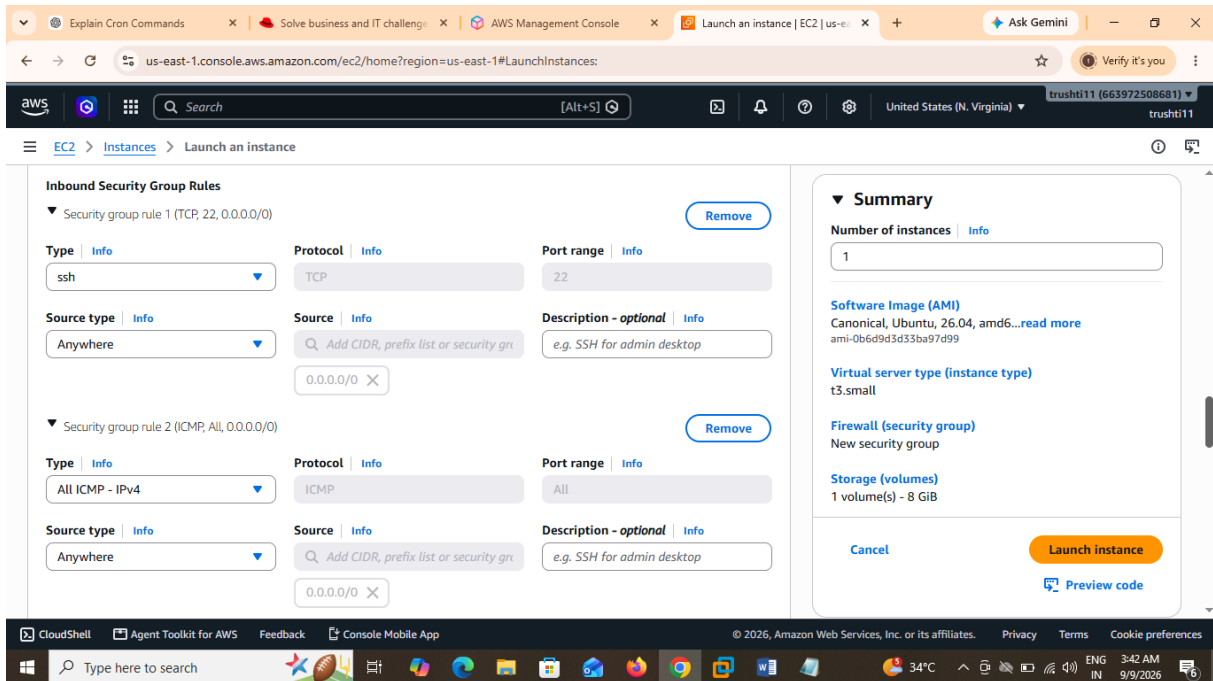


Figure C.9 — AWS Console

SSH verification, creating the cross-account Peering Connection, updating route tables, and testing connectivity

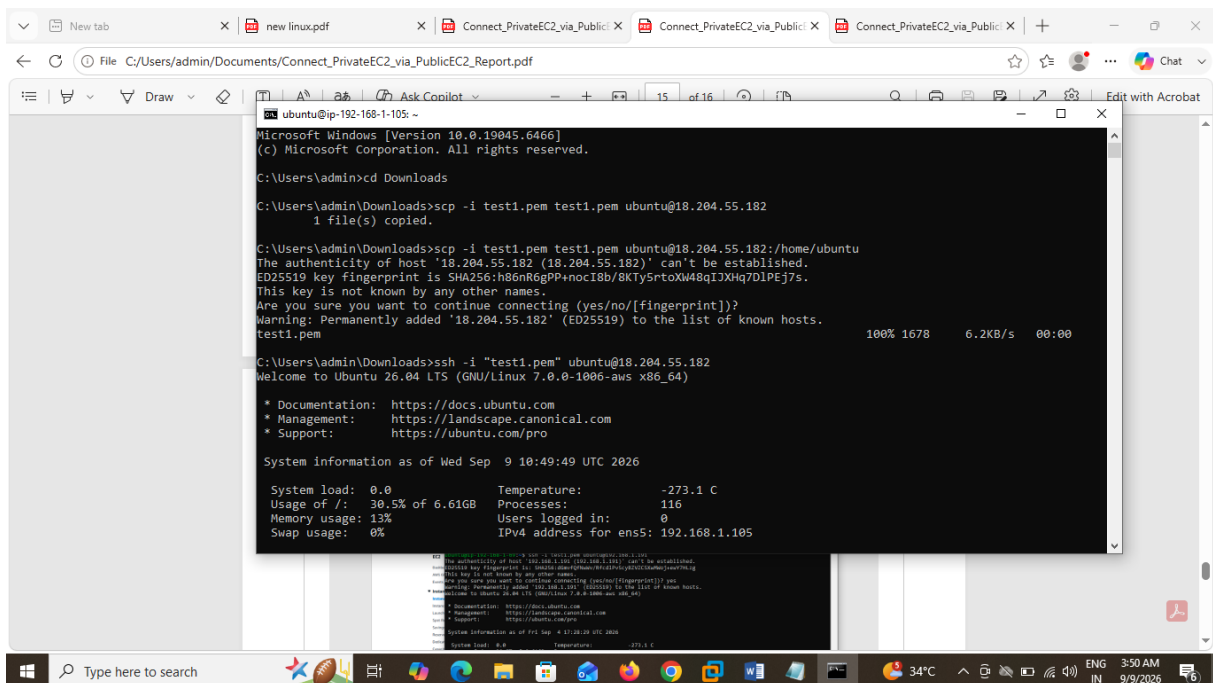


Figure C.1 — AWS Console

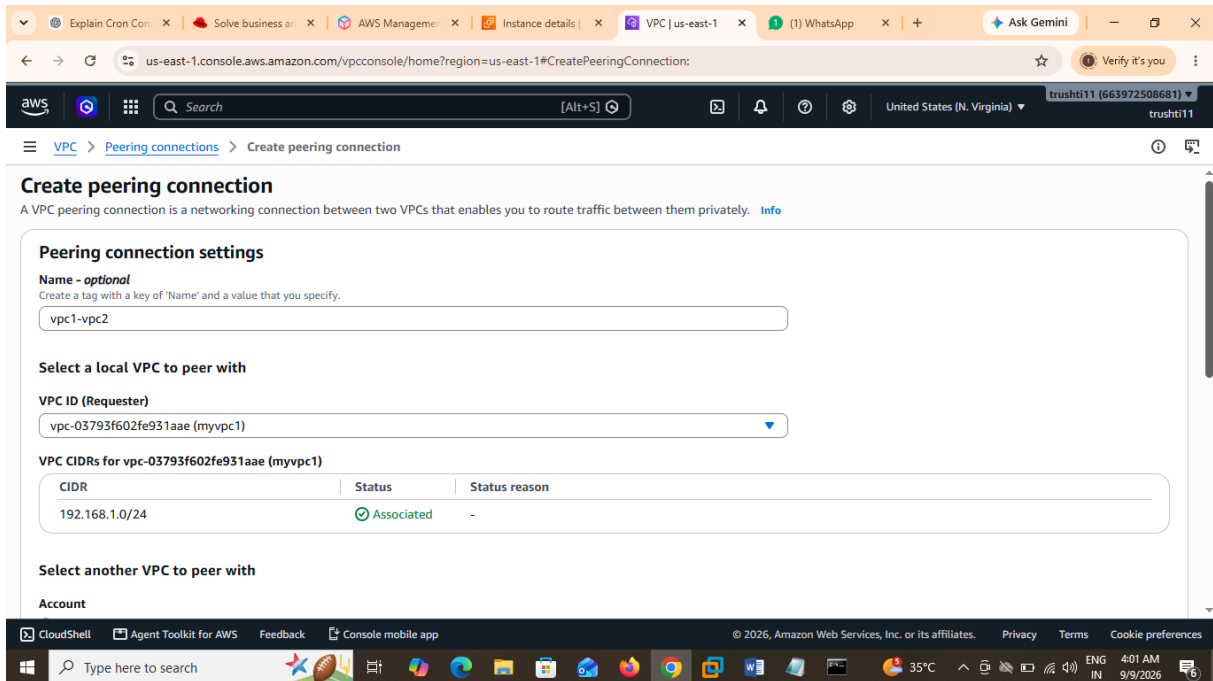


Figure C.2 — AWS Console

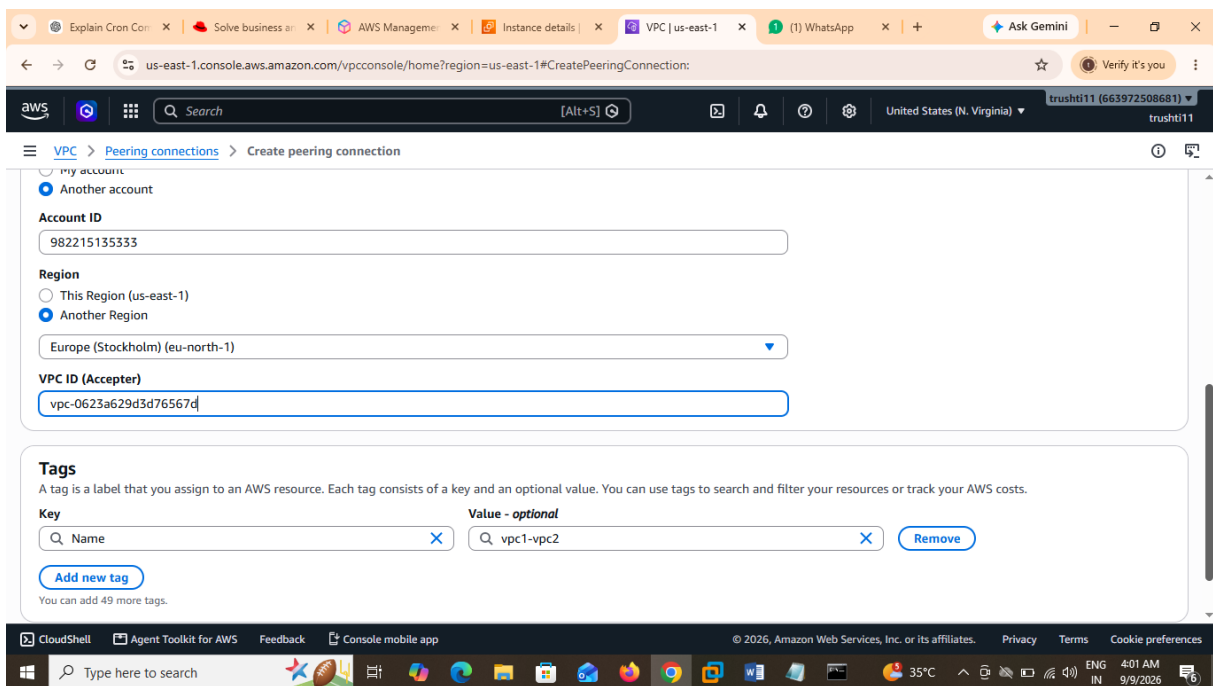
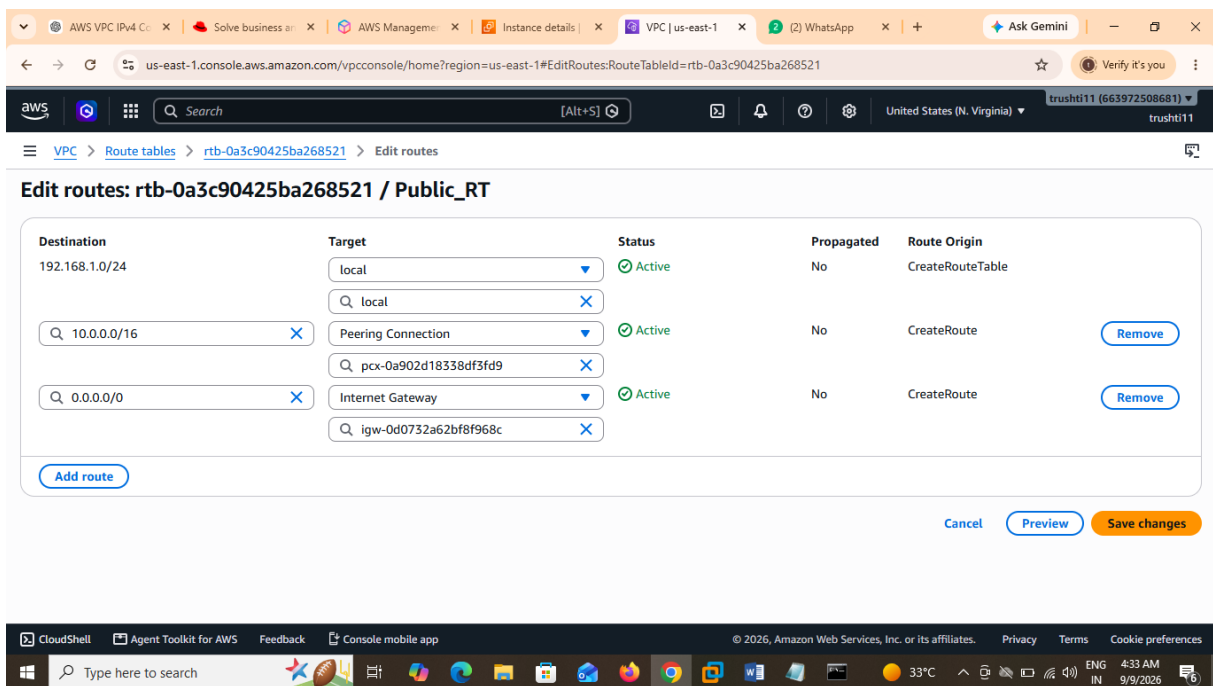
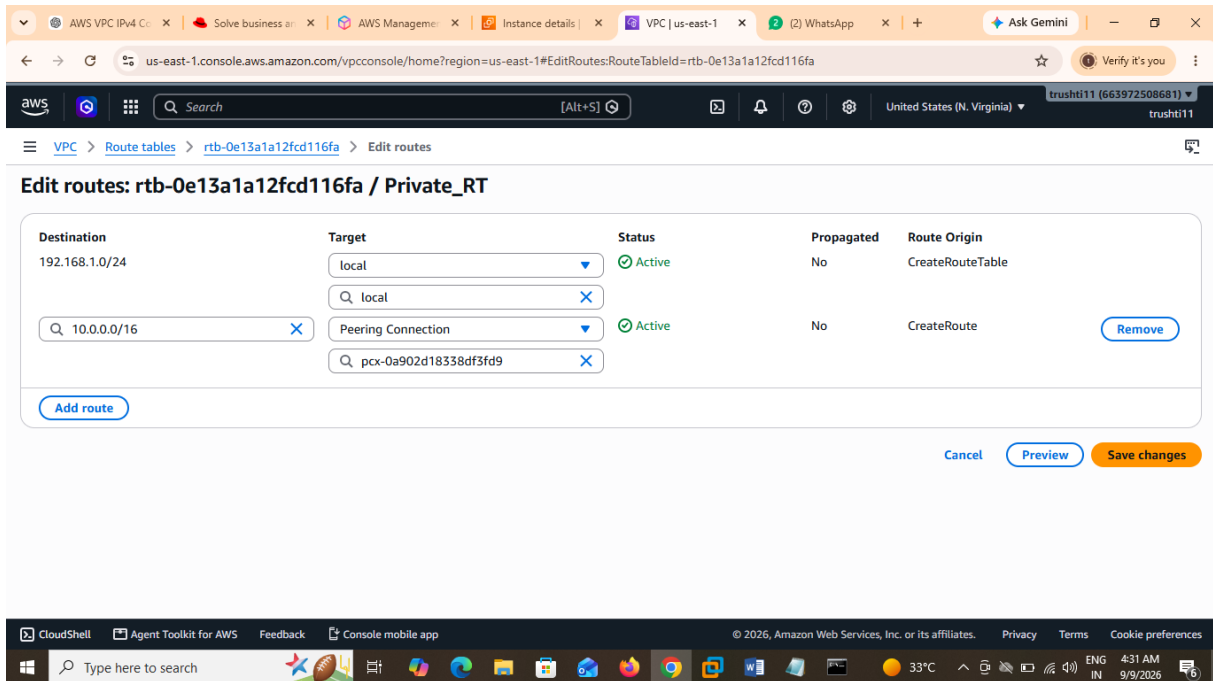


Figure C.3 — AWS Console



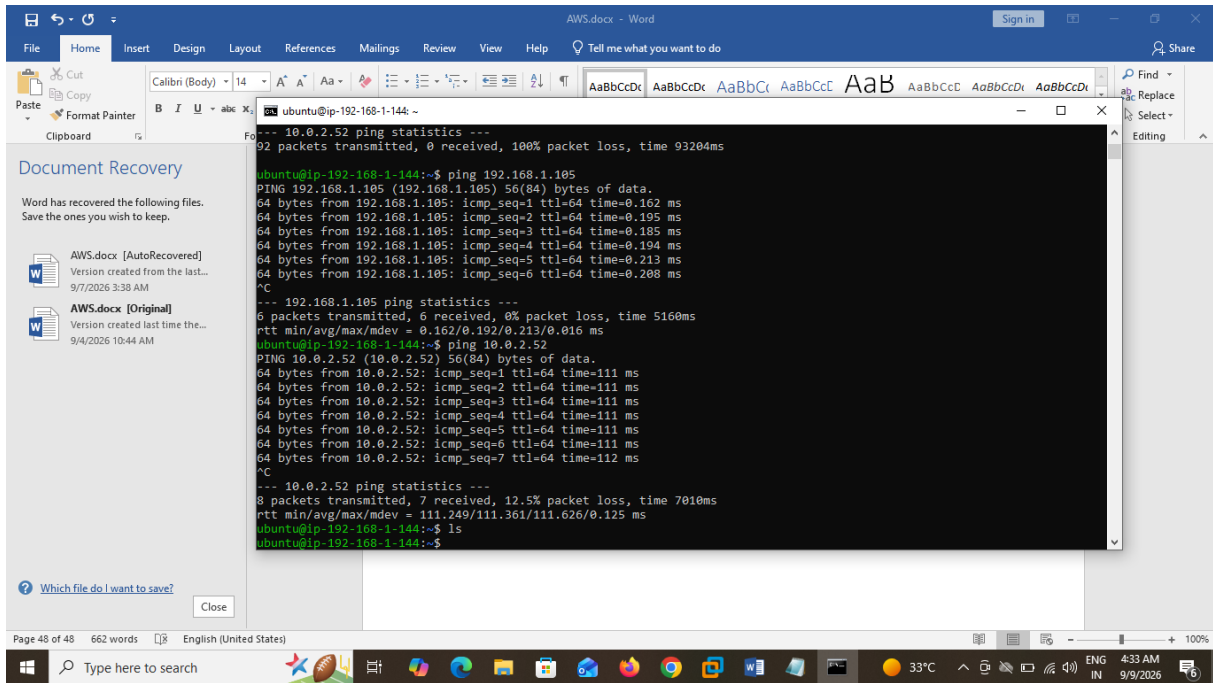


Figure C.6 — AWS Console

5. Comparison Summary

Aspect	Same Region	Different Region	Different Account
Accounts involved	Usually one account	Usually one account	Two different accounts
Regions involved	One region	Two regions	One or two regions
Who accepts the request	Same account, same region console	Same account, but must switch to the acceptor's region	A user in the peer AWS account
Extra info needed to create	Acceptor VPC ID	Acceptor VPC ID + target Region	Peer Account ID (+ Region if different)
Encryption in transit	N/A (stays in one region)	Automatic, physical-layer encryption	Automatic if cross-region; N/A if same region
Typical cost	Standard intra-region data transfer	Inter-region data-transfer charges apply	Depends on region combination
CIDR overlap allowed?	No	No	No
Transitive routing?	No	No	No

6. Troubleshooting Checklist

If instances across a peering connection cannot reach each other after setup, check the following, in order:

- Peering connection status is "Active" (not "Pending Acceptance" or "Failed").
- Route tables on BOTH sides have a route to the peer VPC's CIDR block pointing to the correct Peering Connection ID (pcx-...).
- The correct subnet's route table was edited (a VPC can have multiple route tables, one per subnet or set of subnets).
- Security Groups on the destination instance allow inbound traffic from the peer VPC's CIDR block, on the required port/protocol (and ICMP if testing with ping).

- Network ACLs (if customized) allow the traffic in both directions.
- The two VPCs' CIDR blocks do not overlap.
- For inter-region or cross-account peering, confirm you accepted the request from the correct account/region.
- DNS resolution setting is enabled on the peering connection if private DNS hostnames need to resolve across it.